

Основная волна. Версия от 18.06

Содержание

Задача №1	5
Задача 1.1 (Дальний восток)	5
Задача 1.2 (Центр)	6
Задача №2	7
Задача 2.1 (Дальний восток)	7
Задача 2.2 (Центр)	8
Задача №3	10
Задача 3.1 (Дальний восток)	10
Задача 3.2 (Дальний восток)	11
Задача 3.3 (Центр)	12
Задача №4	14
Задача 4.1 (Дальний восток)	14
Задача 4.2 (Сибирь)	14
Задача 4.3 (Сибирь)	15
Задача №5	16
Задача 5.1 (Дальний восток)	16
Задача 5.2 (Сибирь)	16
Задача 5.3 (Центр)	17
Задача №6	19
Задача 6.1 (Дальний восток)	19
Задача 6.2 (Дальний восток)	20
Задача №7	23
Задача 7.1 (Дальний восток)	23
Задача 7.2 (Дальний восток)	23
Задача 7.3 (Сибирь)	24
Задача 7.4 (Центр)	24
Задача №8	26
Задача 8.1 (Дальний восток)	26
Задача 8.2 (Дальний восток)	30
Задача 8.3 (Сибирь)	31

Задача №9	34
Задача 9.1 (Дальний восток)	34
Задача 9.2 (Сибирь)	35
Задача №10	38
Задача 10.1 (Дальний восток)	38
Задача 10.2 (Дальний восток)	38
Задача 10.3 (Сибирь)	39
Задача 10.4 (Центр)	39
Задача №11	40
Задача 11.1 (Дальний восток)	40
Задача 11.2 (Дальний восток)	40
Задача 11.3 (Центр)	41
Задача №12	42
Задача 12.1 (Дальний восток)	42
Задача 12.2 (Сибирь)	44
Задача №13	49
Задача 13.1 (Дальний восток)	49
Задача 13.2 (Дальний восток)	49
Задача 13.3 (Центр)	51
Задача №14	53
Задача 14.1 (Дальний восток)	53
Задача 14.2 (Дальний восток)	53
Задача 14.3 (Сибирь)	53
Задача 14.4 (Сибирь)	54
Задача 14.5 (Центр)	55
Задача №15	57
Задача 15.1 (Дальний восток)	57
Задача 15.2 (Дальний восток)	58
Задача 15.3 (Дальний восток)	58
Задача 15.4 (Сибирь)	59
Задача 15.5 (Урал)	60
Задача 15.6 (Центр)	61
Задача №16	63
Задача 16.1 (Дальний восток)	63
Задача 16.2 (Дальний восток)	63
Задача 16.3 (Дальний восток)	64
Задача 16.4 (Центр)	65

Задача №17**67**

Задача 17.1 (Дальний восток)	67
Задача 17.2 (Дальний восток)	67
Задача 17.3 (Сибирь)	68
Задача 17.4 (Центр)	68
Задача 17.5 (Центр)	69
Задача 17.6 (Центр)	70

Задача №18**72**

Задача 18.1 (Дальний восток)	72
Задача 18.2 (Дальний восток)	73
Задача 18.3 (Дальний восток)	74

Задача №19-21**76**

Задача 19.1 (Дальний восток)	76
Задача 20.1 (Дальний восток)	77
Задача 21.1 (Дальний восток)	78
Задача 19.2 (Сибирь)	80
Задача 20.2 (Сибирь)	81
Задача 21.2 (Сибирь)	82
Задача 19.3 (Центр)	83
Задача 20.3 (Центр)	85
Задача 21.3 (Центр)	87

Задача №22**90**

Задача 22.1 (Дальний восток)	90
Задача 22.2 (Сибирь)	91
Задача 22.3 (Центр)	92

Задача №23**94**

Задача 23.1 (Дальний восток)	94
Задача 23.2 (Дальний восток)	94
Задача 23.3 (Дальний восток)	95
Задача 23.4 (Сибирь)	96

Задача №24**97**

Задача 24.1 (Дальний восток)	97
Задача 24.2 (Центр)	97

Задача №25**99**

Задача 25.1 (Дальний восток)	99
Задача 25.2 (Сибирь)	100
Задача 25.3 (Урал)	101
Задача 25.4 (Центр)	103
Задача 25.5 (Центр)	105

Задача №26**108**

Задача 26.1 (Дальний восток)	108
Задача 26.2 (Дальний восток)	109

Задача №27**112**

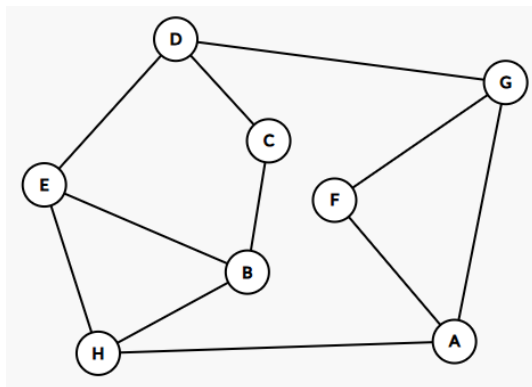
Задача 27.1 (Дальний восток)	112
Задача 27.2 (Дальний восток)	117
Задача 27.3 (Дальний восток)	119
Задача 27.4 (Центр)	125

Задача №1

Задача 1.1 (Дальний восток)

На рисунке схема дорог N -ского района изображена в виде графа, в таблице содержатся сведения о протяжённости каждой из этих дорог (в километрах).

	1	2	3	4	5	6	7	8
1		13	2	5				
2	13			74				39
3	2							1
4	5	74					8	
5						21	30	
6					21		53	3
7				8	30	53		
8		39	1			3		



Так как таблицу и схему рисовали независимо друг от друга, то нумерация населённых пунктов в таблице никак не связана с буквенными обозначениями на графе.

Определите, какова сумма протяжённости дорог из пункта F в пункт G и из пункта B в пункт C . В ответе запишите целое число.

Решение

В самом начале отметим степень каждой вершины (число дорог, исходящих из вершины).

Заметим, что 2 дороги имеют только вершины C и F . Для C соседями являются пункты, не соединённые между собой. Следовательно, вершине C соответствует пункт 3 (так как П1 и П8 не связаны между собой), F — пункт 5. Далее заметим, что C связана с пунктом 1, который не является соседом для пунктов, связанных с F . Отсюда получаем, что вершине B соответствует пункт 1, а D — пункт 8.

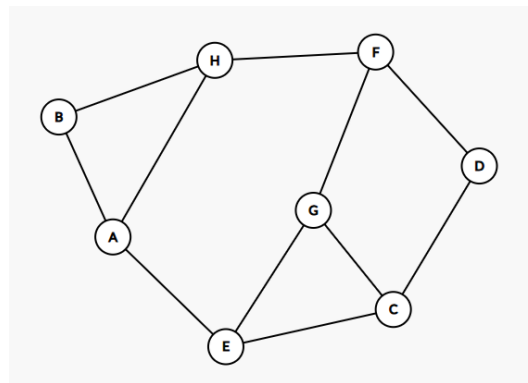
Видим, что пункт 8 связан с пунктом 6, являющимся соседом для вершины F . Значит, вершине G соответствует пункт 6 — все данные, необходимые для подсчёта были найдены. Посчитаем сумму протяжённости дорог из F в G и из B в C : $21 + 2 = 23$.

Ответ: 23

Задача 1.2 (Центр)

На рисунке схема дорог N -ского района изображена в виде графа, в таблице содержатся сведения о протяжённости каждой из этих дорог (в километрах).

	1	2	3	4	5	6	7	8
1					9	15	25	
2			21	24				6
3		21					18	17
4		24					13	
5	9					27		11
6	15				27			
7	25		18	13				
8		6	17		11			



Так как таблицу и схему рисовали независимо друг от друга, то нумерация населённых пунктов в таблице никак не связана с буквенными обозначениями на графе.

Определите, какова сумма протяжённости дорог из пункта B в пункт H и из пункта A в пункт E . В ответе запишите целое число.

Решение

Заметим, что 2 дороги имеют только вершины B и D .

Из пункта G можно попасть только в трёхдорожные пункты F , C и E . Значит, G — это 3, а F , C и E — номера 2, 7, 8 (пока неважно какие).

Пункты C и E соединены между собой, а пункт F с ними не связан. Значит, F — это 7.

Из пункта F попадаем в пункт D — 4 и H — 1. Отсюда пункт B — 6, A — 5 и E — 8.

Теперь можно найти общую сумму протяжённости дорог из пункта B в пункт H и из пункта A в пункт E :

$$15 + 11 = 26$$

Ответ: 26

Задача №2

Задача 2.1 (Дальний восток)

Миша заполнял таблицу истинности логической функции

$$F = (x \rightarrow y) \wedge (y \rightarrow z) \wedge (z \rightarrow w),$$

но успел заполнить лишь фрагмент из трёх различных её строк, даже не указав, какому столбцу таблицы соответствует каждая из переменных.

Ниже представлен частично заполненный фрагмент таблицы истинности функции F , содержащей неповторяющиеся строки.

				F
0			1	1
1		0	1	1
	1		0	1

Определите, какому столбцу истинности функции F соответствует каждая переменная x, y, z, w .

В ответе напишите буквы в том порядке, в котором идут соответствующие им столбцы (сначала буква, соответствующая первому столбцу; затем буква, соответствующая второму столбцу, и т.д.). Буквы в ответе пишите подряд, никаких разделителей между буквами ставить не нужно.

Решение

Напишем программу, которая проверяет все возможные комбинации значений переменных x, y, z, w (0 или 1) и выводит только те наборы, при которых заданное логическое выражение истинно.

```
for x in range(2):
    for y in range(2):
        for z in range(2):
            for w in range(2):
                if (x <= y) and (y <= z) and (z <= w):
                    print(x, y, z, w)
```

После запуска программы получаем таблицу:

x	y	z	w
0	0	0	1
0	0	1	1
0	1	1	1

Заполняем таблицу: второй столбец – единственный, в котором могут быть все единицы, поэтому к нему относится переменная w .

Заполнив второй столбец единицами, теперь мы полностью знаем вторую строчку. В ней есть единственный нолик, ему соответствует переменная x .

У нас нет повторяющихся строк, поэтому в первой строке остаётся единственный вариант для x – нолик. Теперь мы полностью знаем первую строку, находим её в выводе программы. Ей соответствует строка 0 0 1 1. Следственно, первым столбцом будет y , последним – z .

Ответ: ywxz

Задача 2.2 (Центр)

Миша заполнял таблицу истинности логической функции

$$F = (x \wedge \neg z \wedge \neg w) \vee (x \wedge \neg z \wedge y$$

но успел заполнить лишь фрагмент из трёх различных её строк, даже не указав, какому столбцу таблицы соответствует каждая из переменных.

Ниже представлен частично заполненный фрагмент таблицы истинности функции F , содержащей неповторяющиеся строки.

				F
1				1
0		1		1
		0	0	1

Определите, какому столбцу истинности функции F соответствует каждая переменная x, y, z, w .

В ответе напишите буквы в том порядке, в котором идут соответствующие им столбцы (сначала буква, соответствующая первому столбцу; затем буква, соответствующая второму столбцу, и т.д.). Буквы в ответе пишите подряд, никаких разделителей между буквами ставить не нужно.

Решение

Напишем программу, которая проверяет все возможные комбинации значений переменных x, y, z, w (0 или 1) и выводит только те наборы, при которых заданное логическое выражение истинно.

```
for x in range(2):
    for y in range(2):
        for z in range(2):
            for w in range(2):
                if (x and (not z) and (not w)) or (x and (not z) and y):
                    print(x, y, z, w)
```


После запуска программы получаем таблицу:

x	y	z	w
1	0	0	0
1	1	0	0
1	1	0	1

Заполняем таблицу: второй столбец - единственный, в котором могут быть все единицы, поэтому к нему относится переменная x .

Тогда четвёртый столбец - единственный, в котором могут быть все нули, следовательно, его занимает переменная z .

В строке, где две единицы, одну занимает x , а другую - y . Тогда третий столбец - y . Остаётся только первый столбец и это w .

Ответ: wxyz

Задача №3

Задача 3.1 (Дальний восток)

В файле приведён фрагмент базы данных «Поставка товаров» о поставках бытовой химии и средств гигиены в магазины районов города. База данных состоит из трёх таблиц.

Таблица «Движение товаров» содержит записи о поставках товаров в магазины в течение сентября 2023 г., а также информацию о проданных товарах. Поле *Тип операции* содержит значение *Поступление* или *Продажа*, а в соответствующее поле *Количество упаковок, шт.* внесена информация о том, сколько упаковок товара поступило в магазин или было продано в течение дня. Заголовок таблицы имеет следующий вид.

ID операции	Дата	ID магазина	Артикул	Количество упаковок, шт.	Тип операции
-------------	------	-------------	---------	--------------------------	--------------

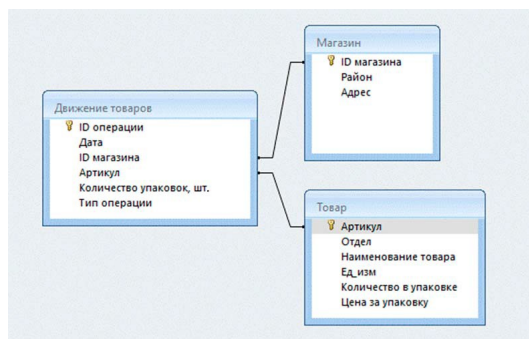
Таблица «Товар» содержит информацию об основных характеристиках каждого товара. Заголовок таблицы имеет следующий вид.

Артикул	Отдел	Наименование товара	Ед. изм.	Количество в упаковке	Цена за упаковку
---------	-------	---------------------	----------	-----------------------	------------------

Таблица «Магазин» содержит информацию о местонахождении магазинов. Заголовок таблицы имеет следующий вид.

ID магазина	Район	Адрес
-------------	-------	-------

На рисунке приведена схема указанной базы данных.



Используя информацию из приведённой базы данных, определите общую стоимость (в руб.) упаковок ультрапастеризованного молока (всех видов), полученных магазинами Нагорного района за период со 2 по 9 октября включительно. В ответе запишите только число.

Решение

В таблице «Движение товаров» создаем четыре новых столбца: «Вид товара», «Район», «Цена за упаковку», «Стоимость». Для получения вида товара в ячейке G2 запишем и протянем вниз формулу:

= ВПР(D2;Товар!A:F;3;0)

Для нахождения района магазина в ячейке H2 запишем и протянем вниз формулу:

$$= \text{ВПР}(C2; \text{Магазин!A:C}; 2; 0)$$

Для переноса цены за упаковку в ячейке I2 запишем и протянем вниз формулу:

$$= \text{ВПР}(D2; \text{Товар!A:F}; 6; 0)$$

Остаётся вычислить стоимость всей партии, умножив количество упаковок на цену за штуку. Для этого в ячейке J2 запишем и протянем вниз формулу:

$$= E2 * I2$$

Теперь воспользуемся фильтром: отфильтруем таблицу по дате (со 2 по 9 октября включительно), району (Нагорный), виду товара (ультрапастеризованное молоко) и по типу операции (поступление). Выделяем последний столбец и получаем искомую сумму.

Ответ:

Задача 3.2 (Дальний восток)

В файле приведён фрагмент базы данных «Поставка товаров» о поставках бытовой химии и средств гигиены в магазины районов города. База данных состоит из трёх таблиц.

Таблица «Движение товаров» содержит записи о поставках товаров в магазины в течение сентября 2023 г., а также информацию о проданных товарах. Поле *Тип операции* содержит значение *Поступление* или *Продажа*, а в соответствующее поле *Количество упаковок, шт.* внесена информация о том, сколько упаковок товара поступило в магазин или было продано в течение дня. Заголовок таблицы имеет следующий вид.

ID операции	Дата	ID магазина	Артикул	Количество упаковок, шт.	Тип операции
-------------	------	-------------	---------	--------------------------	--------------

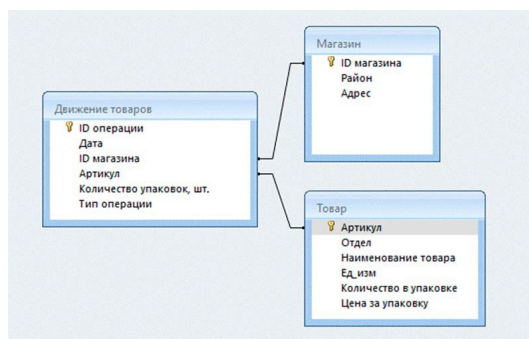
Таблица «Товар» содержит информацию об основных характеристиках каждого товара. Заголовок таблицы имеет следующий вид.

Артикул	Отдел	Наименование товара	Ед. изм.	Количество в упаковке	Цена за упаковку
---------	-------	---------------------	----------	-----------------------	------------------

Таблица «Магазин» содержит информацию о местонахождении магазинов. Заголовок таблицы имеет следующий вид.

ID магазина	Район	Адрес
-------------	-------	-------

На рисунке приведена схема указанной базы данных.



Используя информацию из базы данных «Поставка товаров», определите, сколько килограммов зефира поступило в магазины Первомайского района за период с 1 по 10 июня включительно. В ответе укажите только целое число.

Решение

В таблице «Движение товаров» создаем четыре новых столбца: «Вид товара», «Район», «КГ на штуку», «Общая масса».

Для получения вида товара в ячейке G2 запишем и протянем вниз формулу:

=ВПР(D2;Товар!A:F;3;0)

Для нахождения района магазина в ячейке H2 запишем и протянем вниз формулу:

=ВПР(C2;Магазин!A:C;2;0)

Для переноса веса одного товара в ячейке I2 запишем и протянем вниз формулу:

=ВПР(D2;Товар!A:F;5;0)

Остаётся вычислить общую массу всей партии, умножив количество упаковок на их вес. Для этого в ячейке J2 запишем и протянем вниз формулу:

=E2*I2

Теперь воспользуемся фильтром: отфильтруем таблицу по дате (с 1 по 10 июня включительно), району (Первомайский), виду товара (зефир) и по типу операции (поступление). Выделяем последний столбец и получаем искомое число килограммов.

Задача 3.3 (Центр)

В файле приведён фрагмент базы данных «Поставка товаров» о поставках бытовой химии и средств гигиены в магазины районов города. База данных состоит из трёх таблиц.

Таблица «Движение товаров» содержит записи о поставках товаров в магазины в течение сентября 2023 г., а также информацию о проданных товарах. Поле *Тип операции* содержит значение *Поступление* или *Продажа*, а в соответствующее поле *Количество упаковок, шт.* внесена информация о том, сколько упаковок товара поступило в магазин или было продано в течение дня. Заголовок таблицы имеет следующий вид.

ID операции	Дата	ID магазина	Артикул	Количество упаковок, шт.	Тип операции
-------------	------	-------------	---------	--------------------------	--------------

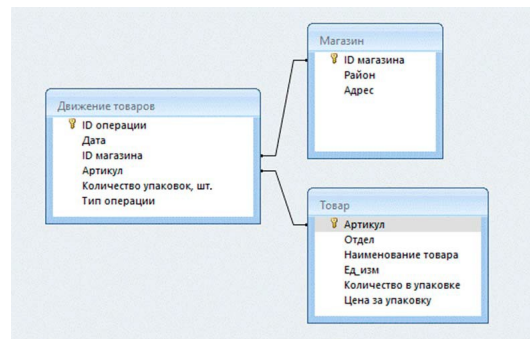
Таблица «Товар» содержит информацию об основных характеристиках каждого товара. Заголовок таблицы имеет следующий вид.

Артикул	Отдел	Наименование товара	Ед. изм.	Количество в упаковке	Цена за упаковку
---------	-------	---------------------	----------	-----------------------	------------------

Таблица «Магазин» содержит информацию о местонахождении магазинов. Заголовок таблицы имеет следующий вид.

ID магазина	Район	Адрес
-------------	-------	-------

На рисунке приведена схема указанной базы данных.



Используя информацию из базы данных «Поставка товаров», определите, сколько упаковок шоколада массой 100г было продано в магазинах Центрального района с 7 по 15 июня.

Решение

Откроем таблицу в редакторе таблиц и в таблице Движение товаров в ячейке G2 запишем следующую формулу:

$$= \text{ВПР}(C2;\$ \text{Магазин}.A:C;2;0)$$

Затем перенесем в таблицу наименование товара и количество грамм с помощью следующих формул в ячейках H2 и I2:

$$= \text{ВПР}(D2;\$ \text{Товар}.A:F;3;0)$$

$$= \text{ВПР}(D2;\$ \text{Товар}.A:F;5;0)$$

Теперь отфильтруем все данные, уберем ненужные данные, оставим только Центральный район, выставим только операции с типом Продажа, оставим из товаров только Шоколад молочный, Шоколад с изюмом, Шоколад с орехом и Шоколад тёмный. После применения фильтра перенесем данные на новый лист. В ячейке J2 запишем следующую формулу

$$= \text{ЕСЛИ}(I2=100;E2;0)$$

Теперь в J2 хранятся только те количества упаковок, которые весили по 100 грамм. Сумма столбца J и будет ответом.

ID операции	Дата	ID магазина	Артикул	Тип операции	Район	Наименование товара	Ед.изм	Количество в упаковке	Цена за упаковку
1110	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1112	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	80	96
1114	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1116	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1118	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1120	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1122	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1124	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1126	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1128	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1130	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1132	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1134	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1136	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1138	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1140	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1142	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1144	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1146	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1148	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1150	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1152	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1154	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1156	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1158	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1160	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1162	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1164	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1166	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1168	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1170	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1172	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1174	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1176	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1178	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1180	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1182	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1184	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1186	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1188	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1190	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1192	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1194	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1196	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1198	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120
1200	07.06.2022	M1	80	120Продажа	Центральный	Шоколад с изюмом	шт.	100	120

Ответ: 5324



Задача №4

Задача 4.1 (Дальний восток)

Для кодирования последовательности, состоящей из букв К, О, Л, Б, решили использовать неравномерный двоичный код, удовлетворяющий условию Фано. Букве К соответствует двоичный код 0, букве О – код 10.

Какова наименьшая суммарная длина кодовых слов для всех букв в слове КОЛОБОК?

Решение

Код для буквы К по условию равен 0. Следовательно, все остальные коды обязаны начинаться с единицы. Код для буквы О равен 10, поэтому будем рассматривать ветку, начинающуюся с 11.

Нам необходимо закодировать 2 дополнительные буквы — Л и Б, поэтому «раздвоим» единственную оставшуюся ветку и получим коды: 110 и 111. Поскольку других вариантов нет, они являются минимальными по длине, благодаря чему суммарная длина кодовых слов для всех букв в слове КОЛОБОК будет наименьшей.

Посчитаем, сколько раз каждая буква встречается в слове КОЛОБОК: К — 2 раза, О — 3 раза, Л — 1 раз, Б — 1 раз. Остаётся найти общую длину: $2 * 1 + 3 * 2 + 1 * 3 + 1 * 3 = 14$

Ответ: 14

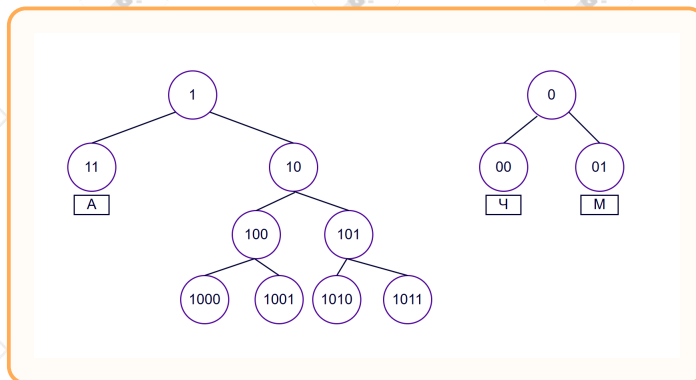
Задача 4.2 (Сибирь)

Для кодирования последовательности, состоящей из букв русского алфавита, решили использовать неравномерный двоичный код, удовлетворяющий условию Фано. Букве Ч соответствует двоичный код 00, букве М – код 01, букве А – код 11.

Какова наименьшая суммарная длина кодовых слов для всех букв в слове КОТ?

Решение

Изобразим дерево Фано и подпишем все известные коды:



По условию нам необходимо закодировать буквы слова КОТ и оставшиеся буквы русского алфавита. Значит, нам понадобится 4 свободных кода, чтобы один из них продлить для оставшихся букв.

Разбиение на коды можно сделать как на дереве: 1000, 1001, 1010 и 1011. Тогда суммарная длина кодовых слов для букв КОТ будет равна:

$$4 \cdot 3 = 12$$

Также можно попробовать одной букве дать код 100, а код 101 продлить до 1010, 10110 и 10111. В таком случае суммарная длина не изменится и будет также равна:

$$3 + 4 + 5 = 12$$

Ответ: 12

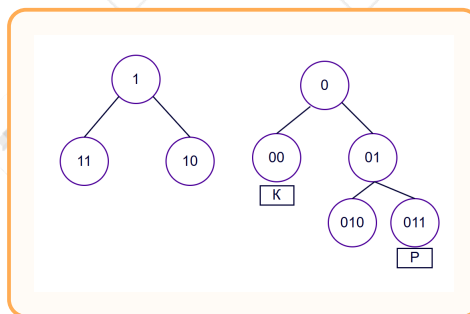
Задача 4.3 (Сибирь)

Для кодирования последовательности, состоящей из букв К, О, Л, Р, решили использовать неравномерный двоичный код, удовлетворяющий условию Фано. Букве К соответствует двоичный код 00, букве Р – код 011.

Какова наименьшая суммарная длина кодовых слов для всех букв в слове КОЛОКОЛ?

Решение

Изобразим дерево Фано и подпишем известные коды:



Нам необходимо найти наименьшую суммарную длину кодовых слов для всех букв слова КОЛОКОЛ. Код для буквы К известен и имеет длину 2. Определим коды для букв О и Л.

Буква О встречается 3 раза, буква Л – два. Рассмотрим два возможных варианта:

1) Буква О имеет код 1, буква Л – 010. Тогда общая длина слова КОЛОКОЛ равна:

$$2 \cdot 2 + 1 \cdot 3 + 3 \cdot 2 = 13$$

2) Буква О имеет код 11, буква Л – 10. Тогда общая длина слова КОЛОКОЛ равна:

$$2 \cdot 2 + 2 \cdot 3 + 2 \cdot 2 = 14$$

Таким образом, выгоднее всего получить слово с общей длиной 13.

Ответ: 13

Задача №5

Задача 5.1 (Дальний восток)

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом:

1. Строится двоичная запись числа N .
2. К этой записи дописываются справа ещё два разряда по следующему правилу:
 - складываются все цифры двоичной записи числа N , и остаток от деления суммы на 2 дописывается в конец числа (справа);
 - над полученной записью производятся те же действия – справа дописывается остаток от деления суммы её цифр на 2.

Полученная таким образом запись является двоичной записью искомого числа R . Укажите наименьшее число N , для которого результат работы алгоритма больше числа 253.

В ответе запишите это число в десятичной системе счисления.

Решение

```
for n in range(1, 1000):  
    s = bin(n)[2:]  
    s += str(s.count('1') % 2)  
    s += str(s.count('1') % 2)  
    r = int(s, 2)  
    if r > 253:  
        print(n)  
        break
```

Ответ: 64

Задача 5.2 (Сибирь)

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом:

1. Строится двоичная запись числа N .
2. К этой записи дописываются справа ещё несколько разрядов по следующим правилам:
 - если число N четное, то справа и слева к этой записи дописывается 11;
 - если число N нечетное, то справа к этой записи дописывается 00, а слева – 11

Полученная таким образом запись является двоичной записью искомого числа R . Укажите наименьшее число R , большее 105, которое могло получиться в результате работы алгоритма.

В ответе запишите число в десятичной системе счисления.

Решение

Для решения необходимо перебрать возможные значения N , преобразовав их по алгоритму из условия:

К двоичной записи числа N приписываются дополнительные разряды в зависимости от чётности проверяемого числа.

Изменённая запись переводится в десятичную систему счисления, после чего добавляется во множество подходящих значений при $R > 105$.

В конце наименьший элемент данного множества выводится на экран в качестве ответа.

```
res = set() # Создаём множество, в котором будут храниться R > 105
for n in range(1, 10000): # Перебираем значения числа N
    r = bin(n)[2:] # Переводим число N в двоичную систему
    if n % 2 == 0: # Если число N - чётное,
        r = "11" + r + "11" # приписываем слева и справа "11"
    else:
        r = "11" + r + "00" # иначе приписываем справа "00", слева "11"
    r = int(r, 2) # Переводим полученную запись в десятичную систему
    if r > 105: # Если найденное число R > 105:
        res.add(r) # добавляем его во множество
print(min(res)) # Выводим в качестве ответа минимальное значение R
```

Ответ: 115

Задача 5.3 (Центр)

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом:

1. Строится двоичная запись числа N .
2. К этой записи дописываются справа ещё несколько разрядов по следующим правилам:
 - если число N чётное, то справа к этой записи дописывается 1 и слева также дописывается 1;
 - если число N нечётное, то справа к этой записи дописывается 10, а слева – 1.

Полученная таким образом запись является двоичной записью искомого числа R . Укажите наибольшее число R , не превышающее 65, которое могло получиться в результате работы алгоритма.

В ответе запишите число в десятичной системе счисления.

Решение

Для решения необходимо перебрать возможные значения N , преобразовав их по алгоритму из условия:

К двоичной записи числа N приписываются дополнительные разряды в зависимости от чётности проверяемого числа.

Изменённая запись переводится в десятичную систему счисления, после чего добавляется во множество подходящих значений при $R \leq 65$.

В конце наибольший элемент данного множества выводится на экран в качестве ответа.

```
res = set() # Создаём множество, в котором будут храниться R <= 65
for n in range(1, 10000): # Перебираем значения числа N
    r = bin(n)[2:] # Переводим число N в двоичную систему
    if n % 2 == 0: # Если число N - чётное,
        r = "1" + r + "1" # приписываем слева и справа "1"
    else:
        r = "1" + r + "10" # иначе приписываем справа "10", слева "1"
    r = int(r, 2) # Переводим полученную запись в десятичную систему
    if r <= 65: # Если найденное число R <= 65:
        res.add(r) # добавляем его во множество
print(max(res)) # Выводим в качестве ответа максимальное значение R
```

Ответ: 62

Задача №6

Задача 6.1 (Дальний восток)

Исполнитель Черепаха передвигается по плоскости и оставляет след в виде линии. Черепаха может выполнять три команды: **Вперёд n** (n – число), **Направо m** (m – число) и **Налево m** (m – число). По команде **Вперёд n** Черепаха перемещается вперёд на n единиц. По команде **Направо m** Черепаха поворачивается на месте на m градусов по часовой стрелке, при этом соответственно меняется направление дальнейшего движения. По команде **Налево m** Черепаха поворачивается на месте на m градусов против часовой стрелки, при этом соответственно меняется направление дальнейшего движения.

В начальный момент Черепаха находится в начале координат и направлена вверх (вдоль положительного направления оси ординат).

Запись **Повтори k [Команда1 Команда2 ... КомандаS]** означает, что заданная последовательность из S команд повторится k раз.

Черепаха выполнила следующую программу:

Направо 45 Повтори 7 [Вперёд 5 Направо 45 Вперёд 10 Направо 135].

Определите, сколько точек с целочисленными координатами будут находиться внутри области, которая ограничена линией, заданной алгоритмом. Точки на линии учитывать не следует.

Решение

```
from turtle import * # Модуль для работы с Черепахой

tracer(50) # Ускорение анимации движения черепахи
scale = 20 # Масштаб для увеличения видимости рисунка
# Начальная ориентация Черепахи
# (по умолчанию - вдоль оси X, поворачиваем на 90° влево)
left(90)

# Основной алгоритм Черепахи
right(45) # Направо 45

for _ in range(7): # Повторить 7 раз
    forward(5 * scale) # Вперёд 5 (умножаем на scale для масштабирования)
    right(45) # Направо 45
    forward(10 * scale) # Вперёд 10
    right(135) # Направо 135

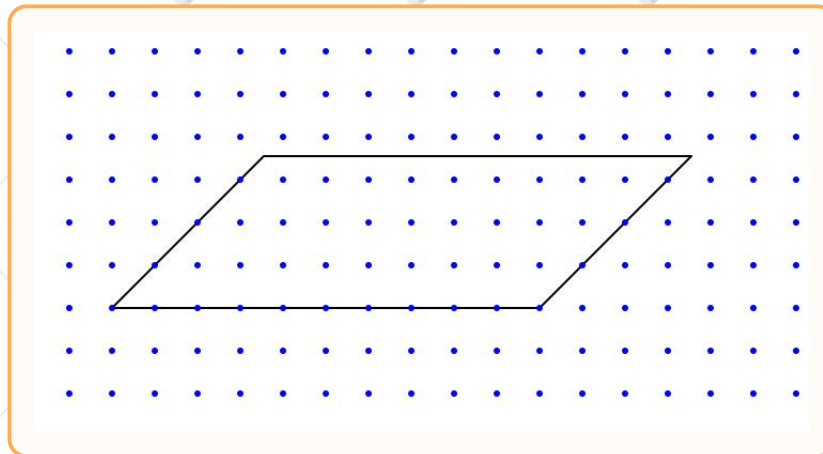
# Расставляем точки с целыми координатами
up() # Поднимаем хвост чтобы не рисовать лишние линии
for x in range(-30, 30): # Перебор абсцисс точек
    for y in range(-30, 30): # Перебор ординат точек
```



```
goto(x * scale, y * scale) # Перемещение к точке (x, y)
dot(3, "blue") # Ставим точку синего цвета размера 3

update() # Обновление экрана с конечным рисунком от черепахи
done() # Завершение работы (окно остаётся открытым)
```

Остается посчитать количество точек с целочисленными координатами, находящихся внутри фигуры, не считая точки на линии.



Ответ: 27

Задача 6.2 (Дальний восток)

Исполнитель Черепаха передвигается по плоскости и оставляет след в виде линии. У исполнителя существует 6 команд: Поднять хвост, означающая переход к перемещению без рисования; Опустить хвост, означающая переход в режим рисования; Вперёд n (где n – целое число), вызывающая передвижение Черепахи на n единиц в том направлении, куда указывает её голова; Назад n (где n – целое число), вызывающая передвижение в противоположном голове направлении; Направо m (где m – целое число), вызывающая изменение направления движения на m градусов по часовой стрелке, Налево m (где m – целое число), вызывающая изменение направления движения на m градусов против часовой стрелки.

В начальный момент Черепаха находится в начале координат и направлена вверх (вдоль положительного направления оси ординат).

Запись **Повтори k [Команда1 Команда2 ... Команда S]** означает, что заданная последовательность из S команд повторится k раз.

Черепаха выполнила следующую программу:

Повтори 2 [Вперёд 14 Налево 270 Назад 12 Направо 90]

Поднять хвост

Вперёд 9 Направо 90 Назад 7 Налево 90

Опустить хвост

Повтори 2 [Вперёд 13 Направо 90 Вперёд 6 Направо 90]

Определите, сколько точек с целочисленными координатами находятся внутри пересечения фигур, ограниченного заданными алгоритмом линиями, включая точки на линиях.

Решение

```
from turtle import * # Модуль для работы с Черепахой

tracer(0) # Отключаем анимацию
m = 10 # Коэффициент масштабирования
left(90) # поворачиваем вдоль положительного направления оси ординат

for i in range(2): # Повторяем 2 раза
    forward(14*m) # Вперед на 14 с учетом масштабирования
    left(270) # Поворот налево на 270
    backward(12*m) # Назад на 12
    right(90) # Поворот направо на 90

pu() # Поднимаем хвост
forward(9*m) # Вперед на 9
right(90) # Поворот направо на 90
backward(7*m) # Назад на 7
left(90) # Поворот налево на 90

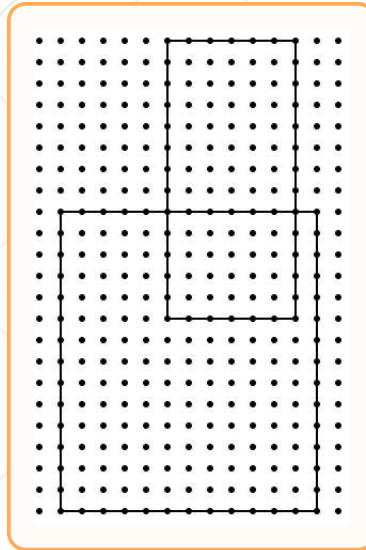
pd() # Опускаем хвост

for i in range(2): # Повторяем 2 раза
    forward(13*m) # Вперед на 13
    right(90) # Поворот направо 90
    forward(6*m) # Вперед на 6
    right(90) # Поворот направо на 90

pu() # Поднимаем хвост
# Расставляем точки с целочисленными координатами
for x in range(-50, 50): # Перебираем абсциссы точек
    for y in range(-50, 50): # Перебираем ординаты точек
        goto(x*m, y*m) # Перемещаемся к точке с координатами (x, y)
        dot(3) # Ставим точку размера 3

done() # Завершаем отрисовку, оставляя окно открытым
```

Получаем следующую фигуру:



Пересечение – это точки, находящиеся одновременно в обеих фигурах. Количество точек в пересечении фигур с учетом точек на линиях:

$$7 \cdot 6 = 42$$

Ответ: 42

Задача №7

Задача 7.1 (Дальний восток)

Лена записывает голосовое сообщение для своей подруги. Перед отправкой сообщение оцифровывается в формате стерео с частотой дискретизации 28000 Гц и глубиной кодирования 8 бит.

Определите **наименьшее целое количество** Кбайт, необходимое для сохранения сообщения в памяти (без учёта заголовка), если его длительность – 2 минуты 20 секунд.

В ответе укажите только число.

Решение

Для нахождения объема аудиофайла используется формула: $\text{Объем} = \text{Количество каналов} * \text{Частота дискретизации} * \text{Глубина кодирования} * \text{Время}$

Определим параметры записи по условию: 1. Формат стерео означает, что запись двухканальная (количество каналов = 2). 2. Частота дискретизации = 28000 Гц. 3. Глубина кодирования = 8 бит (это ровно 1 байт). 4. Длительность записи = 2 минуты 20 секунд. Переведем время в секунды: $2 * 60 + 20 = 140$ секунд.

Найдем объем аудиозаписи в байтах: $\text{Объем} = 2 * 28000 * 1 \text{ байт} * 140 = 56000 * 140 = 7\,840\,000$ байт.

Переведем полученный объем в килобайты, разделив значение на 1024: $7\,840\,000 / 1024 = 7656.25$ Кбайт.

По условию требуется найти наименьшее целое количество Кбайт, необходимое для сохранения файла в памяти. Если округлить результат в меньшую сторону до 7656 Кбайт, то оставшаяся часть файла (0.25 Кбайт) не поместится и сообщение сохранится не полностью. Для сохранения всего файла округляем значение в большую сторону до ближайшего целого: 7656.25 Кбайт округляем до 7657 Кбайт.

Ответ: 7657

Задача 7.2 (Дальний восток)

Лена записывает голосовое сообщение для своей подруги. Перед отправкой сообщение оцифровывается в формате стерео с частотой дискретизации 32 000 Гц и глубиной кодирования 16 бит.

Определите **наименьшее целое количество** Кбайт, необходимое для сохранения сообщения в памяти (без учёта заголовка), если его длительность – 2 минуты 27 секунд.

В ответе укажите только число.

Решение

Для нахождения объема аудиофайла используется формула:

$\text{Объем} = \text{Количество каналов} * \text{Частота дискретизации} * \text{Глубина кодирования} * \text{Время}$

Определим параметры записи по условию:

Формат стерео означает, что запись двухканальная (количество каналов = 2).

Частота дискретизации = 32000 Гц.

Глубина кодирования (разрешение) = 16 бит (это равно 2 байта).

Длительность записи = 2 минуты 27 секунд. Переведем время в секунды: $2 \cdot 60 + 27 = 147$ секунд.

Найдем объем аудиозаписи в байтах:

$$V = 2 \cdot 32000 \cdot 2 \text{ байта} \cdot 147 = 128000 \cdot 147 = 18816000 \text{ байт.}$$

Переведем полученный объем в килобайты, разделив значение на 1024:

$$\frac{18816000}{1024} = 18375 \text{ Кбайт.}$$

Поскольку полученное значение 18375 Кбайт является целым числом, дополнительное округление не требуется.

Ответ: 18375

Задача 7.3 (Сибирь)

Лена записывает голосовое сообщение для своей подруги. Перед отправкой сообщение оцифровывается в формате стерео с частотой дискретизации 16 000 Гц и глубиной кодирования 8 бит.

Определите **наименьшее целое количество** Кбайт, необходимое для сохранения сообщения в памяти (без учёта заголовка), если его длительность – 4 минуты 29 секунд.

В ответе укажите только число.

Решение

Объём звука можно найти по формуле: количество каналов * частота (в Гц) * длительность записи (в секундах) * глубина кодирования. Также важно помнить типы форматов, стерео содержит 2 канала. Получаем $2 \cdot 16000 \cdot 269 \cdot 8 \text{ бит} = 68\,864\,000 \text{ бит}$. Ответ просится записать в Кбайтах, переводим, поделив на 8 и 1024: $\frac{68\,864\,000}{8 \cdot 1024} = 8406,25 \text{ Кбайт}$. Округляем вверх, чтобы весь объём записи влез.

Ответ: 8407

Задача 7.4 (Центр)

Лена записывает голосовое сообщение для своей подруги. Перед отправкой сообщение оцифровывается в формате стерео с частотой дискретизации 20 000 Гц и глубиной кодирования 24 бит.

Определите **наименьшее целое количество** Кбайт, необходимое для сохранения сообщения в памяти (без учёта заголовка), если его длительность – 2 минуты 18 секунд.

В ответе укажите только число.

Решение



Объём звука можно найти по формуле: количество каналов · частота (в Гц) · длительность записи (в секундах) · глубина кодирования. Также важно помнить типы форматов, стерео содержит 2 канала. Получаем $2 \cdot 20000 \cdot 138 \cdot 24 \text{ бит} = 132\,480\,000 \text{ бит}$.

Ответ просится записать в Кбайтах, переводим, поделив на 8 и 1024: $\frac{132\,480\,000}{8 \cdot 1024} = 16171,875$ Кбайт. Округляем вверх, чтобы весь объём записи влез.

Ответ: 16172

Задача №8

Задача 8.1 (Дальний восток)

Определите количество пятизначных чисел, записанных в девятеричной системе счисления, в записи которых ровно две цифры 3, и при этом никакая нечётная цифра не стоит рядом с цифрой 2.

Решение

Решение руками:

Алфавит девятеричной системы счисления: 0, 1, 2, 3, 4, 5, 6, 7, 8. Из них чётные цифры – 0, 2, 4, 6, 8, а нечётные – 1, 3, 5, 7.

Рассмотрим случаи расположения двух троек.

Случай 1: Тройки на позициях 1 и 2

3 3 _ _ _

Варианты с двойкой:

1.1) 2 не используется: на позициях 3, 4, 5 может стоять любая цифра из $\{0, 1, 4, 5, 6, 7, 8\}$.

Итого $7^3 = 343$ варианта.

1.2) 2 на позиции 4: 3 3 _ 2 _.

Позиция 3: $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 5: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого $4 \cdot 4 = 16$ вариантов.

1.3) 2 на позиции 5: 3 3 _ _ 2.

Позиция 4: $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 3: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $7 \cdot 4 = 28$ вариантов.

1.4) 2 на позициях 4 и 5 одновременно: 3 3 _ 2 2.

Позиция 3: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого 4 варианта.

Итого в случае 1: $343 + 16 + 28 + 4 = 391$ вариант.

Случай 2: Тройки на позициях 1 и 3

3 _ 3 _ _

2.1) 2 не используется: на позициях 2, 4, 5 любая цифра из $\{0, 1, 4, 5, 6, 7, 8\}$.

Итого $7^3 = 343$ варианта.

2.2) 2 на позиции 5: 3 _ 3 _ 2.

Позиция 4 (рядом с 2): $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 2: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $7 \cdot 4 = 28$ вариантов.

Итого в случае 2: $343 + 28 = 371$ вариант.

Случай 3: Тройки на позициях 1 и 4
$$\begin{array}{ccccccc} & & 3 & & & 3 & \\ & & _ & _ & _ & _ & \end{array}$$

3.1) 2 не используется: на позициях 2, 3, 5 любая цифра из $\{0, 1, 4, 5, 6, 7, 8\}$.

Итого $7^3 = 343$ варианта.

Итого в случае 3: 343 варианта.

Случай 4: Тройки на позициях 1 и 5
$$\begin{array}{ccccccc} & & 3 & & & & 3 \\ & & _ & _ & _ & _ & \end{array}$$

4.1) 2 не используется: на позициях 2, 3, 4 любая цифра из $\{0, 1, 4, 5, 6, 7, 8\}$.

Итого $7^3 = 343$ варианта.

4.2) 2 на позиции 3: $3 _ 2 _ 3$.

Позиция 2: $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 4: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого $4 \cdot 4 = 16$ вариантов.

Итого в случае 4: $343 + 16 = 359$ вариантов.

Случай 5: Тройки на позициях 2 и 3
$$\begin{array}{ccccccc} & & & 3 & 3 & & \\ & & _ & _ & _ & _ & \end{array}$$

5.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;

позиции 4, 5: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

5.2) 2 на позиции 5: $_ 3 3 _ 2$.

Позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;

позиция 4: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого $6 \cdot 4 = 24$ варианта.

Итого в случае 5: $294 + 24 = 318$ вариантов.

Случай 6: Тройки на позициях 2 и 4
$$\begin{array}{ccccccc} & & & 3 & & 3 & \\ & & _ & _ & _ & _ & \end{array}$$

6.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;

позиции 3, 5: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

Итого в случае 6: 294 варианта.

Случай 7: Тройки на позициях 2 и 5
$$\begin{array}{ccccccc} & & & 3 & & & 3 \\ & & _ & _ & _ & _ & \end{array}$$

7.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;

позиции 3, 4: $\{1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

Итого в случае 7: 294 варианта.

Случай 8: Тройки на позициях 3 и 4

_ _ 3 3 _

8.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;
позиции 2, 5: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

8.2) 2 на позиции 1: 2 _ 3 3 _. Позиция 2: $\{0, 4, 6, 8\}$ – 4 варианта;
позиция 5 не меняется – 7 вариантов.

Итого $4 \cdot 7 = 28$ вариантов.

Итого в случае 8: $294 + 28 = 322$ варианта.

Случай 9: Тройки на позициях 3 и 5

_ _ 3 _ 3

9.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;
позиции 2, 4: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

9.2) 2 на позиции 1: 2 _ 3 _ 3.

Позиция 2: $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 4: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $4 \cdot 7 = 28$ вариантов.

Итого в случае 9: $294 + 28 = 322$ варианта.

Случай 10: Тройки на позициях 4 и 5

_ _ _ 3 3

10.1) 2 не используется: позиция 1: $\{1, 4, 5, 6, 7, 8\}$ – 6 вариантов;
позиции 2, 3: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $6 \cdot 7 \cdot 7 = 294$ варианта.

10.2) 2 на позиции 1: 2 _ _ 3 3.

Позиция 2: $\{0, 4, 6, 8\}$ – 4 варианта;

позиция 3: $\{0, 1, 4, 5, 6, 7, 8\}$ – 7 вариантов.

Итого $4 \cdot 7 = 28$ вариантов.

10.3) 2 на позиции 2: _ 2 _ 3 3.

Позиция 1: $\{4, 6, 8\}$ – 3 варианта;

позиция 3: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого $3 \cdot 4 = 12$ вариантов.

10.4) 2 на позициях 1 и 2 одновременно: 2 2 _ 3 3.

Позиция 3: $\{0, 4, 6, 8\}$ – 4 варианта.

Итого 4 варианта.

Итого в случае 10: $294 + 28 + 12 + 4 = 338$ вариантов.

Итого по всем случаям:

$$391 + 371 + 343 + 359 + 318 + 294 + 294 + 322 + 322 + 338 = 3352.$$

Решение программой через itertools:

```
from itertools import *
c = 0
for i in product('012345678', repeat=5):
    s = ''.join(i)
    t = all(j+'2' not in s and '2'+j not in s for j in '1357')
    if s[0] != '0' and s.count('3') == 2 and t:
        c += 1
print(c)
```

Решение программой через циклы:

```
alf = '012345678'
pos1 = '12345678'
c = 0
for i in pos1:
    for j in alf:
        for k in alf:
            for l in alf:
                for m in alf:
                    s = i+j+k+l+m
                    t = all(j + '2' not in s and '2' + j not in s for j in '1357')
                    if s.count('3') == 2 and t:
                        c += 1
print(c)
```

Ответ: 3352**Задача 8.2 (Дальний восток)**

Все пятибуквенные слова, составленные из букв А, К, Ц, Е, Н, Т, записаны в алфавитном порядке и пронумерованы. Вот начало списка:

1. ААААА
2. ААААЕ
3. ААААК
4. ААААН
5. ААААТ
6. ААААЦ

...

Определите, под каким номером в этом списке стоит первое слово, которое не начинается с букв А, Е и К и при этом содержит в своей записи не менее одной буквы Т.

Примечание. Слово – последовательность идущих подряд букв, не обязательно осмысленная.

Решение

```
# Импортируем функцию product из модуля itertools
from itertools import product
# Создаём счётчик номеров слов
count = 0
# Генерируем все возможные 5-буквенные слова
for i in product("АЕКНТЦ", repeat = 5):
    # Преобразуем кортеж букв в строку
    s = "".join(i)
    # Увеличиваем номер текущего слова
```

```
count += 1
# Проверяем:
# 1) слово не начинается с А, Е, К
# 2) содержит не менее 1 буквы Т
if s[0] not in "АЕК" and s.count("Т") >= 1:
    print(count) # Выводим номер первого слова
    break # Завершаем цикл
```

Ответ: 3893

Задача 8.3 (Сибирь)

Все пятибуквенные слова, составленные из букв Г, Р, А, Ф, И, Н, записаны в алфавитном порядке и пронумерованы. Вот начало списка:

1. ААААА
2. ААААГ
3. ААААИ
4. ААААН
5. ААААР
6. ААААФ

...

Определите, под каким номером в этом списке стоит последнее слово с нечётным номером, которое начинается с букв Г, Р и Ф и при этом содержит в своей записи не менее одной буквы А.

Примечание. Слово – последовательность идущих подряд букв, не обязательно осмысленная.

Решение

Решение руками

Для начала каждой букве алфавита дадим числовое обозначение:

А - 0
Г - 1
И - 2
Н - 3
Р - 4
Ф - 5

Мы получили числа бсс. Теперь построим максимальное число, удовлетворяющее условиям:

ФФФФА - 55550₆

Переведем данное число в десятичную сс и получим 7770₁₀

Для того чтобы узнать порядковый номер данного слова увеличим его на единицу и получим 7771

Решение программой с помощью циклов

Создадим переменную `count`, которая будет хранить номер текущего слова в списке. Изначально она равна нулю. Каждый раз после формирования нового слова будем увеличивать её на единицу. Таким образом, значение переменной `count` всегда будет соответствовать номеру рассматриваемого слова.

Для генерации всех возможных слов используем пять вложенных циклов `for`. Каждый цикл отвечает за выбор одной буквы слова.

1. Первый цикл перебирает все возможные варианты первой буквы. 2. Второй цикл перебирает все возможные варианты второй буквы. 3. Третий цикл перебирает все возможные варианты третьей буквы. 4. Четвёртый цикл перебирает все возможные варианты четвёртой буквы. 5. Пятый цикл перебирает все возможные варианты пятой буквы.

Внутри самого глубокого цикла мы объединяем выбранные буквы в одну строку с помощью операции сложения строк. Полученная строка записывается в переменную `s`. Например, если циклы выбрали буквы А, Г, И, Н, Р, то получится слово "АГИНР".

После формирования очередного слова увеличиваем счётчик на единицу, чтобы зафиксировать его номер в общем списке.

Далее необходимо проверить выполнение всех условий задачи.

1. Первая буква слова не должна быть буквой А. - Для этого первая буква должна принадлежать множеству букв Г, Р или Ф. - Первая буква находится по индексу 0, так как индексация строк в Python начинается с нуля. - Проверка выполняется выражением `s[0] in "ГРФ"`.

2. Слово должно содержать хотя бы одну букву А. - Метод `count()` подсчитывает количество вхождений символа в строку. - Выражение `s.count("А")` возвращает число букв А в слове. - Проверка `s.count("А") >= 1` означает, что буква А присутствует хотя бы один раз.

3. Номер слова должен быть нечётным. - Остаток от деления числа на 2 вычисляется оператором `%`. - Если остаток не равен нулю, число нечётное. - Проверка записывается как `count % 2 != 0`.

Если все проверки одновременно истинны, выводим номер текущего слова.

```
# Строка с буквами в алфавитном порядке
a = "АГИНРФ"
# Счётчик номеров слов
count = 0
# Перебираем первую букву слова
for x1 in a:
    # Перебираем вторую букву слова
    for x2 in a:
        # Перебираем третью букву слова
        for x3 in a:
            # Перебираем четвёртую букву слова
            for x4 in a:
                # Перебираем пятую букву слова
                for x5 in a:
                    # Собираем слово из пяти выбранных букв
                    s = x1+x2+x3+x4+x5
                    # Переходим к следующему номеру слова
```

```
count += 1
# Проверяем все условия задачи
if s[0] in "ГРФ" and s.count("А") >= 1 and count % 2 != 0:
    print(count)
```

Решение программой с помощью модуля itertools

Сначала импортируем функцию `product`. Затем создаём переменную `count`, которая будет хранить номер текущего слова в списке.

Функция `product('АГИНРФ', repeat = 5)` последовательно формирует все возможные наборы из пяти букв. Каждая полученная комбинация представляет собой кортеж. Например:

("А" "А" "А" "А" "А")

или

("А" "А" "А" "А" "Г")

Поскольку буквы в строке "АГИНРФ" записаны в алфавитном порядке, функция `product()` генерирует комбинации точно в том порядке, который требуется по условию задачи.

На каждой итерации цикла переменная `x` содержит очередной кортеж символов. Для дальнейшей работы его необходимо преобразовать в строку. Это выполняется методом `join()`.

После получения строки увеличиваем счётчик на единицу, чтобы получить номер текущего слова.

Далее выполняются те же самые проверки:

1. Первая буква должна быть одной из букв Г, Р или Ф. - Используется проверка `s[0] in 'ГРФ'`.
 2. В слове должна присутствовать хотя бы одна буква А. - Используется проверка `s.count('А') >= 1`.
 3. Номер слова должен быть нечётным. - Используется проверка `count % 2 != 0`.
- Если все условия выполняются одновременно, выводим номер слова.

```
# Импортируем модуль product
from itertools import product
# Счётчик номеров слов
count = 0
# Генерируем все возможные 5-буквенные слова
for x in product("АГИНРФ", repeat=5):
    # Преобразуем кортеж символов в строку
    s = "".join(x)
    # Увеличиваем номер текущего слова
    count += 1
    # Проверяем все условия задачи
    if s[0] in "ГРФ" and s.count("А") >= 1 and count % 2 != 0:
        print(count)
```

Ответ: 7771

Задача №9

Задача 9.1 (Дальний восток)

Откройте файл электронной таблицы, содержащей в каждой строке семь натуральных чисел. Определите количество строк таблицы, содержащих числа, для которых выполнены оба условия:

- одно число повторяется 3 раза, другое 2 раза, остальные различны;
- максимальное из повторяющихся меньше наибольшего из неповторяющихся.

Решение

Решение в электронных таблицах:

Для начала откроем файл электронной таблицы. Далее посчитаем, сколько раз каждое число встречается в строке. Для этого в ячейку H1 запишем следующую формулу и растянем её до ячейки N1 вправо и вниз до конца таблицы:

$$= \text{СЧЁТЕСЛИ}(\$A1:\$G1;A1)$$

Теперь выделим повторяющиеся числа. В ячейку O1 запишем формулу и растянем вправо до U1 и вниз до конца таблицы. Если число повторяется (частота > 1), записываем само число, иначе 0:

$$= \text{ЕСЛИ}(H1>1;A1;0)$$

Аналогично выделим неповторяющиеся числа. В ячейку V1 запишем формулу и растянем вправо до AB1 и вниз до конца таблицы. Если число встречается ровно 1 раз, записываем его, иначе 0:

$$= \text{ЕСЛИ}(H1=1;A1;0)$$

В столбце AC проверим первое условие: одно число повторяется три раза, другое - два раза, остальные два числа различны. В ячейку AC1 запишем формулу и растянем её вниз до конца таблицы:

$$= \text{ЕСЛИ}(\text{И}(\text{СЧЁТЕСЛИ}(H1:N1;3)=3;\text{СЧЁТЕСЛИ}(H1:N1;2)=2;\text{СЧЁТЕСЛИ}(H1:N1;1)=2);1;0)$$

В столбце AD проверим второе условие: максимальное из повторяющихся чисел меньше наибольшего из неповторяющихся. В ячейку AD1 запишем формулу и растянем её вниз до конца таблицы:

$$= \text{ЕСЛИ}(\text{МАКС}(O1:U1)<\text{МАКС}(V1:AB1);1;0)$$

В столбце AE проверим выполнение обоих условий. В ячейку AE1 запишем формулу и растянем вниз до конца таблицы:

$$= AC1*AD1$$

Выделяем (суммируем) столбец АЕ и получаем ответ

Решение программой:

Будем построчно считывать числа из .csv файла, числа разделены точкой с запятой. Для каждой строки собираем числа по частоте: встречающиеся 3 раза, 2 раза и 1 раз. Проверяем первое условие: длины списков равны 3, 2 и 2 соответственно. Для второго условия проверяем, что максимум среди повторяющихся чисел меньше максимума среди неповторяющихся.

```
# Открываем файл таблицы
file = open("9.csv")

# Счётчик подходящих строк
count = 0

# Перебираем строки таблицы
for line in file:
    # Считываем числа из строки
    numbers = list(map(int, line.split(";")))
    # Собираем числа, встречающиеся 3 раза
    rep3 = [x for x in numbers if numbers.count(x) == 3]
    # Собираем числа, встречающиеся 2 раза
    rep2 = [x for x in numbers if numbers.count(x) == 2]
    # Собираем числа, встречающиеся 1 раз
    rep1 = [x for x in numbers if numbers.count(x) == 1]
    # Условие 1: одно число x3, одно x2, два x1
    if len(rep3) == 3 and len(rep2) == 2 and len(rep1) == 2:
        # Условие 2: max повторяющихся < max неповторяющихся
        if max(rep3 + rep2) < max(rep1):
            count += 1

# Выводим количество подходящих строк
print(count)
```

Ответ: 60

Задача 9.2 (Сибирь)

Откройте файл электронной таблицы, содержащей в каждой строке пять натуральных чисел. Определите наименьший номер строки, для которой выполнены оба условия:

- все числа в строке различны;
- удвоенная сумма максимального и минимального больше суммы остальных трех чисел.

Решение

Решение Excel:

Сначала проверим 1 условие. Запишем в ячейку F1 следующую формулу и растянем её вниз до конца таблицы:

= ЕСЛИ(ИЛИ(A1= B1;A1= C1;A1= D1;A1= E1;B1= C1;B1= D1;B1= E1;C1= D1;C1= E1;D1= E1);0;1)

Затем проверим 2 условие. Запишем в ячейку G1 следующую формулу и растянем её вниз до конца таблицы:

= ЕСЛИ((МАКС(A1:E1)+ МИН(A1:E1))*2> СУММ(A1:E1)-МАКС(A1:E1)-МИН(A1:E1);1;0)

Для того чтобы выполнялись оба условия, в ячейку H1 запишем формулу и растянем её вниз до конца таблицы:

= F1*G1

Находим первую строку, в которой в столбце H стоит единица и получаем ответ – 4.

Решение программой:

Сначала мы открываем файл с данными. В каждой строке файла содержится пять натуральных чисел. Для доступа к данным используем функцию `open()`, которая позволяет читать файл построчно.

Далее создаём переменную-счётчик, которая будет хранить количество строк, удовлетворяющих условиям задачи. Изначально счётчик равен нулю.

После этого организуем цикл `for`, который последовательно перебирает все строки файла. На каждой итерации переменная `line` содержит одну строку текста.

Сразу же вводим счётчик строк с помощью переменной `count`.

Затем преобразуем строку в список чисел. Для этого выполняем следующие действия:

1. Разбиваем строку на отдельные элементы с помощью метода `split()`, который разделяет строку по пробелам.

2. Каждый элемент преобразуем в целое число с помощью функции `int()`.

3. Формируем из этих чисел список `a`.

После получения списка выполняем проверку условий задачи.

– Первое условие: все числа в строке различны.

Для проверки этого условия используем множество. Множество автоматически удаляет повторяющиеся элементы.

Если длина множества `set(a)` равна 5 (число элементов в строке), значит все числа различны.

– Второе условие: удвоенная сумма максимального и минимального числа строки больше суммы оставшихся трёх чисел. Для этого:

- находим максимальное число в списке с помощью функции `max(a)`;
- находим минимальное число в списке с помощью функции `min(a)`;
- вычисляем удвоенную сумму максимального и минимального числа (`2 * (max(a) + min(a))`);
- вычисляем сумму оставшихся трёх чисел, вычитая максимальное и минимальное число из общей суммы списка (`sum(a) - max(a) - min(a)`);
- проверяем неравенство `2 * (max(a) + min(a)) > sum(a) - max(a) - min(a)`.

Если оба условия выполняются, выводим номер строки и прекращаем цикл.

```
# Открываем файл для чтения строк
f = open("9-2.txt")
# Создаём переменную-счётчик для подсчёта подходящих строк
count = 0
# Перебираем все строки файла по одной
for line in f:
    count += 1
    # Разбиваем строку на элементы, преобразуем их в целые числа
    # и формируем список из пяти чисел
    a = [int(x) for x in line.split()]
    # Проверяем выполнение двух условий:
    # 1) все числа различны
    # 2) удвоенная сумма максимального и минимального числа
    # больше суммы оставшихся трёх чисел
    if len(set(a)) == 5 and 2*(max(a)+min(a)) > sum(a)-max(a)-min(a):
        # Если оба условия выполняются, выводим номер
        # первой подошедшей строки
        print(count)
        break
```

Ответ: 4

Задача №10

Задача 10.1 (Дальний восток)

С помощью текстового редактора определите, сколько раз в тексте глав 5 и 6 романа в стихах А. С. Пушкина «Евгений Онегин» встречается слово «Что» (или «что») в качестве отдельного слова.

Регистр при поиске учитывать не следует, другие слова, в состав которых входит «что» как часть, учитывать не нужно. В ответе запишите только число.

Решение

Открываем файл в текстовом редакторе и удаляем все главы, кроме 5 и 6.

Нажимаем сочетание клавиш Ctrl + F и открываем окно поиска. Записываем поиск слова «что», ставим галочку на «только слово целиком».

Получаем результат 29 слов. Проверяем каждое из них и исключаем такие слова как «что-нибудь», «что-то» (всего 3 слова). Дефисные слова лингвистически и технически считаются отдельными самостоятельными лексемами (местоимениями), а не чистым словом «что». Итоговый ответ:

$$29 - 3 = 26$$

Ответ: 26

Задача 10.2 (Дальний восток)

С помощью текстового редактора определите, сколько раз встречается отдельное слово «и» или «И» в тексте глав XI и XIV второй части тома 2 романа Л.Н. Толстого «Война и мир». В ответе запишите только число.

Решение

Открываем имеющийся текстовый документ, а также создаём новый – в него мы будем добавлять проверяемые части текста.

Заходим во вкладку «Вид» → «Навигатор», выбираем нужные главы, после чего копируем их и вставляем в созданный файл.

Теперь открываем меню поиска при помощи специальной кнопки или сочетания клавиш Ctrl + H.

Находим «и» в основном документе, поставив галочку возле «Только слово целиком». Полученное число записываем в качестве ответа.

Ответ: 148

Задача 10.3 (Сибирь)

С помощью текстового редактора определите, сколько раз встречается отдельное слово «а» или «А» в тексте глав XXII и XXIII второй части романа Михаила Булгакова «Мастер и Маргарита». В ответе запишите только число.

Решение

Открываем имеющийся текстовый документ, а также создаём новый – в него мы будем добавлять проверяемые части текста.

Заходим во вкладку «Вид» → «Навигатор», выбираем нужные главы, после чего копируем их и вставляем в созданный файл.

Теперь открываем меню поиска при помощи специальной кнопки или сочетания клавиш Ctrl + Н.

Находим «а» в основном документе, поставив галочку возле «Только слово целиком». Полученное число записываем в качестве ответа.

Ответ: 62

Задача 10.4 (Центр)

Определите, сколько раз в тексте главы VII рассказав А. Куприна «Поединок» встречается сочетание букв «При» или «при» только в составе других слов, включая сложные слова, соединённые дефисом, но не как отдельное слово. В ответе укажите только число.

Решение

Открываем файл и удаляем все главы, кроме седьмой.

Нажимаем сочетание клавиш Ctrl + F, чтобы открыть окно поиска.

Вводим в поиск слово «при» без учета регистра и получаем 24 слова. Далее ставим галочку на «слово целиком» и получаем 1 слово.

Проверив все найденные слова, убеждаемся, что сложных слов с дефисом нет. Значит, ответ $24 - 1 = 23$ слова.

Ответ: 23

Задача №11

Задача 11.1 (Дальний восток)

На предприятии каждой изготовленной детали присваивают серийный номер из 225 символов. Для хранения каждого номера отведено одинаковое и минимально возможное число байт; используется посимвольное кодирование – все символы кодируются одинаковым и минимально возможным числом бит. Известно, что для хранения 1270 серийных номеров отведено не более 104 Кбайт памяти.

Определите максимально возможную мощность алфавита, используемого для записи серийных номеров.

Решение

Переводим общий объём в байты:

$$104 \text{ Кбайт} = 104 \cdot 1024 = 106496 \text{ байт}$$

На один номер отведено одинаковое число байт. Максимум байт на номер:

$$\frac{106496}{1270} = 83,85 \approx 83 \text{ байта} = 664 \text{ бита}$$

Округляем в меньшую сторону, иначе превысим допустимый объём.

Номер из 225 символов по i бит занимает $225 \cdot i$ бит, при этом он не должен превышать 664 бит.

$$i = \frac{664}{225} \approx 2,95 = 2 \text{ бита}$$

Округляем в меньшую сторону, иначе превысим допустимый объём.

Максимальная мощность алфавита при i битах на символ 2^i :

$$2^2 = 4$$

Ответ: 4

Задача 11.2 (Дальний восток)

При регистрации в компьютерной системе каждому объекту присваивается идентификатор, состоящий из 257 символов и содержащий только цифры семнадцатеричной системы счисления и символы из 4080-символьного специального алфавита. В базе данных для хранения каждого идентификатора отведено одинаковое и минимально возможное целое число байт. При этом используется посимвольное кодирование идентификаторов, все символы кодируются одинаковым и минимально возможным количеством бит. Определите объём памяти (в Мбайт), необходимый для хранения 8 388 608 идентификаторов.

Решение

Общий алфавит для идентификатора содержит $17 + 4080 = 4097$ символов.

Для кодирования такого количества символов потребуется 13 бит, так как $2^{12} < 4097 \leq 2^{13}$.

Один идентификатор занимает $257 \cdot 13 = 3341 = 417.625 \approx 418$. Округляем в большую сторону, так как иначе не сможем закодировать полную информацию об идентификаторе.

Для хранения 8388608 идентификаторов потребуется $418 \cdot 8388608 = 3506438144$ байт = 3344 Мбайт

Ответ: 3344

Задача 11.3 (Центр)

На предприятии каждой изготовленной детали присваивают серийный номер, состоящий из 199 символов. В базе данных каждый серийный номер занимает одинаковое и минимально возможное целое число байт. При этом используется посимвольное кодирование серийных номеров, все символы кодируются одинаковым и минимально возможным числом бит. Известно, что для хранения 257 384 серийных номеров отведено не более 74 Мбайт памяти. Определите максимально возможную мощность алфавита, используемого для записи серийных номеров. В ответе запишите только целое число.

Решение

Так как для хранения 257 384 серийных номеров отведено не более 74 Мбайт памяти, то для хранения одного номера доступно не более:

$$\frac{74 \cdot 1024 \cdot 1024}{257384} = 301,474 \sim 301 \text{ байт}$$

Округляем в меньшую сторону, чтобы не выйти за рамки доступной памяти.

Серийный номер состоит из 199 символов, значит, для хранения одного символа нужно:

$$\frac{301 \cdot 8}{199} = 12,10 \sim 12 \text{ бит}$$

Округляем в меньшую сторону, чтобы не выйти за рамки доступной памяти.

Максимальная возможная мощность алфавита равна:

$$2^{12} = 4096$$

Ответ: 4096

Задача №12

Задача 12.1 (Дальний восток)

Исполнитель МТ представляет собой читающую и записывающую головку, которая может передвигаться вдоль бесконечной горизонтальной ленты, разделённой на равные ячейки. В каждой ячейке находится ровно один символ из алфавита исполнителя (множество символов $A = \{a_0, a_1, \dots, a_{n-1}\}$), включая специальный пустой символ a_0 . Время работы исполнителя делится на дискретные такты (шаги). На каждом такте головка МТ находится в одном из множества допустимых состояний $Q = \{q_0, q_1, \dots, q_{n-1}\}$. В начальный момент времени головка находится в начальном состоянии q_0 .

На каждом такте головка обзереает одну ячейку ленты, называемую текущей ячейкой. За один такт головка исполнителя может переместиться в ячейку справа или слева от текущей, не меняя находящийся в ней символ, или заменить символ в текущей ячейке без сдвига в соседнюю ячейку. После каждого такта головка переходит в новое состояние или остаётся в прежнем состоянии.

Программа работы исполнителя МТ задаётся в табличном виде.

	a_0	a_1	$\dots$	a_{n-1}
q_0	команда	команда	$\dots$	команда
q_1	команда	команда	$\dots$	команда
$\dots$	команда	команда	$\dots$	команда
q_{n-1}	команда	команда	$\dots$	команда

В первой строке перечислены все возможные символы в текущей ячейке ленты, в первом столбце – возможные состояния головки. На пересечении i -й строки и j -го столбца находится команда, которую выполняет МТ, когда головка обзереает j -й символ, находясь в i -м состоянии. Если пара «символ – состояние» невозможна, то клетка для команды остаётся пустой. Каждая команда состоит из трёх элементов, разделённых запятыми: первый элемент – записываемый в текущую ячейку символ алфавита (может совпадать с тем, который там уже записан). Второй элемент – один из четырёх символов «L», «R», «N», «S». Символы «L» и «R» означают сдвиг в левую или правую ячейку соответственно, «N» – отсутствие сдвига, «S» – завершение работы исполнителя МТ после выполнения текущей команды. Сдвиг происходит после записи символа в текущую ячейку. Третий элемент – новое состояние головки после выполнения команды. Например, команда $0, L, q_3$ выполняется следующим образом: в текущую ячейку записывается символ «0», затем головка сдвигается в соседнюю слева ячейку и переходит в состояние q_3 .

Приведём пример выполнения программы, заданной таблично.

На ленте записано неизвестное ненулевое количество расположенных подряд в соседних ячейках символов «Z», все остальные ячейки ленты заполнены пустым символом «λ». В начальный момент времени головка находится на неизвестном ненулевом расстоянии справа от самого правого символа «Z».

Программа

	λ	Z
q_0	λ, L, q_0	X, L, q_1
q_1	λ, S, q_1	X, L, q_1

заменяет на ленте все символы « Z » на « X » и останавливает исполнителя в первой ячейке слева от последовательности символов « X ».

Возможное начальное состояние исполнителя:

...	λ	λ	Z	Z	Z	Z	λ	λ	...
$\blacktriangle q_0$									

Конечное состояние исполнителя после завершения выполнения программы:

...	λ	λ	X	X	X	X	λ	λ	...
$\blacktriangle q_1$									

Выполните задание.

На ленте в соседних ячейках записано двоичное представление числа 2026 без ведущих нулей. Ячейки справа и слева от последовательности заполнены пустыми символами « λ ». В начальный момент времени головка расположена в ближайшей слева от последовательности ячейке.

Программа работы исполнителя:

	λ	0	1
q_0	λ, R, q_1		
q_1	$1, R, q_2$	$0, R, q_1$	$1, R, q_1$
q_2	λ, R, q_3		
q_3	λ, S, q_3		

Определите результат выполнения программы. В ответе запишите получившееся число в десятичной системе счисления.

Решение

Решение руками:

Исполнитель идет слева направо, не производя замен символов исходной последовательности. При этом символ λ справа от последовательности заменяется на 1.

Исходная последовательность: 11111101010

Добавляем 1 справа и находим конечное число:

$$111111010101_2 = 4053_{10}$$

Решение Python:

```
a = {
    'q0.': ['.', 'R', 'q1'],
    'q10': ['0', 'R', 'q1'],
    'q11': ['1', 'R', 'q1'],
    'q1.': ['1', 'R', 'q2'],
    'q2.': ['.', 'R', 'q3'],
    'q3.': ['.', 'S', 'q3'],
}

# двоичная запись числа 2026
s1 = bin(2026)[2:]
# лента с пустыми символами по краям
s = list('...' + s1 + '...')

# головка стоит на ближайшей слева пустой ячейке
i, c, p = 2, 0, 'q0'

while c != 'S':
    s[i], c, p = a[p + s[i]]

    if c == 'R':
        i += 1

print(int(''.join(s).replace('.', ''), 2))
```

Ответ: 4053**Задача 12.2 (Сибирь)**

Исполнитель МТ представляет собой читающую и записывающую головку, которая может передвигаться вдоль бесконечной горизонтальной ленты, разделённой на равные ячейки. В каждой ячейке находится ровно один символ из алфавита исполнителя (множество символов $A = \{a_0, a_1, \dots, a_{n-1}\}$), включая специальный пустой символ a_0 . Время работы исполнителя делится на дискретные такты (шаги). На каждом такте головка МТ находится в одном из множества допустимых состояний $Q = \{q_0, q_1, \dots, q_{n-1}\}$. В начальный момент времени головка находится в начальном состоянии q_0 .

На каждом такте головка обозревает одну ячейку ленты, называемую текущей ячейкой. За один такт головка исполнителя может переместиться в ячейку справа или слева от текущей, не меняя находящийся в ней символ, или заменить символ в текущей ячейке без сдвига в соседнюю ячейку. После каждого такта головка переходит в новое состояние или остаётся в прежнем состоянии.

Программа работы исполнителя МТ задаётся в табличном виде.

	a_0	a_1	$\dots$	a_{n-1}
q_0	команда	команда	$\dots$	команда
q_1	команда	команда	$\dots$	команда
$\dots$	команда	команда	$\dots$	команда
q_{n-1}	команда	команда	$\dots$	команда

В первой строке перечислены все возможные символы в текущей ячейке ленты, в первом столбце – возможные состояния головки. На пересечении i -й строки и j -го столбца находится команда, которую выполняет МТ, когда головка обзрывает j -й символ, находясь в i -м состоянии. Если пара «символ – состояние» невозможна, то клетка для команды остаётся пустой. Каждая команда состоит из трёх элементов, разделённых запятыми: первый элемент – записываемый в текущую ячейку символ алфавита (может совпадать с тем, который там уже записан). Второй элемент – один из четырёх символов «L», «R», «N», «S». Символы «L» и «R» означают сдвиг в левую или правую ячейки соответственно, «N» – отсутствие сдвига, «S» – завершение работы исполнителя МТ после выполнения текущей команды. Сдвиг происходит после записи символа в текущую ячейку. Третий элемент – новое состояние головки после выполнения команды. Например, команда $0, L, q_3$ выполняется следующим образом: в текущую ячейку записывается символ «0», затем головка сдвигается в соседнюю слева ячейку и переходит в состояние q_3 .

Приведём пример выполнения программы, заданной таблично.

На ленте записано неизвестное ненулевое количество расположенных подряд в соседних ячейках символов «Z», все остальные ячейки ленты заполнены пустым символом «λ». В начальный момент времени головка находится на неизвестном ненулевом расстоянии справа от самого правого символа «Z».

Программа

	λ	Z
q_0	λ, L, q_0	X, L, q_1
q_1	λ, S, q_1	X, L, q_1

заменяет на ленте все символы «Z» на «X» и останавливает исполнителя в первой ячейке слева от последовательности символов «X».

Возможное начальное состояние исполнителя:

$\dots$	λ	λ	Z	Z	Z	Z	λ	λ	$\dots$
$\blacktriangle q_0$									

$\dots$	λ	λ	X	X	X	X	λ	λ	$\dots$
$\blacktriangle q_1$									

Конечное состояние исполнителя после завершения выполнения программы:

Выполните задание.

На ленте в соседних ячейках записано двоичное представление числа 2027 без ведущих нулей. Ячейки справа и слева от последовательности заполнены пустыми символами «λ». В начальный момент времени головка расположена в ближайшей слева к последовательности

ячейке.

Программа работы исполнителя:

	λ	0	1
q_0	λ, R, q_1		
q_1	$1, R, q_2$	$0, R, q_1$	$1, R, q_1$
q_2	$1, R, q_3$		
q_3	λ, S, q_3		

Определите результат работы программы. В ответе запишите получившееся на ленте число в десятичной системе счисления.

Решение

Решение руками:

Переведём число 2027 из десятичной системы счисления в двоичную:

$$2027_{10} = 11111101011_2$$

Далее проведем все действия над этой строкой согласно программе работы исполнителя:

$$\overset{q_0}{\lambda\lambda\lambda\lambda}11111101011\lambda\lambda\lambda\lambda$$

В начальный момент времени каретка находится над пустым символом и делает шаг вправо со сменой состояния на q_1 :

$$\overset{q_1}{\lambda\lambda\lambda\lambda}11111101011\lambda\lambda\lambda\lambda$$

Далее происходит проход по всей последовательности без фактической замены символов, так как 0 меняется на 0, 1 на 1:

$$\overset{q_1}{\lambda\lambda\lambda\lambda}11111101011\lambda\lambda\lambda\lambda$$

Как только каретка оказывается на первом пустом символе справа от последовательности, выполняется действие $(1, R, q_2)$, то есть пустой символ становится единицей, делаем шаг вправо и меняем состояние на q_2 :

$$\overset{q_2}{\lambda\lambda\lambda\lambda}11111101011\lambda\lambda\lambda\lambda$$

В состоянии q_2 на символе λ она также меняется на единицу и делается шаг вправо, состояние меняется на q_3 :

$$\overset{q_3}{\lambda\lambda\lambda\lambda}111111010111\lambda\lambda\lambda$$

В состоянии q_3 над пустым символом программа останавливается.

В результате работы программы у нас справа от последовательности добавляется два символа:

11.

Переведём число результат в десятичную систему счисления:

$$1111110101111_2 = 8111_{10}$$

Решение программой:

Данная программа на языке Python моделирует работу машины Тьюринга, заданной в условии задачи таблично, и используется для определения результата её работы над входным числом – двоичным представлением числа.

Программа машины Тьюринга хранится в виде словаря, где каждой паре «текущее состояние – символ в ячейке» сопоставляется команда, состоящая из записываемого символа, действия головки (сдвиг вправо либо остановка) и нового состояния. Пустой символ λ обозначается точкой.

Лента моделируется списком символов, в который помещается двоичная запись числа 2027, окружённая пустыми ячейками, а головка в начальный момент времени располагается в ячейке слева от числа и находится в состоянии q_0 .

Работа машины реализована циклом, в котором на каждом шаге по текущему состоянию и символу выбирается соответствующая команда, производится запись нового символа, сдвиг головки и переход в новое состояние. Цикл выполняется до получения команды с действием S , означающей завершение работы. В ходе выполнения программы к исходной двоичной записи числа приписываются дополнительные биты в соответствии с правилами машины Тьюринга из условия.

После остановки машины из ленты извлекаются все символы 0 и 1, формирующие итоговое двоичное число, которое затем переводится в десятичную систему счисления и выводится на экран.

```
from itertools import product
```

```
# Таблица правил машины Тьюринга в виде словаря
```

```
# Ключ = текущее_состояние + текущий_символ_на_ленте
```

```
# Значение = [символ_для_записи, направление_движения, следующее_состояние]
```

```
a = {
```

```
    "q0.": [".", "R", "q1"],
```

```
    "q1.": ["1", "R", "q2"],
```

```
    "q2.": ["1", "R", "q3"],
```

```
    "q3.": [".", "S", "q3"],
```

```
    "q10": ["0", "R", "q1"],
```

```
    "q11": ["1", "R", "q1"],
```

```
}
```

```
# Переводим десятичное число 2027 в 2 СС
```

```
st = bin(2027)[2:]
```

```
# Создаём ленту: ....исходная_последовательность....
```

```
# Добавляем пустые символы по бокам
```

```
s = list("..." + st + "...")
```

```
# Инициализация параметров машины Тьюринга
```

```
i = 3      # Индекс головки (начинаем в ближайшей слева от последовательности ячейке)
```

```
c = 0      # Переменная для хранения команды движения (будет перезаписана)
```

```
p = "q0"    # Начальное состояние q0
# Основной цикл выполнения программы
while c != "S": # Выполняем, пока не получим команду "S" (стоп)
    # Получаем команду по ключу "состояние+текущий символ"
    # Записываем новый символ, получаем движение и новое состояние
    s[i], c, p = a[p + s[i]]
    # Перемещаем головку в соответствии с командой
    if c == "L":
        i -= 1 # Движение влево
    else:
        i += 1 # Движение вправо (также для случая "S", но цикл завершится)
# Формируем результат: собираем только символы "0" и "1" с ленты
res = "".join(i for i in s if i in "01")
# Выводим ответ в 10 СС
print(int(res, 2))
```

Ответ: 8111

Задача №13

Задача 13.1 (Дальний восток)

Сеть задана IP-адресом одного из входящих в неё узлов 102.162.200.51 и сетевой маской 255.255.255.0.

Найдите в данной сети наибольший IP-адрес, который может быть назначен компьютеру. В ответе укажите сумму числовых значений октетов найденного IP-адреса.

Например, если бы найденный адрес был равен 100.20.3.4, то в ответе следовало бы записать: 127.

Решение

Решение руками:

По условию задачи IP-адрес узла равен 102.162.200.51, а маска сети - 255.255.255.0. Сетевая маска 255.255.255.0 означает, что первые три байта IP-адреса относятся к адресу сети, а последний байт отвечает за номер узла в этой сети.

Таким образом, адрес сети равен 102.162.200.0.

Диапазон всех адресов в этой сети - от 102.162.200.0 до 102.162.200.255.

Адрес сети (102.162.200.0) и широковещательный адрес (102.162.200.255) не могут быть назначены компьютерам.

Следовательно, наибольший IP-адрес, который может быть назначен компьютеру, равен 102.162.200.254.

Найдем сумму числовых значений октетов этого адреса: $102 + 162 + 200 + 254 = 718$.

Решение программой:

Полученная сеть содержит список всех адресов от адреса сети (индекс 0) до широковещательного адреса (последний индекс, или -1). Максимальный доступный для компьютера адрес (последний хост) будет находиться по индексу -2 .

Далее преобразуем этот адрес в строку, разобьем по символу точки на отдельные октеты, приведем их к целочисленному типу и найдем сумму.

```
from ipaddress import ip_network

# Создаем сеть по адресу узла и маске
net = ip_network('102.162.200.51/255.255.255.0', strict=0)

# Получаем наибольший IP-адрес, доступный для компьютера
max_ip = net[-2]

# Преобразуем IP-адрес в список целых чисел (октетов) и находим их сумму
octets_sum = sum(map(int, str(max_ip).split('.')))

print(octets_sum)
```

Ответ: 718

Задача 13.2 (Дальний восток)

В терминологии сетей TCP/IP маской сети называют двоичное число, которое показывает, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу узла в этой сети. Обычно маска записывается по тем же правилам, что и IP-адрес, – в виде 4 байтов, причем каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и его маске.

Например, если IP-адрес узла равен 231.32.255.131, а маска равна 255.255.240.0, то адрес сети равен 231.32.240.0.

Узел имеет IP-адрес 210.185.140.126. Маска сети равна 255.255.255.252. Определите наименьший возможный IP-адрес в этой сети (адрес сети) и в ответе запишите сумму значений его октетов.

Решение**Решение руками:**

Наименьшим возможным IP-адресом в сети является адрес самой сети. Чтобы его найти, применим поразрядную конъюнкцию (логическое «И») к IP-адресу узла и маске.

Так как первые три байта маски равны 255, первые три байта адреса сети полностью совпадают с IP-адресом: 210.185.140.

Для четвертого байта выполним операцию «И» в двоичном виде:

126 в двоичной системе: 01111110

252 в двоичной системе: 11111100

Перемножим разряды:

$$01111110 \& 11111100 = 01111100$$

Полученное двоичное число 01111100 в десятичной системе равно 124. Таким образом, адрес сети равен 210.185.140.124.

Сложим значения его октетов:

$$210 + 185 + 140 + 124 = 659$$

Решение Python:

Параметр `strict=0` позволяет использовать адрес узла. Адрес сети (наименьший IP) хранится в свойстве `network_address`. Преобразуем его в строку, делим по точкам на части, переводим их в целые числа и складываем.

```
from ipaddress import ip_network
# Создаем объект сети
net = ip_network('210.185.140.126/255.255.255.252', strict=0)
# Получаем адрес сети в виде строки
net_addr = str(net.network_address)
# Разбиваем строку по точкам, переводим части в числа и суммируем их
ans = sum(int(x) for x in net_addr.split('.'))
print(ans)
```

Ответ: 659**Задача 13.3 (Центр)**

В терминологии сетей TCP/IP маской сети называют двоичное число, которое показывает, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу узла в этой сети. Адрес сети получается в результате применения поразрядной конъюнкции к заданному адресу узла и его маске. Широковещательным адресом называется специализированный адрес, в котором на месте нулей в маске стоят единицы. Адрес сети и широковещательный адрес не могут быть использованы для адресации сетевых устройств.

Сеть задана IP-адресом одного из входящих в неё узлов 188.163.128.179 и сетевой маской 255.255.240.0.

Определите **наименьший** IP-адрес данной сети, который может быть присвоен компьютеру. В ответе укажите сумму октетов у найденного IP-адреса.

Например, если бы найденный адрес был равен 111.22.3.44, то в ответе следовало бы записать 180.

Решение**Решение руками**

Переведём IP-адрес узла и маску сети в двоичную систему счисления:

IP узла	10111100.10100011.10000000.10110011
Маска	11111111.11111111.11110000.00000000

Узлы в сети имеют вид: 10111100.10100011.1000xxxx.xxxxxx

Нам нужно найти наименьший IP-адрес в данной сети, который может быть назначен компьютеру. Но мы не можем назначать компьютеру адрес сети.

Поэтому, первый возможный IP-адрес будет равен:

$$10111100.10100011.10000000.00000001 = 188.163.128.1_{10}$$

Сумма октетов равна:

$$188 + 163 + 128 + 1 = 480$$

Решение программой

Для нахождения наименьшего IP-адреса компьютера в сети необходимо определить адрес сети и взять адрес, следующий за ним. Адрес адрес является первым адресом в диапазоне сети и не может быть использован для компьютеров. Следовательно, наименьший допустимый адрес для компьютера будет вторым адресом в сети.

В Python для работы с IP-адресами удобно использовать модуль `ipaddress`, который позволяет легко выполнять операции с сетевыми диапазонами. Создаем объект сети с помощью метода `ip_network()`, используя IP-адрес узла и маску подсети. Затем, используя индексацию `[1]`, получаем второй адрес в сети, который и будет искомым наименьшим адресом для компьютера.

```
# Импортируем модуль для работы с IP-адресами
from ipaddress import *

# Создаем объект сети с указанным IP-адресом и маской
net = ip_network("188.163.128.179/255.255.240.0", 0)

# Получаем второй адрес в сети:
# net[0] - первый адрес (адрес сети)
# net[1] - второй адрес
print(net[1])

# Вывод: 188.163.128.1
# Сумма октетов: 188 + 163 + 128 + 1 = 480
```

Ответ: 480

Задача №14

Задача 14.1 (Дальний восток)

Операнды арифметического выражения записаны в системе счисления с основанием 22.

$$63x89875_{22} + 17x51_{22} + 75x3_{22}$$

В записи чисел переменной x обозначена неизвестная цифра из алфавита 22-ричной системы счисления. Определите наибольшее значение x , при котором значение данного арифметического выражения кратно 21. Для найденного значения x вычислите частное от деления значения арифметического выражения на 21 и укажите его в ответе в десятичной системе счисления. Основание системы счисления указывать не нужно.

Решение

```
alf = '0123456789abcdefghijklmnopqrstuvwxyz'
```

```
for x in alf:
    n = int(f'63{x}89875', 22) + int(f'17{x}51', 22) + int(f'75{x}3', 22)
    if n % 21 == 0:
        print(x, n//21)
```

Ответ: 733155579

Задача 14.2 (Дальний восток)

Значение арифметического выражения

$$4 \cdot 16^{25} + 2 \cdot 8^{30} - 64^{10}$$

записали в двоичной системе счисления. Сколько цифр 0 содержится в этой записи?

Решение

Для решения задачи мы вычисляем значение арифметического выражения, преобразуем его в двоичную строку с помощью встроенной функции `bin()` (отсекая технический префикс `0b` с помощью среза `[2:]`) и подсчитываем количество нулей методом `.count('0')`.

```
num = 4 * 16**25 + 2 * 8**30 - 64**10
# Переводим число в двоичный вид и убираем служебный префикс '0b'
binary_str = bin(num)[2:]
# Считаем количество нулей в полученной строке
zeros = binary_str.count('0')
print(zeros)
```

Ответ: 71

Задача 14.3 (Сибирь)

Значение арифметического выражения

$$5 \cdot 7776^{2013} + 4 \cdot 1296^{2015} - 3 \cdot 216^{2017} + 2 \cdot 36^{2017} - 6^{2019} + 2021$$

записали в 36-ричной системе счисления. Определите сумму цифр больших 9 в этой записи.

Для решения задачи мы вычисляем значение арифметического выражения, реализуем алгоритм перевода в 36-сс, проверяем каждую цифру - если она больше 9, складываем в сумму.

Решение

```
num = 5 * 7776**2013 + 4 * 1296**2015 - 3 * 216**2017 + 2 * 36**2017 - 6**2019 + 2021
summ = 0 # счётчик для суммы цифр
while num > 0:
    # если очередная цифра больше 9, добавляем её в сумму
    if num % 36 > 9:
        summ += num % 36
    num //= 36
print(summ)
```

Ответ: 70483

Задача 14.4 (Сибирь)

Значение арифметического выражения

$$5 \cdot 7776^{2013} + 4 \cdot 1296^{2015} - 3 \cdot 216^{2017} + 2 \cdot 36^{2017} - 6^{2019} + 2023$$

записали в 36-ричной системе счисления. Определите количество цифр, не превышающих 9 в этой записи.

Решение

Для решения задачи вычисляем значение арифметического выражения, реализуем алгоритм перевода в 36-сс, проверяем каждую цифру - если она не превышает 9, увеличиваем счётчик.


```
num = 5*7776**2013 + 4*1296**2015 - 3*216**2017 + 2*36**2017 - 6**2019 + 2023
cnt = 0 # счётчик для количества цифр
while num > 0:
    # если очередная цифра не превышает 9, увеличиваем количество таких
    if num % 36 <= 9:
        cnt += 1
    num //= 36
print(cnt)
```

Ответ: 3018

Задача 14.5 (Центр)

Значение арифметического выражения

$$5^{150} + 5^{100} - x,$$

где x – целое положительное число, меньшее 2030, записали в пятеричной системе счисления. Определите наименьшее значение x , при котором количество нулей в пятеричной записи числа, являющегося значением данного арифметического выражения, максимально. В ответе запишите число в десятичной системе счисления.

Решение

В цикле переберём значения x от 1 до 2029 включительно. Для каждого значения найдём значение выражения в десятичной системе счисления и переведём его в пятеричную запись.

Далее начнём подсчитывать количество нулей и обновлять максимум. Каждый раз, когда будет найден новый максимум, будем запоминать значение x , при котором он достигнут.

Так как для сравнения используется строгий знак равенства, будет выведен наименьший возможный x .

```
mx = temp = 0
# Перебираем все возможные значения x от 1 до 2029
for x in range(1, 2030):
    # Вычисляем значение выражения в десятичной системе
    ans = 5**150 + 5**100 - x
    # Строка для хранения 5-ричной записи числа
    res = ""
    # Переводим число ans в 5-ричную систему счисления
    while ans > 0:
        # Добавляем символ, соответствующий остатку от деления на 5,
        # в начало строки
        res = str(ans % 5) + res
        # Делим число на 5 нацело
```

```
ans //= 5
# Если количество нулей превышает максимум
if res.count("0") > mx:
    mx = res.count("0") # Обновляем максимум
    temp = x # Запоминаем x
print(temp)
```

Ответ: 625

Задача №15

Задача 15.1 (Дальний восток)

Обозначим через $\text{ДЕЛ}(n, m)$ утверждение «натуральное число n делится без остатка на натуральное число m ». Задан отрезок $B = [15; 30]$. Для какого наибольшего натурального числа A формула

$$\text{ДЕЛ}(x, A) \vee (\text{ДЕЛ}(x, 23) \rightarrow \neg(x \in B))$$

тождественно истинна (т.е. принимает значение 1) при любом натуральном значении переменной x ?

Решение

Решение руками:

Для того чтобы формула была тождественно истинна, необходимо, чтобы при ложности правой части выражения:

$$(\text{ДЕЛ}(x, 23) \rightarrow \neg(x \in B)) = 0$$

левая часть выражения обязательно была истинной:

$$\text{ДЕЛ}(x, A) = 1 \text{ (то есть } x \text{ делится на } A)$$

Логическое следование (импликация) $P \rightarrow Q$ ложно только в одном случае: когда P истинно, а Q ложно. В нашем случае импликация $\text{ДЕЛ}(x, 23) \rightarrow \neg(x \in B)$ принимает значение 0, только когда одновременно:

$\text{ДЕЛ}(x, 23) = 1$ (число x делится на 23);

$\neg(x \in B) = 0$, что означает $x \in B$ (число x принадлежит отрезку $[15; 30]$).

Найдем все такие натуральные числа x , которые одновременно делятся на 23 и лежат на отрезке $[15; 30]$. На данном отрезке существует только одно число, кратное 23 – это само число 23.

Таким образом, единственным значением переменной, при котором правая часть формулы становится ложной, является $x = 23$. При всех остальных x правая часть истинна.

Чтобы вся формула оставалась истинной и при $x = 23$, левая часть $\text{ДЕЛ}(23, A)$ обязана быть истинной. То есть число 23 должно делиться на A без остатка.

Нам требуется найти наибольшее натуральное число A . Так как 23 – простое число, его наибольшим натуральным делителем является само число 23.

Решение Python:

Если хотя бы для одного x выражение ложно, то это A нам не подходит. В конце работы программы последним выведенным числом будет наибольшее подходящее A .

```
for a in range(1, 1000):
    # флаг: 0 - условие выполняется для всех x, 1 - хотя бы один случай нарушает
    c = 0
    for x in range(1, 1000):
        if ((x % a == 0) or ((x % 23 == 0) <= (not (15 <= x <= 30)))) == False:
            c = 1 # если выражение ложно, ставим флаг
```

```
break
# если флаг остался 0, выражение истинно для всех x
if c == 0:
    print(a) # выводим A
```

Ответ: 23

Задача 15.2 (Дальний восток)

Обозначим через $\text{ДЕЛ}(x, y)$ утверждение «натуральное число x делится без остатка на натуральное число y ». Для какого наибольшего натурального числа A логическое выражение

$$(\neg \text{ДЕЛ}(x, 7) \wedge \text{ДЕЛ}(x, 13)) \rightarrow (x > A - 40)$$

истинно при любом натуральном значении переменной x ?

Решение

```
# перебор возможных значений A в убывающем порядке (ищем наибольшее)
for a in range(1000, 0, -1):
    flag = True # флаг: True - формула истинна для всех x
    for x in range(1, 2000): # перебор натуральных x
        # (¬ДЕЛ(x,7) ∧ ДЕЛ(x,13)) → (x > A - 40)
        if (((x % 7 != 0) and (x % 13 == 0)) <= (x > (a - 40))) == False:
            flag = False # формула ложна, отмечаем нарушение
            break # прекращаем проверку текущего A
    if flag:
        print(a) # выводим наибольшее подходящее A
        break # прекращаем поиск
```

Ответ: 52

Задача 15.3 (Дальний восток)

Для какого наименьшего натурального числа A формула

$$(y > 10) \vee (x \cdot A > y + x)$$

тождественно истинна, то есть принимает значение 1 при любых натуральных x и y ?

Решение

```
# перебор возможных значений A в возрастающем порядке (ищем наименьшее)
for a in range(1, 1001):
    flag = True # флаг: True - формула истинна для всех x и y
    for x in range(1, 1001): # перебор натуральных x
        for y in range(1, 1001): # перебор натуральных y
            # (y > 10) (x * A > y + x)
            if ((y > 10) or (x * a > y + x)) == False:
                flag = False # формула ложна, отмечаем нарушение
                break # прекращаем проверку по y
        if not flag:
            break # прекращаем проверку по x
    if flag:
        print(a) # выводим наименьшее подходящее A
        break # прекращаем поиск
```

Ответ: 12**Задача 15.4 (Сибирь)**

Для какого наибольшего целого неотрицательного числа A логическое выражение

$$(x + y \leq 33) \vee (y \leq x + 9) \vee (y \geq A)$$

тождественно истинно (т.е. принимает значение 1) при любых целых положительных x и y ?

Решение

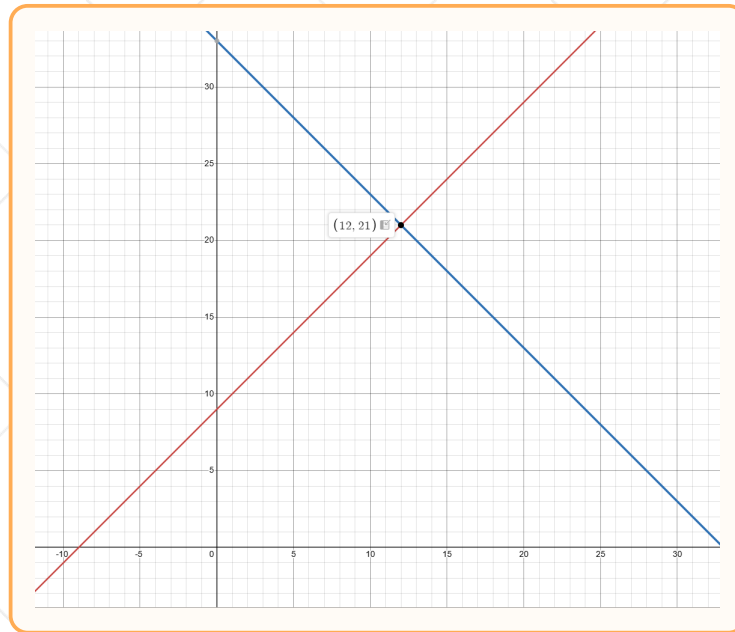
Воспользуемся методом отрицания известной части и найдем (x, y) , при которых

$$(x + y \leq 33) \vee (y \leq x + 9) = 0$$

Это выполняется, когда

$$(x + y > 33) \wedge (y > x + 9) = 1$$

Построим прямые $y = 33 - x$ и $y = x + 9$



Наименьшая возможная пара, при которых условие выполняется – (12, 22).

При этом условие $A \leq y$ должно быть истинно. Значит, A должна быть меньше или равна самому маленькому значению y . $A \leq 22$, $A_{\max} = 22$.

Решение Python

```
mx = 0 # Максимальное возможное значение A
for a in range(100): # Перебираем значения A
    # Создаём переменную-флаг, показывающую, что текущее значение A подходит
    flag = 1
    for x in range(1,100): # Перебираем значение X
        for y in range(1,100): # Перебираем значение Y
            # Если исходное выражение даёт ложь,
            if ((x+y<=33) or (y <= x+9) or (y >= a)) == 0:
                flag = 0 # присваиваем флагу значение 0,
                break # останавливаем цикл для Y
        if not flag: # Если флаг опущен (равен 0),
            break # останавливаем цикл для X
    if flag: # Если флаг поднят (равен 1),
        mx = max(mx, a) # присваиваем максимальное значение переменной mx
print(mx) # Выводим ответ на экран
```

Ответ: 22

Задача 15.5 (Урал)

Обозначим через $\text{ДЕЛ}(n, m)$ утверждение «натуральное число n делится без остатка на натуральное число m ». Задан отрезок $B = [65; 85]$. Для какого наибольшего натурального

числа A формула

$$\text{ДЕЛ}(x, A) \vee ((x \in B) \rightarrow \text{ДЕЛ}(x, 15))$$

тождественно истинна (т.е. принимает значение 1) при любом целом положительном значении переменной x ?

Решение

Для начала упростим выражение

$$\text{ДЕЛ}(x, A) \vee (x \notin B) \vee \text{ДЕЛ}(x, 15)$$

Отрицаем известную часть

$$(x \in B) \wedge \neg \text{ДЕЛ}(x, 15)$$

x , при котором известная часть ложна, принадлежит отрезку B и не делится нацело на 15. Это следующие числа: [65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85].

Нам нужно найти такое A , которое будет являться делителем каждого из данных чисел x . Такое A единственное и равно 1.

```
#создаем отрезок b
b = [x for x in range(65,86)]
#перебираем значения a
for a in range(1,100):
#создаем флаг, который будет показывать, подходит ли текущее a или нет
    flag = True
    #перебираем x
    for x in range(1,1000):
        #если условие не выполняется
        if ((x % a == 0) or (x not in b) or (x % 15 == 0)) == False:
            #меняем значение флага
            flag = False
            #сбрасываем цикл, так как a не подходит
            break
    #если флаг не поменял своего значения
    if flag == True:
        #значит a подходит
        print(a)
```

Ответ: 1

Задача 15.6 (Центр)

Обозначим через $\text{ДЕЛ}(n, m)$ утверждение «натуральное число n делится без остатка на натуральное число m ». Задан отрезок $B = [30; 40]$. Определите минимальное количество элементов множества A при котором формула

$$(x \in A) \vee (\text{ДЕЛ}(x, 12) \rightarrow (x \notin B))$$

тождественно истинна (т.е. принимает значение 1) при любом целом положительном значении переменной x ?

Решение

Для поиска значения A найдем x , при которых значение выражения зависит от результата сколки с A .

Раскроем импликацию:

$$(x \in A) \vee (\neg \text{ДЕЛ}(x, 12) \vee (x \notin B))$$

Выполним отрицание известной части:

$$\text{ДЕЛ}(x, 12) \wedge (x \in B)$$

Отрицание известной части выполнится x для кратных 12 и принадлежащих B . Это только одно число — 36.

Это x обязательно должно быть в A , следовательно, минимальное множества $A = \{36\}$ и имеет 1 элемент.

Ответ: 1

Задача №16

Задача 16.1 (Дальний восток)

Алгоритм вычисления значения функции $F(n)$, где n — целое неотрицательное число, задан следующими соотношениями:

$F(n) = n$, если $n < 10$;

$F(n) = n + F(n - 3)$, если $n \geq 10$;

Определите значение $F(2566)/F(2557)$. В ответе запишите целую часть.

Решение

```
from functools import *

@lru_cache(None)
def f(n):
    if n < 10:
        return n
    if n >= 10:
        return n + f(n-3)

# Заполняем кэш от наименьшего к наибольшему,
# поскольку при n < 10 функция вычисляется нерекурсивно
for i in range(2558):
    f(i)
print(f(2566)/f(2557))
```

Ответ: 1

Задача 16.2 (Дальний восток)

Алгоритм вычисления значения функции $F(n)$, где n — натуральное число, задан следующими соотношениями:

$F(n) = 1$, если $n = 1$;

$F(n) = n \cdot F(n - 1)$, если $n > 1$.

Определите значение $(F(3038) + 3 \cdot F(3037))/F(3036)$. В ответе запишите целую часть.

Решение

Для решения необходимо создать рекурсивную функцию, которая будет повторять логику, описанную в условии. Однако, чтобы не вызвать проблем с глубиной рекурсии, стоит добавить декоратор `lru_cache`, сохраняющий полученные значения функции, а затем "прогреть" кэш, просчитав результаты выполнения функции для числа n , начиная с базового случая (то есть при $n = 1$) и заканчивая 3038 (ближайшим числом, требуемым программой для получения ответа). В конце значение $(f(3038) + 3 \cdot f(3037))/f(3036)$ выводится на экран

```
from functools import *

@lru_cache(None) # Создаём рекурсивную функцию, повторяющую формулы из условия
def f(n):
    if n == 1:
        return 1
    if n > 1:
        return n * f(n-1)

# "Прогреваем" кэш, просчитывая значения функции для n от 1 до 3038
for i in range(1,3039):
    f(i)

print((f(3038)+3*f(3037))/f(3036)) # Выводим ответ на экран
```

Ответ: 9235517

Задача 16.3 (Дальний восток)

Алгоритм вычисления значения функции $F(n)$, где n — целое неотрицательное число, задан следующими соотношениями:

$F(n) = n$, , если $n \leq 2$;

$F(n) = F(n-2) + 3 \cdot n$, если $n > 2$ и нечетно;

$F(n) = (n+5) \cdot F(n-1)$, если $n > 2$ и четно;

Определите значение $F(2026)/F(2025)$.

Решение

Для решения необходимо создать рекурсивную функцию, которая будет повторять логику, описанную в условии. Однако, чтобы не вызвать проблем с глубиной рекурсии, стоит добавить декоратор `lru_cache`, сохраняющий полученные значения функции, а затем прогреть кэш, просчитав результаты выполнения функции для числа n , начиная с базового случая (то есть от $n \leq 2$) и заканчивая 2026 (ближайшим числом, требуемым программой для получения ответа). В конце значение $f(2026)/f(2025)$ выводится на экран.

```
from functools import lru_cache # импортируем декоратор lru_cache из functools

@lru_cache(None) # Добавляем декоратор lru_cache, чтобы записывать полученные
# значения функции
def f(n): # создаём рекурсивную функцию, повторяющую формулы из условия
    if n <= 2:
        return n
    if n % 2 != 0 and n > 2:
```



```
    return f(n - 2) + 3 * n
if n % 2 == 0 and n > 2:
    return (n + 5) * f(n - 1)

# "Прогреваем" кэш, просчитывая значения функции для n от 1 до 2026
for n in range(1, 2027):
    f(n)
print(f(2026) / f(2025)) # выводим ответ на экран
```

Ответ: 2031

Задача 16.4 (Центр)

Алгоритм вычисления значения функции $F(n)$, где n — целое неотрицательное число, задан следующими соотношениями:

$F() = 1$, , если $n = 1$;

$F(n) = n \cdot F(n - 1)$, если $n > 1$.

Определите значение $\frac{F(3489) // 2 + F(3487)}{F(3486)}$.

Решение

Решение программой

Для решения необходимо создать рекурсивную функцию, которая будет повторять логику, описанную в условии. Однако, чтобы не вызвать проблем с глубиной рекурсии, стоит добавить декоратор `lru_cache`, сохраняющий полученные значения функции, а затем «прогреть» кэш, просчитав результаты выполнения функции для числа n , начиная с базового случая (то есть при $n = 1$) и заканчивая 3489 (ближайшим числом, требуемым программой для получения ответа). В конце значение $(f(3489) // 2 + f(3487)) / f(3486)$ выводится на экран

```
from functools import lru_cache
# Создаём рекурсивную функцию, повторяющую формулы из условия
@lru_cache(None)
def f(n):
    if n == 1:
        return 1
    return n * f(n - 1)
# "Прогреваем" кэш, просчитывая значения функции для n от 1 до 3489
for i in range(1, 3490):
    f(i)
# Обязательно делим нацело, иначе будет ошибка
print((f(3489) // 2 + f(3487)) / f(3486))
```

Решение руками

Расписываем числитель:

$$\begin{aligned}\frac{f(3489)}{2} + f(3487) &= \frac{3489 \cdot 3488 \cdot f(3487)}{2} + f(3487) = \\ &= 6084816 \cdot f(3487) + f(3487) = 6084817 \cdot f(3487)\end{aligned}$$

Возвращаемся к основной дроби:

$$\frac{6084817 \cdot f(3487)}{f(3486)} = \frac{6084817 \cdot 3487 \cdot f(3486)}{f(3486)} = 6084817 \cdot 3487 = 21217756879$$

Ответ: 21217756879

Задача №17

Задача 17.1 (Дальний восток)

В файле содержится последовательность целых чисел. Определите количество пар идущих подряд элементов, в которых элементы не равны друг другу, а абсолютная разность между ними кратна минимальному положительному элементу последовательности, который кратен 9. В ответе запишите два числа: сначала количество найденных пар, затем максимальную сумму элементов таких пар.

Решение

```
f = open('1')
a = [int(x) for x in f]
# 1)Число положительно, если оно больше нуля
# 2)Число кратно 9, если остаток при делении на 9 равен 0
minim9 = min([x for x in a if x>0 and x%9==0])
c = 0
mx = -10**10
for i in range(len(a)-1):
    # 1)Элементы пары различны
    # 2)Абсолютная разность элементов кратна minim9
    if a[i] != a[i+1] and abs(a[i] - a[i+1]) % minim9 == 0:
        c += 1
        mx = max(mx, a[i]+a[i+1])
print(c, mx)
```

Ответ: 240 17914

Задача 17.2 (Дальний восток)

В файле содержится последовательность целых чисел. Определите количество пар идущих подряд элементов, в которых остаток от деления хотя бы одного элемента пары на 33 равен минимальному числу в последовательности. В ответе запишите два числа: сначала количество найденных пар, затем максимальную сумму элементов таких пар.

Решение

```
f = open('17.txt')
a = [int(x) for x in f]
# Находим минимальное число в последовательности
mn = min(a)
c = 0
mx = -10**10
```

```
for i in range(len(a)-1):
    # Остаток от деления на 33 хотя бы одного числа равен минимуму
    if a[i] % 33 == mn or a[i+1] % 33 == mn:
        c += 1
        mx = max(mx, a[i] + a[i+1])
print(c, mx)
```

Ответ: 53 19166

Задача 17.3 (Сибирь)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество пар идущих подряд элементов, в которых хотя бы одно число отрицательное, а их сумма больше количества чисел в файле, которые делятся на 27 без остатка. В ответе запишите два числа: сначала количество найденных пар, затем максимальную сумму элементов таких пар.

Решение

Сначала создадим список чисел, кратных 27, и найдём его длину. Затем для каждой пары будем проверять, что хотя бы один элемент - отрицательный, а сумма больше количества кратных 27. Количество таких пар и максимальную сумму элементов таких пар будем сохранять в отдельные переменные, после завершения перебора выведем их значения.

```
f = open('17.txt') # считываем все числа из файла в список
a = [int(x) for x in f]

cnt27 = len([x for x in a if x % 27 == 0]) # находим количество чисел, делящихся на 27
# переменная для подсчёта количества подходящих пар
k = 0
# переменная для хранения максимальной суммы подходящих пар
# (берём маленькое число для начала)
mx = -10 ** 10
# перебираем все пары подряд идущих элементов
for i in range(len(a) - 1):
    # проверяем, что хотя бы одно число пары отрицательное
    # и сумма пары больше кол-ва чисел, делящихся на 27
    if (a[i] < 0 or a[i + 1] < 0) and a[i] + a[i + 1] > cnt27:
        k += 1
        # обновляем максимальную сумму, если нашли большее значение
        if (a[i] + a[i + 1]) > mx:
            mx = a[i] + a[i + 1]
print(k, mx) # выводим количество подходящих пар и максимальную сумму их элементов
```

Ответ: 2412 9757

Задача 17.4 (Центр)

В файле содержится последовательность целых чисел от -100 000 до 100 000. Определите количество пар идущих подряд элементов, в которых ровно одно число отрицательное, а их сумма меньше количества чисел в файле, которые делятся на 23 без остатка. В ответе запишите два числа: сначала количество найденных пар, затем максимальную сумму элементов таких пар.

Решение

Сначала создадим список чисел, кратных 23, и найдём его длину. Затем для каждой пары будем проверять, что ровно один элемент – отрицательный, а сумма меньше количества кратных 23. Количество таких пар и максимальную сумму элементов таких пар будем сохранять в отдельные переменные, после завершения перебора выведем их значения.

```
# Открываем файл для чтения
f = open('17.txt')
# Читаем все строки из файла, преобразуем каждую
# в целое число и сохраняем в список a
a = [int(x) for x in f]

# Находим количество чисел в списке, которые кратны 23
cr23 = len([int(i) for i in a if int(i) % 23 == 0])
# Инициализируем счётчик пар, удовлетворяющих условию
c = 0

# Инициализируем переменную для максимальной суммы пары
mx = -10**10

# Перебираем все пары соседних элементов
for i in range(len(a)-1):
    # Проверяем два условия:
    # (a[i] < 0) != (a[i+1] < 0) - числа имеют разные знаки
    # (одно отрицательное, другое неотрицательное)
    # (a[i] + a[i+1]) < cr23 - сумма пары меньше количества чисел, кратных 23
    if ((a[i] < 0) != (a[i+1] < 0)) and (a[i] + a[i+1]) < cr23:
        # Если оба условия выполнены, увеличиваем счётчик подходящих пар
        c += 1
        # Обновляем максимальную сумму, если текущая сумма больше сохранённой
        mx = max(mx, a[i] + a[i+1])

# Выводим количество найденных пар и максимальную сумму среди них
print(c, mx)
```

Ответ: 27344 4346

Задача 17.5 (Центр)

В файле содержится последовательность целых чисел от 1 до 100 000. Определите количество пар идущих подряд элементов, у которых сумма остатков от деления на 20 равна минимальному числу в файле. В ответе запишите два числа: сначала количество найденных пар, затем максимальную сумму элементов таких пар.

Решение

Сначала создадим список всех чисел, находим минимальное число в файле. Затем проходимся по парам, для каждой будем проверять, что сумма остатков от деления на 20 равна минимальному числу в файле. Количество таких пар и максимальную сумму элементов таких пар будем сохранять в отдельные переменные, после завершения перебора выведем их значения.

```
# считываем все числа из файла в список
f = open('17_1.txt')
a = [int(x) for x in f]

# находим минимальное число в файле
mn = min(a)

# переменная для подсчёта количества подходящих пар
k = 0
# переменная для хранения максимальной суммы подходящих пар
# (берём маленькое число для начала)
mx = -10 ** 10
# перебираем все пары подряд идущих элементов
for i in range(len(a) - 1):
    # проверяем, что сумма остатков от деления
    # на 20 равна минимальному числу в файле
    if ((a[i] % 20) + (a[i + 1] % 20)) == mn:
        k += 1
        # обновляем максимальную сумму, если нашли большее значение
        if (a[i] + a[i + 1]) > mx:
            mx = a[i] + a[i + 1]

# выводим количество подходящих пар и максимальную сумму их элементов
print(k, mx)
```

Ответ: 362 19737

Задача 17.6 (Центр)

В файле содержится последовательность целых чисел. Определите количество пар идущих подряд элементов, у которых сумма элементов меньше минимального числа из последова-

тельности, которое положительно и кратно 7. В ответе запишите два числа: сначала количество найденных пар, затем абсолютное значение максимальной суммы.

Решение

Решение программой

Сначала создадим список всех чисел, затем найдём минимальное число в файле, которое положительно и кратно 7. Затем пройдемся в цикле по парам, для каждой будем проверять, что её сумма меньше минимального положительного числа файла, кратного 7.

Сумму всех подходящих пар будем сохранять в списке. В конце в качестве количества таких пар выведем длину списка, а в качестве максимальной суммы – максимальный элемент списка.

```
# Открываем файл для чтения данных
f = open("17-3.txt")
# Считываем все числа из файла и сохраняем их в список
a = [int(i) for i in f]
# Находим минимальное положительное число, кратное 7
mn_7 = min([x for x in a if x % 7 == 0 and x > 0])
# Создаём список для хранения результатов подходящих пар
ans = []
# Перебираем индексы первых элементов всех соседних пар
for i in range(len(a)-1):
    # Получаем пару из двух подряд идущих элементов
    t = a[i:i+2]
    # Проверяем выполнение условия задачи
    if sum(t) < mn_7:
        # Сохраняем сумму подходящей пары
        ans.append(sum(t))
# Выводим количество найденных пар и модуль максимального результата
print(len(ans), abs(max(ans)))
```

Ответ: 4851 13

Задача №18

Задача 18.1 (Дальний восток)

Исполнитель Робот стоит в левом верхнем углу поля, разлинованного на клетки. Он может перемещаться по клеткам, выполняя за одно перемещение одну из двух команд: вправо или вниз. По команде вправо Робот перемещается в соседнюю правую клетку; по команде вниз – в соседнюю нижнюю. Между соседними клетками квадрата также могут быть внутренние стены. Сквозь стену Робот пройти не может.

Перед каждым запуском Робота в каждой клетке квадрата лежит монета достоинством от 1 до 100. Посетив клетку, Робот забирает монету с собой; это также относится к начальной и конечной клеткам маршрута Робота.

В «угловых» клетках поля – тех, которые справа и снизу ограничены стенами, Робот не может продолжать движение, поэтому накопленная сумма считается итоговой. Таких конечных клеток на поле может быть несколько, включая правую нижнюю клетку поля. При разных запусках итоговые накопленные суммы могут различаться. Определите максимальную и минимальную денежные суммы, среди всех возможных итоговых сумм, которые может собрать Робот, пройдя из левой верхней клетки в конечную клетку маршрута.

Исходные данные записаны в файле в виде электронной таблицы, каждая ячейка которой соответствует клетке поля. В ответе запишите два числа – сначала максимальную сумму, которую может собрать Робот, затем – минимальную.

Решение

Нам дано поле 20 на 20, создадим рядом еще одно поле такого же размера (ячейки A23 : T42). В левую верхнюю клетку нового поля, записываем значение из левой верхней клетки исходного поля – 27.

Сначала просимулируем ход вправо. Для этого в ячейке B23 запишем формулу и протянем до конца строки:

$$=A23+B1$$

Теперь просимулируем ход вниз. Для этого в ячейке A24 запишем формулу и протянем до нижнего конца таблицы:

$$=A23+A2$$

Найдем максимальное значение суммы. Рассмотрим ячейку B2, в нее мы можем попасть из B1 и A2, тогда, чтобы в этой клетке суммы была максимальной, необходимо выбрать максимальную сумму из тех двух клеточек, из которых можем попасть в эту. В ячейку B24 запишем формулу:

$$=МАКС(A24;B23)+B2$$

После растягивания формулы, стенки в новом поле пропали, для того чтобы их вернуть копируем изначальное поле и на новое вставим следующим образом: ПКМ – Специальная вставка – форматирование. Стенки вернулись. Теперь в ячейках, которые находятся правее и/или ниже стенок необходимо заменить формулы. В ячейках которые ниже стенок напомним формулы по аналогии с

теми, которые мы записывали в верхней строке, а в ячейках, которые правее стенок – по аналогии с самым левым столбцом.

Заметим, что в периметр с M32 по N37 робот не сможет попасть, так как робот движется только вниз и вправо, поэтому выделим этот периметр и сотрем этот прямоугольник.

Теперь найдем ячейки, где робот может остановиться: O42, T42, S40, Q34, и выделим их красным цветом. Теперь напишем в свободной ячейке формулу:

$$= \text{МАКС}(\text{O42;Q34;S40;T42})$$

Посмотрим на нее, увидим первый ответ на задание: 2362.

Для того чтобы найти минимальное заменим все МАКС на МИН. Сделать это можно с помощью функции Найти и Заменить. Увидим значение минимума в той же ячейке, где смотрели максимум: 1205.

Ответ: 2362 1205

Задача 18.2 (Дальний восток)

Исполнитель Робот может перемещаться по клеткам квадратного поля размером $N \times N$, заполненного числами. За один ход Робот может переместиться на одну клетку вправо или вниз.

Маршрут Робота начинается в левой верхней клетке и должен обязательно завершиться в одной из нескольких финишных клеток, расположенных в самом нижнем ряду таблицы, **номера столбцов** которых делятся на 3.

Определите максимальную и минимальную денежную сумму, которую может собрать Робот, пройдя по такому маршруту. В ответе укажите сначала максимальное значение, затем минимальное через пробел.

Решение

Открываем файл в редакторе электронных таблиц. Копируем исходную таблицу и вставляем ниже только с учётом форматов, чтобы сохранить стены.

Робот стартует из левой верхней клетки. Поэтому в ячейку A22 вставляем значение из клетки A1.

Перейдём к заполнению клеток первой строки. Для этого в ячейку B22 вставляем формулу и растягиваем её вправо конца строки:

$$= \text{A22} + \text{B1}$$

Перейдём к заполнению клеток первого столбца. Для этого в ячейку A23 вставляем формулу и растягиваем её вниз до конца столбца:

$$= \text{A22} + \text{A2}$$

Остается заполнить остальную часть поля. Для этого в ячейку B23 вставляем формулу и растягиваем её на остаток поля:

$$= \text{МАКС}(\text{A23;B22}) + \text{B2}$$

После этого у нас пропали все стены, поэтому снова копируем исходную таблицу и вставляем поверх нашей только с учётом форматов.

Очищаем ячейки, в которые Робот вообще не может попасть и обозначаем «угловые» клетки – те клетки в нижнем ряду таблицы, номера столбцов которых кратны 3.

Также сменим формулы около стен. В те клетки, которые находятся правее от стены, запишем формулу из первого столбца таблицы. В клетках, которые находятся под стеной, нужно записать формулу из первой строки.

Максимальное значение будет равно максимуму из «угловых» клеток.

Для поиска минимального значения с помощью сочетания клавиш Ctrl+H заменяем МАКС на МИН и определяем его как минимальное значение из «угловых» клеток.

Задача 18.3 (Дальний восток)

Исполнитель Робот может перемещаться по клеткам квадратного поля размером $N \times N$, заполненного числами. За один ход Робот может переместиться на одну клетку вправо или вниз.

В «угловых» клетках поля – тех, которые справа и снизу ограничены стенами, Робот не может продолжать движение, поэтому накопленная сумма считается итоговой. Итоговой, может считаться только **сумма, кратная 3**, если в угловой клетке число не кратное 3, оно не рассматривается. При этом Робот не подстраивает маршрут под кратность — кратность проверяется автоматически по факту собранной суммы, и маршруты с некрatной суммой просто отбрасываются. Таких конечных клеток на поле может быть несколько, включая правую нижнюю клетку поля.

Определите максимальную и минимальную денежные суммы, среди всех возможных итоговых сумм, которые может собрать Робот, пройдя из левой верхней клетки в конечную клетку маршрута. В ответе укажите сначала максимальное значение, затем минимальное через пробел.

Решение

Открываем файл в редакторе электронных таблиц. Копируем исходную таблицу и вставляем ниже только с учётом форматов, чтобы сохранить стены.

Робот стартует из левой верхней клетки. Поэтому в ячейку A22 вставляем значение из клетки A1.

Перейдём к заполнению клеток первой строки. Для этого в ячейку B22 вставляем формулу и растягиваем её вправо конца строки:

$$= A22 + B1$$

Перейдём к заполнению клеток первого столбца. Для этого в ячейку A23 вставляем формулу и растягиваем её вниз до конца столбца:

$$= A22 + A2$$

Остается заполнить остальную часть поля. Для этого в ячейку B23 вставляем формулу и растягиваем её на остаток поля:

$$= \text{МАКС}(A23; B22) + B2$$

После этого у нас пропали все стены, поэтому снова копируем исходную таблицу и вставляем поверх нашей только с учётом форматов. Очищаем ячейки, в которые Робот вообще не может попасть и обозначаем «угловые» клетки. Также сменим формулы около стен. В те клетки, которые находятся правее от стены, запишем формулу из первого столбца таблицы. В клетках, которые находятся под стеной, нужно записать формулу из первой строки.

Максимальное значение будет равно максимуму, кратному 3, из «угловых» клеток.

Для поиска минимального значения с помощью сочетания клавиш Ctrl+H заменяем МАКС на МИН и определяем его как минимальное значение, кратное 3, из «угловых» клеток.

Задача №19-21

Задача 19.1 (Дальний восток)

Два игрока, Петя и Ваня, играют в следующую игру. Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Петя. За один ход игрок может:

- добавить в одну из куч (по своему выбору) 4 камня;
- увеличить количество камней в одной из куч (по своему выбору) в 2 раза.

Например, пусть в одной куче 20 камней, а в другой 30 камней; такую позицию в игре обозначим $(20, 30)$. Тогда за один ход можно получить любую из четырёх позиций: $(24, 30)$, $(20, 34)$, $(40, 30)$, $(20, 60)$.

Для того чтобы делать ходы, у каждого игрока есть неограниченное количество камней. Игра завершается в тот момент, когда суммарное количество камней в двух кучах становится не менее 154. Победителем считается игрок, сделавший последний ход, то есть первым получивший такую игровую позицию, при которой в двух кучах суммарно 154 камня или больше. В начальный момент в первой куче 11 камней, во второй куче – S камней; $1 \leq S \leq 142$.

Будем говорить, что игрок имеет выигрышную стратегию, если он может выиграть при любых ходах противника.

Известно, что Ваня выиграл своим первым ходом после неудачного хода Пети. Укажите минимальное значение S , при котором это возможно.

Решение

Решение руками:

Чтобы Ваня победил своим первым ходом, Петя должен совершить такой неудачный ход, после которого Ваня следующим же действием сможет набрать в сумме не менее 154 камней. Быстрее всего увеличить кучу можно с помощью удваивания. Сначала Петя удваивает вторую кучу S , а затем Ваня удваивает полученную кучу, в то время как первая куча остается без изменений (11 камней). При каких S будет выполняться неравенство $2 * 2 * S + 11 \geq 154$? $4 * S + 11 \geq 154$ $4 * S \geq 143$ $S \geq 35.75$ Минимальное целое значение S , удовлетворяющее этому условию, равно 36. Следовательно, ответ 36.

Решение программой:

Для решения на Python используем рекурсию с проверкой всех возможных ходов (+ 4, * 2), чтобы определить, чья это выигрышная позиция.

Функция возвращает 0, если игра уже завершена (камней в сумме ≥ 154), положительное число — если при оптимальной игре побеждает текущий игрок, и отрицательное — если побеждает соперник.

Так как первый ход делает Петя, цикл запускается от его лица: если значение функции положительное, выигрывает Петя, а если отрицательное — Ваня.

Если функция вернула 1, то Петя победил первым ходом. Но он походил неудачно и передал победу Ване.

При переборе возможных значений программа выводит минимум 36.

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 154: # Если камней в куче стало больше 154
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
        return -max(moves)

for s in range(1, 143):
    # Если в данной позиции после неудачного хода Пети возможен выигрыш Вани
    if game(11 + 4, s) == 1 or game(11, s + 4) == 1 \
        or game(11 * 2, s) == 1 or game(11, s * 2) == 1:
        print(s) # Вывод минимального значения S
        break
```

Ответ: 36

Задача 20.1 (Дальний восток)

Найдите два наименьших значения S , при которых у Пети есть выигрышная стратегия, причём одновременно выполняются два условия:

- Петя не может выиграть за один ход;
- Петя может выиграть своим вторым ходом независимо от ходов Вани.

Решение

Решение руками

Петя должен выиграть своим вторым ходом независимо от ходов Вани. Это означает, что Петя своим первым ходом должен получить такую позицию, из которой Ваня любым ходом создаст выигрышное положение для Пети на следующем ходу.

Таковыми позициями для Вани (с которых он вынужден подставиться под выигрыш соперника) являются позиции, сумма которых при удваивании большей кучи лежит в диапазоне от 150 до 153. Тогда любой ход Вани приведет к тому, что Петя сможет набрать в сумме 154 или более камней.

Петя может получить такие позиции своими первыми ходами:

Если Петя удваивает вторую кучу и получает (11, 70), сумма при удваивании Ваней составит 151 (диапазон от 150 до 153). Это возможно при начальном $S = 35$.

Если Петя удваивает первую кучу и получает (22, 64), сумма при удваивании Ваней составит 150 (диапазон от 150 до 153). Это возможно при начальном $S = 64$.

Следовательно, два наименьших значения S – это 35 и 64.

Решение БУ (программой):

Для решения на Python используем рекурсию с проверкой всех возможных ходов (+ 4, * 2), чтобы определить, чья это выигрышная позиция.

Функция возвращает 0, если игра уже завершена (камней в сумме ≥ 154), положительное число – если при оптимальной игре побеждает текущий игрок, и отрицательное – если побеждает соперник.

Так как первый ход делает Петя, цикл запускается от его лица: если значение функции положительное, выигрывает Петя, а если отрицательное – Ваня.

Если функция вернула 2, то Петя победил вторым ходом.

Программа перебирает и выводит значения S , в ответ берем два наименьших (два первые) числа.

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 154: # Если камней в куче стало больше 154
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
        return -max(moves)

for s in range(1, 143):
    if game(11, s) == 2:
        print(s) # Вывод минимального значения S
```

Ответ: 35, 64

Задача 21.1 (Дальний восток)

Найдите минимальное значение S , при котором:

- у Вани есть выигрышная стратегия, позволяющая ему выиграть первым или вторым ходом при любой игре Пети;
- у Вани нет стратегии, гарантирующей выигрыш первым ходом.

Решение**Решение руками**

Из предыдущей задачи 20 мы знаем, что позиция с 64 камнями во второй куче является выигрышной для того игрока, который делает из нее свой ход.

В задаче 21 первый ход делает Петя. Если он походит «+ 4 камня во вторую кучу», игра перейдет в позицию $(11, S + 4)$.

Чтобы Ваня гарантированно выиграл из этого положения, полученная куча должна стать для него выигрышной (то есть равной 64 из задачи 20): $S + 4 = 64$ $S = 60$

При $S = 60$ все остальные возможные ходы Пети (добавление 4 в первую кучу, удвоение первой или второй кучи) также ведут к гарантированной победе Вани первым или вторым ходом. Следовательно, минимальное значение S равно 60.

Решение БУ (программой):

Для решения на Python используем рекурсию с проверкой всех возможных ходов (+ 4, * 2), чтобы определить, чья это выигрышная позиция. Функция возвращает 0, если игра уже завершена (камней в сумме ≥ 154), положительное число – если при оптимальной игре побеждает текущий игрок, и отрицательное – если побеждает соперник.

Так как первый ход делает Петя, цикл запускается от его лица: если значение функции положительное, выигрывает Петя, а если отрицательное — Ваня.

Если функция вернула -2, то Ваня гарантированно побеждает своим первым или вторым ходом. Программа перебирает значения S и выводит минимальное подходящее число.

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 154: # Если камней в куче стало больше 154
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
```



```
    return -max(moves)

# Поиск минимального значения S для выигрыша Вани первым или вторым ходом
for s in range(1, 143):
    if game(11, s) == -2:
        print(s) # Вывод минимального значения S
        break
```

Ответ: 60

Задача 19.2 (Сибирь)

Два игрока, Петя и Ваня, играют в следующую игру. Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Петя. За один ход игрок может:

- добавить в одну из куч (по своему выбору) 4 камня;
- увеличить количество камней в одной из куч (по своему выбору) в 2 раза.

Например, пусть в одной куче 20 камней, а в другой 30 камней; такую позицию в игре обозначим $(20, 30)$. Тогда за один ход можно получить любую из четырёх позиций: $(24, 30)$, $(20, 34)$, $(40, 30)$, $(20, 60)$.

Для того чтобы делать ходы, у каждого игрока есть неограниченное количество камней. Игра завершается в тот момент, когда суммарное количество камней в двух кучах становится не менее 165. Победителем считается игрок, сделавший последний ход, то есть первым получивший такую игровую позицию, при которой в двух кучах суммарно 165 камня или больше. В начальный момент в первой куче 14 камней, во второй куче – S камней; $1 \leq S \leq 150$.

Будем говорить, что игрок имеет выигрышную стратегию, если он может выиграть при любых ходах противника.

Известно, что Ваня выиграл своим первым ходом после некоторого хода Пети. Укажите минимальное значение S , при котором такая ситуация возможна.

Решение

Решение программой:

Для решения на Python используем рекурсию с проверкой всех возможных ходов $(+ 4, * 2)$, чтобы определить, чья выигрышная позиция.

Функция возвращает 0, если игра уже завершена (камней в сумме ≥ 165), положительное число — если при оптимальной игре побеждает текущий игрок, и отрицательное — если побеждает соперник.

Важно учесть, что Ваня выигрывает не гарантированно, а после какого-то одного из всевозможных ходов Пети. В том числе он мог и сходить неудачно. Наша функция учитывает только правильные, «удачные» ходы, поэтому необходимо прописать все ходы Пети вручную и проверить, что Ваня из какого-либо из них выиграл за 1 ход

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 165: # Если камней в куче стало больше 165
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
        return -max(moves)

# Поиск минимального значения S для выигрыша Вани первым ходом
for s in range(1, 151):
    # Если в данной позиции после какого-то хода Пети возможен выигрыш Вани
    if game(14 + 4, s) == 1 or game(14, s + 4) == 1 \
        or game(14 * 2, s) == 1 or game(14, s * 2) == 1:
        print(s) # Вывод минимального значения S
        break
```

Ответ: 38

Задача 20.2 (Сибирь)

Найдите два наименьших значения S , при которых у Пети есть выигрышная стратегия, причём одновременно выполняются два условия:

- Петя не может выиграть за один и за два хода;
- Петя может выиграть своим третьим ходом независимо от ходов Вани.

Решение

Решение программой:

Для решения на Python используем рекурсию с проверкой всех возможных ходов (+ 4, * 2), чтобы определить, чья это выигрышная позиция.

Функция возвращает 0, если игра уже завершена (камней в сумме 165), положительное число – если при оптимальной игре побеждает текущий игрок, и отрицательное – если побеждает соперник.

Так как первый ход делает Петя, цикл запускается от его лица: если значение функции положительное, выигрывает Петя, а если отрицательное – Ваня.

Если функция вернула 3, то Петя победил третьим ходом. Программа перебирает и выводит значения S , в ответ берем два наименьших (два первые) числа.

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 165: # Если камней в куче стало больше 165
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
        return -max(moves)

# Поиск минимального значения S для выигрыша Пети третьим ходом
for s in range(1, 151):
    if game(14, s) == 3:
        print(s)
```

Ответ: 32, 59

Задача 21.2 (Сибирь)

Найдите минимальное значение S , при котором:

- у Вани есть выигрышная стратегия, позволяющая ему выиграть вторым или третьим ходом при любой игре Пети;
- у Вани нет стратегии, гарантирующей выигрыш вторым ходом.

Решение

Решение программой:

Для решения на Python используем рекурсию с проверкой всех возможных ходов (+ 4, * 2), чтобы определить, чья это выигрышная позиция. Функция возвращает 0, если игра уже завершена (камней в сумме ≥ 165), положительное число – если при оптимальной игре побеждает текущий игрок, и отрицательное – если побеждает соперник.

Так как первый ход делает Петя, цикл запускается от его лица: если значение функции положительное, выигрывает Петя, а если отрицательное – Ваня.

Если функция вернула -3, то Ваня гарантированно побеждает своим вторым или третьим ходом (но не всегда вторым).

Программа перебирает значения S и выводит минимальное подходящее число.

```
from functools import lru_cache

@lru_cache(None)
def game(first_heap, second_heap):
    if first_heap + second_heap >= 165: # Если камней в куче стало больше 165
        return 0 # Прекращаем игру
    moves = [
        game(first_heap + 4, second_heap),
        game(first_heap, second_heap + 4),
        game(first_heap * 2, second_heap),
        game(first_heap, second_heap * 2),
    ] # Генерация всех возможных ходов
    petya_win = [i for i in moves if i <= 0]
    if petya_win:
        return -max(petya_win) + 1
    else:
        return -max(moves)

# Поиск минимального значения S для выигрыша Вани вторым или третьим ходом
for s in range(1, 151):
    if game(14, s) == -3:
        print(s)
        break
```

Ответ: 57

Задача 19.3 (Центр)

Два игрока, Петя и Ваня, играют в следующую игру. Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Петя. За один ход игрок может:

- добавить в одну из куч (по своему выбору) 3 камня;
- увеличить количество камней в одной из куч (по своему выбору) в 3 раза.

Например, пусть в одной куче 20 камней, а в другой 30 камней; такую позицию в игре обозначим $(20, 30)$. Тогда за один ход можно получить любую из четырёх позиций: $(23, 30)$, $(20, 33)$, $(60, 30)$, $(20, 90)$.

Для того чтобы делать ходы, у каждого игрока есть неограниченное количество камней. Игра завершается в тот момент, когда суммарное количество камней в двух кучах становится не менее 176. Победителем считается игрок, сделавший последний ход, то есть первым получивший такую игровую позицию, при которой в двух кучах суммарно 176 камня или больше. В начальный момент в первой куче 14 камней, во второй куче – S камней; $1 \leq S \leq 161$.

Будем говорить, что игрок имеет выигрышную стратегию, если он может выиграть при любых ходах противника.

Известно, что Ваня выиграл своим первым ходом после неудачного хода Пети. Укажите

минимальное значение S , при котором такая ситуация возможна.

Решение

Решение руками

Сильнее всего количество камней может увеличить умножение на 3, значит такие ходы со стороны Пети будут максимально приближать Ваню к победе.

Из ситуации $(14, S)$ Петя может сделать $(14 \cdot 3 = 42, S)$ или $(14, 3S)$. Дальше Ване тоже будет выгоднее всего сделать умножение, если он хочет выиграть за 1 ход. Ваня может получить:

$(42 \cdot 3 = 126, S)$

$(42, 3S)$

$(14 \cdot 3 = 42, 3S)$

$(14, 9S)$

Значит получается $126 + S$, $42 + 3S$ или $14 + 9S$ камней в кучах. Найдём, при каком минимальном S какая-либо из этих сумм даст число ≥ 176 :

$$126 + S \geq 176$$

$$S \geq 50$$

$$42 + 3S \geq 176$$

$$3S \geq 134$$

$$S \geq 45$$

$$14 + 9S \geq 176$$

$$9S \geq 162$$

$$S \geq 18$$

Значит минимальный $S = 18$.

Решение программой

Напишем рекурсивную функцию, которая будет проверять все возможные ходы $(+3, \cdot 3)$ и определять характер позиции: выигрышная ли она, и за сколько ходов. Функция возвращает 0, если количество камней в кучах ≥ 176 , так как в этой ситуации, игрок уже победил. Проигрышные позиции возвращают отрицательные числа – за сколько ходов игрок проиграет, а выигрышные – положительные числа – за сколько ходов он выиграет.

Функция делает только удачные ходы, а Петя первый ход делает неудачный, значит нам нужно это сделать вместо него. Нам нужно, чтобы после него Ваня выиграл первым ходом, то есть позиция после хода Пети должна оказаться выигрышной за 1 ход.

```
from functools import lru_cache
# Добавляем кэширование, чтобы ускорить ход
@lru_cache(None)
def game(first, second):
```



```
# Условие победы
if first + second >= 176:
    return 0
# Все возможные ходы
moves = [
    game(first + 3, second),
    game(first * 3, second),
    game(first, second + 3),
    game(first, second * 3),
]
# Победные ходы
win_moves = [x for x in moves if x <= 0]

# Если есть победные ходы, то позиция выигрышная
if win_moves:
    return -max(win_moves) + 1
# иначе проигрышная
return -max(moves)

# Перебираем S
for s in range(1, 162):
    # Петя сделал неудачный ход, делаем его за него.
    # Проверяем, что после этого хода Ваня выигрывает за 1 ход
    if (
        game(14 + 3, s) == 1
        or game(14 * 3, s) == 1
        or game(14, s + 3) == 1
        or game(14, s * 3) == 1
    ):
        # Выводим минимальное подходящее S
        print(s)
        break
```

Ответ: 18

Задача 20.3 (Центр)

Найдите два наименьших значения S , при которых у Пети есть выигрышная стратегия, причём одновременно выполняются два условия:

- Петя не может выиграть за один ход;
- Петя может выиграть своим вторым ходом независимо от ходов Вани.

Решение

Решение руками



Из прошлого задания знаем, что если Петя первым ходом умножает первую кучу, то при $S \geq 45$ Ваня выигрывает своим следующим ходом, а если умножает вторую, то при $S \geq 18$. Если же Ваня не выигрывает, то он точно не захочет умножать, так как это только ускорит его проигрыш. Значит он будет только прибавлять. Тогда, чтобы успеть выиграть за 2 хода при минимальном S , Петя должен умножать, иначе игра будет слишком долгой.

Рассмотрим сначала $S < 18$. Тут игра точно будет длиться больше 3 ходов: Петя может сделать $(14, 17) \rightarrow (14, 51)$. Ваня не захочет умножать 51, так как Петя тогда выиграет, значит он к 14 прибавит 3. И в итоге игра будет долго длиться.

При $18 \leq S < 45$ Петя может умножить первую кучу первым ходом: $(14, S) \rightarrow (42, S)$. Ваня отсюда может сделать $(45, S)$ или $(42, S + 3)$. При этом ему выгоднее добавлять камни в кучу, в которой меньше камней.

При $18 \leq S \leq 42$, Ваня будет добавлять во 2-ую кучу:

$$\begin{aligned} 42 \cdot 3 + S + 3 &\geq 176 \\ S &\geq 47 \end{aligned}$$

$$\begin{aligned} 42 + 3(S + 3) &\geq 176 \\ 3(S + 3) &\geq 134 \\ S + 3 &\geq 45 \\ S &\geq 42 \end{aligned}$$

Пот $42 < S$, Ваня будет добавлять в 1-ую кучу:

$$\begin{aligned} 45 \cdot 3 + S &\geq 176 \\ S &\geq 41 \end{aligned}$$

То есть при $42 \leq S < 45$ Петя выигрывает за 2 хода при любой игре Вани. 2 минимальных значений оттуда – это 42 и 43.

Решение программой

Используем ту же функцию, что и в задании 19 и ищем выигрышную позицию за 2 хода.

```
from functools import lru_cache

# Добавляем кэширование, чтобы ускорить ход
@lru_cache(None)
def game(first, second):
    # Условие победы
    if first + second >= 176:
        return 0
    # Все возможные ходы
    moves = [
        game(first + 3, second),
        game(first * 3, second),
        game(first, second + 3),
        game(first, second * 3),
```

```
]
# Победные ходы
win_moves = [x for x in moves if x <= 0]

# Если есть победные ходы, то позиция выигрышная
if win_moves:
    return -max(win_moves) + 1
# иначе проигрышная
return -max(moves)

# Перебираем S
for s in range(1, 162):
    # Проверяем, может ли Петя выиграть за 2 хода
    if game(14, s) == 2:
        # Выводим подходящие S
        print(s)
```

Ответ: 42 43

Задача 21.3 (Центр)

Найдите минимальное значение S , при котором:

- у Вани есть выигрышная стратегия, позволяющая ему выиграть первым или вторым ходом при любой игре Пети;
- у Вани нет стратегии, гарантирующей выигрыш первым ходом.

Решение

Решение руками

Петя проигрывает, значит он захочет растянуть игру и не будет умножать. Значит Петя будет из $(14, S)$ делать $(17, S)$ или $(14, S + 3)$. При этом ему лучше прибавлять камни в кучу, в которой меньше камней, так как тогда умножения Вани будут добавлять меньше камней к сумме.

При $S < 14$ Петя будет делать $(14, S + 3)$. При максимально возможном $S = 13$ будем иметь следующее: $(14, 13) \rightarrow (14, 16) \rightarrow (14, 48) \rightarrow (17, 48) \rightarrow (17, 144)$, то есть Ваня не успеет победить за 2 хода, а при меньших S тем более.

Значит $S \geq 14$ и Петя прибавляет камни в первую кучу. После того как Петя сделал первый ход, мы имеем $(17, S)$ и ход Вани. А это уже задача, аналогичная 20, только Петя и Ваня поменялись местами.

Найдём, когда Ваня может умножить первую кучу. То есть Петя потом не выиграет одним ходом, при этом оптимальным таким ходом для него было бы умножение. Значит они оба умножат одну и ту же кучу.

$$\begin{aligned}17 \cdot 9 + S &< 176 \\ S &< 23\end{aligned}$$

$$17 + 9S < 176$$

$$9S < 159$$

$$S < 18$$

То есть первую кучу можно умножать при $S < 23$, а вторую при $S < 18$.

Рассмотрим $14 \leq S < 18$. Ваня здесь сделает $(17, 3S)$, потом Петя прибавит в 1-ую кучу: $(20, 3S)$, а Ваня ещё раз умножит 2-ую кучу, потому что в ней больше камней, тогда он победит при:

$$20 + 9S \geq 176$$

$$9S \geq 156$$

$$S \geq 18$$

Это невозможно.

Рассмотрим $18 \leq S < 23$. Ваня сделает $(42, S)$, Петя – $(42, S + 3)$, а Ваня потом умножит 1-ую кучу.

$$42 \cdot 3 + S + 3 \geq 176$$

$$S \geq 47$$

Такое тоже невозможно. Значит Ваня первым ходом прибавляет камни в кучу, а $S \geq 23$. То есть из $(17, S)$ он может сделать $(20, S)$ или $(17, S + 3)$. Петя умножит только если он из-за этого победит, пока будем считать, так что пока будем считать, что он обязательно проиграет и будет добавлять камни в меньшую кучу. Меньшая будет точно 1-ая. Тогда Петя может сделать $(23, S)$ или $(20, S + 3)$. А Ваня тогда должен победить, умножив большую кучу: это точно 2-ая.

$$23 + 3S \geq 176$$

$$3S \geq 153$$

$$S \geq 51$$

$$20 + 3(S + 3) \geq 176$$

$$3(S + 3) \geq 156$$

$$S + 3 \geq 52$$

$$S \geq 49$$

Проверим теперь, может ли при $S = 49$ Петя выиграть 2-ым ходом. $(17, 49 + 3)$, даже если Петя умножит 2-ую кучу, будет только 173 камня. Значит 49 – ответ.

Решение программой

Используем ту же функцию, что и в задании 19 и ищем проигрышную позицию за 2 хода.

```
from functools import lru_cache
```

```
# Добавляем кэширование, чтобы ускорить ход
```

```
@lru_cache(None)
def game(first, second):
    # Условие победы
    if first + second >= 176:
        return 0
    # Все возможные ходы
    moves = [
        game(first + 3, second),
        game(first * 3, second),
        game(first, second + 3),
        game(first, second * 3),
    ]
    # Победные ходы
    win_moves = [x for x in moves if x <= 0]

    # Если есть победные ходы, то позиция выигрышная
    if win_moves:
        return -max(win_moves) + 1
    # иначе проигрышная
    return -max(moves)

# Перебираем S
for s in range(1, 162):
    # Проверяем, может ли Ваня выиграть за 2 хода
    if game(14, s) == -2:
        # Выводим минимальное подходящее S
        print(s)
        break
```

Ответ: 49

Задача №22

Задача 22.1 (Дальний восток)

В файле содержится информация о совокупности N вычислительных процессов, которые могут выполняться параллельно или последовательно. Приостановка выполнения процесса не допускается. Будем говорить, что процесс B зависит от процесса A , если для выполнения процесса B необходимы результаты выполнения процесса A . В этом случае процессы A и B могут выполняться только последовательно.

Информация о процессах представлена в файле в виде таблицы. В первом столбце таблицы указан идентификатор процесса (ID), во втором столбце таблицы – время его выполнения в миллисекундах, в третьем столбце перечислены с разделителем «;» ID процессов, от которых зависит данный процесс. Если процесс независимый, то в таблице указано значение 0.

Определите **максимальное** количество процессов, которые начнут выполняться не ранее 15-й секунды (время начала 15 секунд также учитывается).

Типовой пример организации данных в файле

ID процесса B	Время выполнения процесса B (мс)	ID процесса(-ов) A
1	3	0
2	4	1
3	2	2;4
4	5	0
5	8	1;4
6	3	1

Для приведённой таблицы процесс 3 начинается на 8-й мс, заканчивается на 9-й мс.

Типовой пример имеет иллюстративный характер. Для выполнения задания используйте данные из прилагаемого файла.

Решение

Для начала откроем файл электронной таблицы и разделим процессы в столбце C при помощи функции "Данные по столбцам" (разделитель — точка с запятой). На месте пустых ячеек с номерами процессов и в самом низу столбца A необходимо вписать 0, дабы функции ВПР() и МАКС() корректно находили искомые данные. Теперь в ячейке F2 запишем и растянем вниз следующую формулу:

$$= \text{ВПР}(C2; \$A:\$I; 9; 0)$$

Необходимо дополнительно протянуть данную формулу от столбца F до H, чтобы найти перенести время выполнения каждого процесса. Далее в ячейке I2 запишем и растянем данную формулу:

$$= \text{МАКС}(F2:H2) + B2$$

Таким образом мы получим время завершения каждого процесса в таблице. Теперь необходимо посчитать время начала каждого процесса — для этого в ячейке J2 запишем и растянем следующую формулу:

$$= I2 - B2 + 1$$

Остаётся посчитать количество процессов, которые начнутся не ранее, чем с 15 секунды — в ячейке K2 запишем и растянем данную формулу:

$$= \text{ЕСЛИ}(J2 \geq 15; 1; 0)$$

Выделим столбец J и найдём сумму его значений, получив тем самым ответ.

Задача 22.2 (Сибирь)

В файле содержится информация о совокупности N вычислительных процессов, которые могут выполняться параллельно или последовательно. Приостановка выполнения процесса не допускается. Будем говорить, что процесс B зависит от процесса A , если для выполнения процесса B необходимы результаты выполнения процесса A . В этом случае процессы A и B могут выполняться только последовательно. Все независимые процессы запускаются в начальный момент времени. Если процесс B получает данные от процесса A , то выполнение процесса B начинается сразу же после завершения процесса A . Количество одновременно выполняемых процессов может быть любым, длительность процесса не зависит от других параллельно выполняемых процессов.

Информация о процессах представлена в файле в виде таблицы. В первом столбце таблицы указан идентификатор процесса (ID), во втором столбце таблицы – время его выполнения в миллисекундах, в третьем столбце перечислены с разделителем «;» ID процессов, от которых зависит данный процесс. Если процесс независимый, то в таблице указано значение 0.

Определите количество активных процессов на 13 мс после запуска первого процесса.

Типовой пример организации данных в файле

ID процесса B	Время выполнения процесса B (мс)	ID процесса(-ов) A
1	3	0
2	4	1
3	2	2;4
4	5	0
5	8	1;4
6	3	1

Для приведённой таблицы процесс 3 начинается на 8-й мс, заканчивается на 9-й мс.

Типовой пример имеет иллюстративный характер. Для выполнения задания используйте данные из прилагаемого файла.

Решение

Для начала откроем файл электронной таблицы и разделим процессы в столбце C при помощи функции "Данные по столбцам" (разделитель — точка с запятой). На месте пустых ячеек с номерами процессов и в самом низу столбца A необходимо вписать 0, дабы функции ВПР() и МАКС() корректно находили искомые данные. Теперь в ячейке F2 запишем и протянем вниз следующую формулу:

$$= \text{ВПР}(C2; \$A:\$I; 9; 0)$$

Необходимо дополнительно протянуть данную формулу от столбца F до H, чтобы найти перенести время выполнения каждого процесса. Далее в ячейке I2 запишем и протянем вниз данную формулу:

$$= \text{МАКС}(F2:H2) + B2$$

Таким образом мы получим время завершения каждого процесса в таблице. Теперь необходимо посчитать время начала каждого процесса — для этого в ячейке J2 запишем и протянем вниз следующую формулу:

$$= I2 - B2 + 1$$

Остаётся посчитать количество процессов, которые активны на 13 мс после запуска совокупности процессов — в ячейке K2 запишем и протянем вниз данную формулу:

$$= \text{ЕСЛИ}(\text{И}(J2 \leq 13; I2 >= 13); 1; 0)$$

Выделим столбец J и найдём сумму его значений, получив тем самым ответ.

Задача 22.3 (Центр)

В файле содержится информация о совокупности N вычислительных процессов, которые могут выполняться параллельно или последовательно. Приостановка выполнения процесса не допускается. Будем говорить, что процесс B зависит от процесса A , если для выполнения процесса B необходимы результаты выполнения процесса A . В этом случае процессы A и B могут выполняться только последовательно. Все независимые процессы запускаются в начальный момент времени. Если процесс B получает данные от процесса A , то выполнение процесса B начинается сразу же после завершения процесса A . Количество одновременно выполняемых процессов может быть любым, длительность процесса не зависит от других параллельно выполняемых процессов.

Информация о процессах представлена в файле в виде таблицы. В первом столбце таблицы указан идентификатор процесса (ID), во втором столбце таблицы — время его выполнения в миллисекундах, в третьем столбце перечислены с разделителем «;» ID процессов, от которых зависит данный процесс. Если процесс независимый, то в таблице указано значение 0.

Определите минимальное время, в которое закончат свою работу 18 процессов.

Типовой пример организации данных в файле

ID процесса B	Время выполнения процесса B (мс)	ID процесса(-ов) A
1	3	0
2	4	1
3	2	2;4
4	5	0
5	8	1;4
6	3	1

Для приведённой таблицы процесс 3 начинается на 8-й мс, заканчивается на 9-й мс.

Типовой пример имеет иллюстративный характер. Для выполнения задания используйте данные из прилагаемого файла.

Решение

Для начала откроем файл электронной таблицы и удалим значения в первой строчке, затем разделим процессы в столбце С при помощи функции "Данные по столбцам" (разделитель — точка с запятой). На месте пустых ячеек с номерами процессов и в самом верху столбца А необходимо вписать 0, дабы функция ВПР() корректно находили искомые данные. Теперь в ячейке Е2 запишем и протянем вниз следующую формулу:

=ВПР(С2;А:G;7;0)

Необходимо дополнительно протянуть данную формулу от столбца Е до F, чтобы перенести время выполнения каждого процесса. Далее в ячейке G2 запишем и протянем вниз данную формулу:

=B2+МАКС(E2:F2)

Теперь отсортируем столбец G по возрастанию для того чтобы мы видели во сколько завершаются самые первые процессы. После этого в ячейке G19 будет записано минимальное время выполнения 18 процессов.

Ответ: 29

Задача №23

Задача 23.1 (Дальний восток)

Исполнитель Робот преобразует число, записанное на экране. У исполнителя есть две команды, которым присвоены номера:

1. Прибавь 1;
2. Поменять цифры в разряде единиц и десятков местами, если разряд десятков меньше разряда единиц.

Сколько есть программ, которые преобразуют число 100 в число 150?

Решение

Создаём функцию, которая будет составлять траекторию. x – текущая позиция, y – целевая. Если мы уже превысили конец ($x > y$), возвращаем 0, путь невозможен. Если дошли до конца, то всё хорошо, возвращаем 1. Если ни в один из этих случаев не зашли, то путь продолжается, делаем ходы. Для трёхзначного числа индекс 0 отвечает за разряд сотен, 1 – за десятки, 2 – за единицы. ВАЖНО: это актуально только для трёхзначного числа! Далее переводим наше текущее число в строку, берём срезом разряд десятков и единиц, сравниваем. Если разряд единиц больше, меняем их местами (не забывая про ход + 1), иначе только прибавляем 1

```
def f(x, y):
    if x > y:
        return 0
    if x == y:
        return 1
    # str(x)[0] – первый разряд, сотни
    # str(x)[1] – десятки
    # str(x)[2] – единицы
    if str(x)[1] < str(x)[2]:
        return f(x + 1, y) + f(int(str(x)[0] + str(x)[2] + str(x)[1]), y)
    else:
        return f(x + 1, y)

print(f(100, 150))
```

Ответ: 35

Задача 23.2 (Дальний восток)

Исполнитель преобразует число на экране. У исполнителя есть две команды:

1. Вычти 1
2. Найди целую часть от деления на 2

Сколько существует программ, которые преобразуют исходное число 40 в число 6, при этом траектория вычислений обязательно содержит число 15?

Решение

Создаём функцию, которая будет считать количество траекторий из x в y , где x — текущая позиция, y — целевая. Если программа получила число, меньшее y , то возвращаем 0, ведь дальше получить искомым путь невозможно. Если программа дошла до конца, получив y , то необходимо засчитать данную траекторию, вернув 1. Если ни в один из этих случаев не зашли, то рекурсия продолжается — выполняем доступные команды. В конце, чтобы учесть наличие числа 15 в каждой траектории, необходимо посчитать количество путей из 40 в 15, а затем перемножить его на количество путей из 15 в 6.

```
def f(x, y): # Задаём рекурсивную функцию для поиска количества траектория
    if x < y: # Если текущее значение "x" меньше числа, до которого нужно найти,
        return 0 # возвращаем нулевое количество траекторий
    if x == y: # Если текущее значение "x" достигло искомого числа,
        return 1 # возвращаем единицу, засчитывая тем самым данную траекторию
    # Возвращаем суммарное количество возможных траекторий
    return f(x - 1, y) + f(x // 2, y)

# Находим количество траекторий из 40 в 15 и перемножаем их на количество
# траекторий из 15 в 6, учитывая тем самым обязательное наличие числа 15
# в траектории от 40 до 6
print(f(40, 15) * f(15, 6))
```

Ответ: 60

Задача 23.3 (Дальний восток)

Исполнитель преобразует число на экране. У исполнителя есть две команды, которые обозначены номерами:

1. Прибавь 1
2. Поменяй местами

Первая из этих команд увеличивает число на экране на 1. Вторая команда может применяться только к числу, у которого цифра разряда десятков по значению меньше цифры, стоящей в разряде единиц, и действует, заменяя число на экране числом, в котором цифры двух младших разрядов поменяны местами. Программа для исполнителя — это последовательность команд.

Сколько существует программ, для которых при исходном числе 110 результатом является число 154?

Решение

```
def f(x, y):
    if x > y:
        return 0
    if x == y:
```

```
        return 1
    # str(x)[0] - первый разряд, сотни
    # str(x)[1] - десятки
    # str(x)[2] - единицы
    if str(x)[1] < str(x)[2]:
        return f(x + 1, y) + f(int(str(x)[0] + str(x)[2] + str(x)[1]), y)
    else:
        return f(x + 1, y)

print(f(110, 154))
```

Ответ: 34

Задача 23.4 (Сибирь)

Исполнитель Робот преобразует число, записанное на экране. У исполнителя есть две команды, которым присвоены номера:

1. Прибавь 1;
2. Поменять цифры в разряде единиц и десятков местами, если разряд десятков меньше разряда единиц.

Сколько есть программ, которые преобразуют число 100 в число 144?

Решение

Создаём функцию, которая будет составлять траекторию. x - текущая позиция, y - целевая. Если мы уже превысили конец ($x > y$), возвращаем 0, путь невозможен. Если дошли до конца, то всё хорошо, возвращаем 1. Если ни в один из этих случаев не зашли, то путь продолжается, делаем ходы. Для трёхзначного числа индекс 0 отвечает за разряд сотен, 1 - за десятки, 2 - за единицы. ВАЖНО: это актуально только для трёхзначного числа! Далее переводим наше текущее число в строку, берём срезом разряд десятков и единиц, сравниваем. Если разряд единиц больше, меняем их местами (не забывая про ход + 1), иначе только прибавляем 1

```
def f(x, y):
    if x > y:
        return 0
    if x == y:
        return 1
    # str(x)[0] - первый разряд, сотни, str(x)[1] - десятки, str(x)[2] - единицы
    if str(x)[1] < str(x)[2]:
        return f(x + 1, y) + f(int(str(x)[0] + str(x)[2] + str(x)[1]), y)
    else:
        return f(x + 1, y)

print(f(100, 144))
```

Ответ: 34

Задача №24

Задача 24.1 (Дальний восток)

Текстовый файл состоит не более чем из 10^6 символов и содержит только десятичные цифры, а также знаки «+» и «*» (сложения и умножения). Определите максимальное количество символов в непрерывной последовательности, являющейся корректным арифметическим выражением с целыми неотрицательными числами (без знака). В этом выражении никакие два знака арифметических операций не стоят рядом. В записи чисел отсутствуют незначащие (ведущие) нули. В ответе укажите количество символов в найденном выражении.

Решение

Для решения данной задачи воспользуемся регулярными выражениями. В начале необходимо записать шаблон числа: "[1-9][0-9]*|0", где "[1-9]" — первая цифра, "[0-9]*" — остальные цифры в числе, "0" — "или 0". После этого записываем общий шаблон арифметического выражения: "([1-9][0-9]*|0)([+*]([1-9][0-9]*|0))+", где "([1-9][0-9]*|0)" — первое число, "[+*]([1-9][0-9]*|0)+" — число со знаком, повторяющееся 1 и более раз. Остаётся перебрать все подстроки при помощи функции `finditer()` и найти длину самого длинного арифметического выражения.

```
from re import * # импортируем модуль re для работы с регулярными выражениями

s = open("test1.txt").readline() # открываем файл и считываем всю строку
patterns = "([1-9][0-9]*|0)([+*]([1-9][0-9]*|0))+" # создаём регулярное
# выражение под условие задачи
max_len = 0 # создаём переменную, в которую будет записан наш ответ
for i in finditer(patterns, s): # перебираем все найденные выражения
    max_len = max(max_len, len(i.group())) # записываем максимальную длину
print(max_len) # выводим ответ на экран
```

Ответ: 62

Задача 24.2 (Центр)

Текстовый файл состоит не более чем из 10^6 символов и содержит только цифры 0, 4, 5, 6, 7, а также знаки «-» и «*» (вычитание и умножение). Определите максимальное количество символов в непрерывной последовательности, являющейся корректным арифметическим выражением с целыми неотрицательными числами (без знака). В этом выражении никакие два знака арифметических операций не стоят рядом. В записи чисел отсутствуют незначащие (ведущие) нули и число 0 не имеет знака. В ответе укажите количество символов в найденном выражении.

Решение

Для решения данной задачи воспользуемся регулярными выражениями. В начале необходимо записать шаблон числа: `"0|[4567][04567]*"`, где `"[4567]"` — первая цифра, `"[04567]*"` — остальные цифры в числе, `0` — "или 0". После этого записываем общий шаблон арифметического выражения: `"(0|[4567][04567]*)([-*](0|[4567][04567]*))+"`, где `"(0|[4567][04567]*)"` — первое число, `"([-*](0|[4567][04567]*))+"` — число со знаком, повторяющееся 1 и более раз. Остаётся перебрать все подстроки при помощи функции `finditer()` и найти длину самого длинного арифметического выражения.

```
from re import * # Импортируем все функции из модуля re
f = open("24-2.txt") # Открываем файл для чтения
s = f.readline() # Читаем строку из файла
# Регулярное выражение для поиска выражений с цифрами 0, 4, 5, 6, 7
p = "(0|[4567][04567]*)([-*](0|[4567][04567]*))+"
# Переменная для хранения максимальной длины найденного выражения
mx = 0
res = "" # Переменная для хранения самого длинного выражения
# Ищем все совпадения с паттерном в строке s
for i in finditer(p, s):
    t = i.group() # Получаем найденную подстроку (выражение)
    # Если текущее выражение длиннее,
    # обновляем максимальную длину
    mx = max(mx, len(t))

print(mx) # Находим значение выражения
```

Ответ: 129

Задача №25

Задача 25.1 (Дальний восток)

Напишите программу, которая перебирает целые числа, большие 7 513 048, в порядке возрастания и ищет среди них числа, представленные в виде произведения ровно двух простых множителей, не обязательно различных, каждый из которых содержит в своей записи хотя бы одну цифру 1 и хотя бы одну цифру 6.

В ответе для первых 5 найденных чисел запишите само число и наибольший из его простых множителей в соответствующие столбцы таблицы.

Решение

Сначала вводится функция `divisors(n)`, которая находит все нетривиальные делители числа, то есть все, кроме 1 и самого числа.

Дополнительно вводится функция `prime(x)`, которая проверяет число на простоту. Она перебирает возможные делители от 2 до корня: если находится хотя бы один делитель, число нам не подходит, возвращаем ложь.

Далее перебираются все числа x , большие 7 513 048. Проверяем, что нашлось 1 или 2 делителя: `len(d) == 1 or len(d) == 2`. Затем проверяем, что оба делителя простые: `all(prime(x) for x in d)`.

После этого проверяется второе условие: у каждого найденного делителя в записи должно быть хотя бы по одной цифре 1 и 6.

Как только таких чисел становится 5, перебор завершается, ответ получен.

```
def divisors(n):
    # функция для поиска нетривиальных делителей
    divs = set() # заводим множество делителей
    for i in range(2, int(n ** 0.5) + 1):
        # если i - делитель
        if n % i == 0:
            # добавляем i и его парный делитель
            divs.add(i)
            divs.add(n // i)
    return divs
```

```
def prime(x):
    # функция для проверки на простоту
    for j in range(2, int(x ** 0.5) + 1):
        # если нашли хотя бы один делитель
        # кроме 1 и себя - число не простое
        if x % j == 0:
            return False
    # не забываем, что 1 - не простое число
```



```
return x > 1

cnt = 0 # счётчик выведенных чисел
for x in range(7_513_048 + 1, 10_000_000):
    d = divisors(x)
    # если получили 1 повторяющийся делитель или два
    if (len(d) == 1 or len(d) == 2) and all(prime(x) for x in d):
        # проверяем, что все делители содержат 1 и 6
        if all('1' in str(x) and '6' in str(x) for x in d):
            print(x, max(d))
            cnt += 1
            if cnt == 5:
                # вывели 5 чисел - остановили цикл
                break
```

Ответ:

7513309 123169
7514761 16301
7514789 46103
7514909 11369
7517219 12263

Задача 25.2 (Сибирь)

Напишите программу, которая перебирает целые числа, большие 1 103 285 717, в порядке возрастания и ищет среди них числа, представленные в виде произведения ровно двух простых множителей, не обязательно различных, каждый из которых содержит ровно один раз в своей записи последовательность цифр «16». В ответе запишите в первом столбце таблицы первые пять найденных чисел в порядке возрастания, во втором столбце – для каждого из них наименьший найденный множитель для каждого из них.

Решение

Сначала вводится функция `is_prime(n)`, которая проверяет число на простоту. Она перебирает возможные делители от 2 до корня из числа: если находится хотя бы один делитель, число нам не подходит, возвращаем ложь.

Далее задается переменная `limit = 1000000` – это граница, до которой мы ищем простые числа. Создается список `pr16`, в который собираются все простые числа от 2 до `limit`, у которых в записи комбинация '16' встречается ровно один раз. Для этого используется генератор списка с двумя условиями: `is_prime(num)` и `str(num).count("16") == 1`.

После этого мы перебираем все возможные пары чисел из списка `pr16`. Перебор организован так, что $i \leq j$, чтобы не дублировать пары. Для каждой пары вычисляется произведение.

Проверяем, что произведение больше заданного порога 1103285717. Если условие выполняется, добавляем в список `result` пару `[product, min(pr16[i], pr16[j])]`, где `product` – само число, а `min` –

меньший из двух множителей. А также прерываем цикл, так как остальные произведения при переборе будут больше.

Когда все пары перебраны, сортируем список result по возрастанию произведения.

Как только таких чисел становится 5, выводим их на экран и завершаем программу.

```
def is_prime(n):
    # Проверка на простоту: перебираем делители до sqrt(n)
    for i in range(2, int(n**0.5) + 1):
        # Если делитель найден, возвращаем False
        if n % i == 0:
            return False
    # Если делителей не было, число простое
    return n > 1

limit = 1103285717 // 163

# Список простых чисел, содержащих "16" ровно один раз
pr16 = [num for num in range(2, limit)
        if is_prime(num) and str(num).count("16") == 1]

result = [] # Список для подходящих чисел

# Перебор всех пар (i <= j) для уникальных комбинаций
for i in range(len(pr16)):
    for j in range(i, len(pr16)):
        # Произведение двух простых чисел
        product = pr16[i] * pr16[j]
        # Фильтр: ищем числа больше 1 103 285 717
        if product > 1103285717:
            # Сохраняем [произведение, меньший множитель]
            result.append([product, min(pr16[i], pr16[j])])
            break # Прерываем внутренний цикл

result.sort() # Сортируем по возрастанию
# Выводим первые 5 чисел
for num in result[:5]:
    print(num)
```

Ответ:

```
1103299319 1693
1103309477 1693
1103322107 16187
1103323021 1693
1103328547 3169
```

Задача 25.3 (Урал)

Напишите программу, которая перебирает целые числа, большие 4 501 347 296, в порядке возрастания и ищет среди них числа, представленные в виде произведения ровно двух простых множителей, не обязательно различных, каждый из которых содержит ровно один раз в своей записи последовательность цифр «53».

В ответе запишите в первом столбце таблицы первые пять найденных чисел в порядке возрастания, во втором столбце – для каждого из них наименьший найденный множитель для каждого из них.

Решение

Сначала вводится функция `is_prime(n)`, которая проверяет число на простоту. Она перебирает возможные делители от 2 до корня из числа: если находится хотя бы один делитель, число нам не подходит, возвращаем ложь.

Далее вводится функция `min_prime_div(n)`, которая находит минимальный простой делитель числа, состоящего из двух простых множителей, содержащих в своей записи ровно по одной последовательности цифр "53". Для этого она проверяет первые встретившиеся делители: если они не удовлетворяют условию, то функция возвращает -1.

В конце остаётся перебрать числа, превышающие 4501347296, и выбрать из них первые 5, для которых `min_prime_div(x) > 0` (то есть число подходит и минимальный подходящий делитель существует).

```
def is_prime(n): # Функция проверки числа на простоту
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        if n % i == 0: # Если число n поделилось на i,
            return False # возвращаем ложь - оно не простое
    # Возвращаем истину, если число не является 1, при этом цикл выше закончился
    return n > 1

def min_prime_div(n): # Функция нахождения минимального множителя в подходящем числе
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        # Если число n поделилось на i, то проверяем условие
        # на наличие "53" в каждом делителе
        if n % i == 0:
            if str(i).count("53") == str(n // i).count("53") == 1:
                # Если оба делителя оказались простыми числами,
                if is_prime(i) and is_prime(n // i):
                    return i # возвращаем наименьший из делителей
            # возвращаем -1, если число i оказалось делителем,
            # но какие-то условия не выполнились
            return -1
    # возвращаем -1, если число оказалось простым,
    # из-за чего никакие делители не были найдены
```

```
return -1
```

```
c = 0 # Задаём счётчик количества выведенных чисел
# Перебираем числа, превышающие 4501347296
for x in range(4_501_347_296 + 1, 5_000_000_000):
    t = min_prime_div(x) # Находим искомый минимальный простой множитель
    if t > 0: # Если число удовлетворяет условию и данный множитель существует,
        # выводим само число вместе с минимальным множителем в качестве ответа
        print(x, t)
        c += 1 # увеличиваем счётчик на 1
    if c == 5: # Если количество выведенных чисел равно 5,
        break # останавливаем перебор
```

***Вариант решения от БУ:**

```
def prime(x):
    for i in range(2, int(x**0.5)+1):
        if x % i == 0:
            return False
    return x > 1

n = 8501347296+1
primes = [i for i in range(53, int(n**0.5)+1) if str(i).count('53') == 1 and prime(i)]

for x in range(n, n+1000000):
    c = set()
    for i in range(53, int(x**0.5)+1):
        if x % i == 0:
            if not i in primes or str(i).count('53') != 1 or
               str(x // i).count('53') != 1:
                break
            elif prime(x // i):
                c.add(i)

    if c:
        print(x, *c)
```

Ответ:

```
4501351109 53
4501367009 53
4501371143 53
4501382209 50153
4501382909 53
```

Задача 25.4 (Центр)

Напишите программу, которая перебирает целые числа, большие 2 018 974 440, в порядке возрастания и ищет среди них числа, представленные в виде произведения ровно двух простых множителей, не обязательно различных, каждый из которых содержит ровно один раз в своей записи последовательность цифр «43».

В ответе запишите в первом столбце таблицы первые пять найденных чисел в порядке возрастания, во втором столбце – для каждого из них наименьший найденный множитель для каждого из них.

Решение

Сначала вводится функция `is_prime(n)`, которая проверяет число на простоту. Она перебирает возможные делители от 2 до корня из числа: если находится хотя бы один делитель, то число нам не подходит, возвращаем ложь.

Далее вводится функция `min_prime_div(n)`, которая находит минимальный простой делитель числа, состоящего из двух простых множителей, содержащих в своей записи ровно по одной последовательности цифр «43». Для этого она проверяет первые встретившиеся делители: если они не удовлетворяют условию, то функция возвращает -1.

В конце остаётся перебрать числа, превышающие 2018974440, и выбрать из них первые 5, для которых `min_prime_div(x) > 0` (то есть число подходит и минимальный подходящий делитель существует).


```
def is_prime(n): # Функция проверки числа на простоту
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        if n % i == 0: # Если число n поделилось на i,
            return False # возвращаем ложь - оно не простое
    # Возвращаем истину, если число не является 1, при этом цикл выше закончился
    return n > 1

def min_prime_div(n): # Функция нахождения минимального множителя в подходящем числе
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        # Если число n поделилось на i, то проверяем условие
        # на наличие "43" в каждом делителе
        if n % i == 0:
            if str(i).count("43") == str(n // i).count("43") == 1:
                # Если оба делителя оказались простыми числами,
                if is_prime(i) and is_prime(n // i):
                    return i # возвращаем наименьший из делителей
            # возвращаем -1, если число i оказалось делителем,
            # но какие-то условия не выполнились
            return -1
    # возвращаем -1, если число оказалось простым,
    # из-за чего никакие делители не были найдены
    return -1

c = 0 # Задаём счётчик количества выведенных чисел
# Перебираем числа, превышающие 2018974440
for x in range(2_018_974_440 + 1, 5_000_000_000):
    t = min_prime_div(x) # Находим искомый минимальный простой множитель
    if t > 0: # Если число удовлетворяет условию и данный множитель существует,
        # выводим само число вместе с минимальным множителем в качестве ответа
        print(x, t)
        c += 1 # увеличиваем счётчик на 1
    if c == 5: # Если количество выведенных чисел равно 5,
        break # останавливаем перебор
```

Ответ:

2018977769 27143
2018980091 9437
2018983349 643
2018997619 43
2019003907 4643

Задача 25.5 (Центр)

Пусть M – сумма наименьшего и наибольшего простых делителей числа.

Напишите программу, которая перебирает целые числа, превышающие 8 007 000 000, такие что для них число M простое, больше 80000 и содержит ровно один раз в своей записи последовательность цифр «567».

В ответе запишите первые пять найденных чисел в порядке возрастания.

Решение

Опишем логику поиска числа M с помощью функции $f(x)$: для того чтобы сумма наименьшего и наибольшего простых делителей числа была простой, мы должны взять один четный и один нечетный делитель (если взять делители одной четности, их сумма будет чётна). Единственный и минимальный чётный простой делитель – это 2. Значит, нам нужно найти максимальный нечетный простой делитель. Поэтому среди пар делителей числа ищется наибольший простой делитель: перебираются делители от 2 до $\sqrt{x}$, и при нахождении делителя проверяется простота парного ему числа $x // i$. Первый найденный простой парный делитель будет максимальным простым делителем числа, так как перебор идёт в порядке возрастания i . После этого функция возвращает сумму минимального и максимального простых делителей. В конце останется проверить, что эта сумма превышает 80000, является простой и содержит последовательность цифр «567».

```
def is_prime(n): # Функция проверки числа на простоту
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        if n % i == 0: # Если число n поделилось на i,
            return False # возвращаем ложь - оно не простое
    # Возвращаем истину, если число не является 1, при этом цикл выше закончился
    return n > 1

def find_m(n): # Функция нахождения числа M
    # Задаём начальные значения
    min_d = 2 # минимального
    max_d = 0 # и максимального простых делителей
    for i in range(2, int(n ** 0.5) + 1): # Перебираем возможные делители числа n
        if n % i == 0: # Если число n поделилось на i,
            # то проверяем основной делитель и парный ему на простоту:
            if is_prime(n // i): # Если n // i - простое число,
                # записываем его в качестве максимального простого делителя
                max_d = n // i
                break
    return min_d + max_d

c = 0 # Задаём счётчик количества выведенных строк
for x in range(8_007_000_000, 10_000_000_000, 2):
    m = find_m(x) # Находим число M
```

```
# Если m больше 80000 и содержит 567 и является простым числом,  
if m > 80000 and is_prime(m) and str(m).count("567") == 1:  
    print(x, m) # выводим искомые значения на экран  
    c += 1 # увеличиваем счётчик на 1  
if c == 5: # Если количество выведенных строк равно 5  
    break # останавливаем перебор
```

Ответ:

```
8007027164 2001756793  
8007029812 5670703  
8007031342 4003515673  
8007032790 29655679  
8007066012 1956763
```

Задача №26

Задача 26.1 (Дальний восток)

Входной файл содержит информацию о заявках граждан, обращающихся во многофункциональный центр (МФЦ) в течение календарных суток. В заявке указаны время начала и время окончания приёма специалистом (в минутах от начала суток). Рабочие места специалистов МФЦ (окна) пронумерованы натуральными числами начиная с 1. Приём одного гражданина ведёт свободный специалист в окне с минимальным номером. Новый посетитель может обратиться к освободившемуся специалисту, начиная со следующей минуты после завершения приёма предыдущего. Если в момент обращения в МФЦ свободных специалистов нет, то гражданин уходит. Определите, сколько граждан сможет попасть на приём в МФЦ в течение 24 часов, и каков номер окна специалиста, который начнёт принимать посетителя последним. Если таких окон несколько, укажите наименьший номер окна.

Входные данные представлены в файле следующим образом. Первая строка входного файла содержит натуральное число K , не превышающее 1000, – количество окон в МФЦ. Во второй строке записано натуральное число N ($N \leq 10\,000$), обозначающее количество граждан. Каждая из следующих N строк содержит два натуральных числа, каждое из которых не превышает 1440: указанные в заявке время начала и время окончания приёма (в минутах от начала суток).

Запишите в ответе два числа: количество граждан, которые смогут воспользоваться услугами МФЦ, и номер окна, в котором специалист примет последнего гражданина.

Решение

В самом начале необходимо получить все данные из файла, после чего отсортировать время прихода и ухода каждого посетителя по возрастанию.

Далее создаётся список `places`, который будет содержать время ухода для K посетителей, обслуживаемых в данный момент.

Затем происходит последовательный подбор мест для каждого посетителя из списка: если время прихода текущего посетителя больше времени ухода предыдущего, значит человек может занять это место.

При записи нового посетителя необходимо увеличивать счётчик общего количества занятых мест на 1, а также записывать номер последнего занятого места (важно помнить, что нумерация идёт с 1).

```
f = open('file.txt') # открываем файл
k = int(f.readline()) # считаем число K
n = int(f.readline()) # считываем число N
a = [list(map(int, i.split())) for i in f] # считываем заявки из файла
a.sort() # сортируем заявки по возрастанию времени начала и конце

places = [-1 for _ in range(k)] # создаём k мест, которые можно занять
count = last_place = 0 # создаём переменные, в которые будет записан ответ
```

```
for start, end in a: # проверяем время прихода и ухода каждого посетителя
    for j in range(k): # подбираем подходящее место
        if start > places[j]: # если время прихода текущего посетителя больше
                                # времени ухода предыдущего,
            places[j] = end # занимаем данное место, присвоив ему время ухода
                                # текущего посетителя
            count += 1 # увеличиваем счётчик количества посетителей, которые
                        # смогли прийти
            last_place = j + 1 # записываем номер последнего место (помним,
                                # что нумерация начинается с 1)
        break # останавливаем подбор подходящих мест

print(count, last_place) # выводим ответ на экран
```

Ответ: 344 53

Задача 26.2 (Дальний восток)

В центр обработки информации приходят запросы на сервер, имеющий ограниченный запас памяти. Для каждого запроса дана время регистрации, идентификатор клиентского устройства и объём данных. Если в какой-то момент приходит информация, а свободного объёма памяти сервера не хватает для сохранения этой информации, сервер делает резервную копию и отправляет её в облако, после чего память сервера обнуляется, новая информация добавляется на сервер.

Входные данные

Первая строка входного файла содержит два натуральных числа: N – количество строк, K – вместимость специального раздела памяти сервера в Кб. Каждая из следующих N строк содержит информацию об одном выполненном запросе: время регистрации в формате ЧЧ:ММ:СС и два натуральных числа: – идентификатор клиентского устройства, S – объём данных запроса в Кб.

Выходные данные

Два целых положительных числа: сначала идентификатор клиентского устройства, который отправил наибольший объём запросов, а затем максимальный суммарный объём двух резервных копий, которые отправлялись в облако до 12 часов дня.

В задаче есть один сервер вместительностью K , который принимает запросы. Если в какой-то момент объём запроса становится больше вместительности K , то накопленный на сервере объём переходит в облако и память обнуляется, сохраняя информацию только о новом клиенте.

Первым делом нужно разобраться со входными данными – время дано в часах, минутах и секундах. Далее нужно будет отсортировать запросы по времени их отправки, поэтому удобно преобразовать эти три числа к одному. Несложно выразить всё через секунды по формуле: часы $\cdot 3600$ + минуты $\cdot 60$ + секунды. Тогда получим для каждого запроса сколько прошло секунд с начала суток, далее отсортируем по этому числу.

Следующим шагом создаём списки: `volumes` для хранения суммарного отправленного объёма каждым устройством (индекс списка соответствует `id`), `reserves` для хранения объёмов резервных

копий до 12 часов дня. `cur_volume` будет хранить, сколько на текущий момент занято памяти на сервере.

Затем запускаем перебор запросов, в котором накапливаем объём на сервере. Если новые данные не влезают на сервер, сбрасываем объём на сервере, перед этим проверив, была ли отправка до 12 часов дня. Если влезают, то просто увеличиваем суммарный объём. Последним действием в этом цикле увеличиваем для текущего запроса `r` его суммарный объём на `r[2]`.

Наконец, осталось посчитать ответ. Проходимся по всем суммарным объёмам из `volumes`, где, напомним, индексом является `id` запроса, а значением - сколько было отправлено. Ищем максимальное значение и соответствующий для него индекс.

В завершении сортируем по возрастанию список `reserves` с резервными копиями, выводим найденный максимальный индекс и сумму двух наибольших резервных копий

Решение

```
f = open('26.txt')
n, k = map(int, f.readline().split())

# создаём список для хранения запросов в виде
# [время в секундах от начала суток,
# id устройства,
# объём запроса]
requests = []
for i in range(n):
    # считываем строку из 3 чисел и разделяем по пробелу
    s = f.readline().split()
    time, cid, volume = s[0], int(s[1]), int(s[2])
    time = time.split(':') # разделяем время по двоеточию
    # получаем часы, минуты и секунды в числовом виде
    hours = int(time[0])
    minutes = int(time[1])
    seconds = int(time[2])
    # переводим в секунды от начала суток,
    # чтобы потом легче было сортировать
    time = hours * 3600 + minutes * 60 + seconds
    requests.append([time, cid, volume])

# сортируем по времени от начала суток
requests.sort()
# создаём список, в котором индексом будет id
# устройства, а значением - суммарный отправленный объём
volumes = [0 for _ in range(50_000)] # 50_000 взято случайно,
# если будет ошибка индексации, необходимо увеличить
reserves = [] # храним резервные копии, отправленные до 12
cur_volume = 0 # текущий занятый объём на сервере
for r in requests:
    # проверяем, влезает ли новый объём r[2]
```

```
if r[2] + cur_volume > k:
    # если не влезает,
    # делаем резервную копию и сбрасываем сервер
    if r[0] < 3600 * 12:
        # если резервная копия отправилась до 12 дня,
        # сохраняем её объём
        reserves.append(cur_volume)
    # сбрасываем сервер
    cur_volume = r[2]
else:
    # если влезает, увеличиваем суммарный объём
    cur_volume = r[2] + cur_volume
# увеличиваем суммарный объём запросов для нашего устройства
volumes[r[1]] += r[2]

max_id = 0
max_volume = 0
for i in range(len(volumes)):
    # проходимся по суммарным объёмам для каждого устройства,
    # ищем максимальный объём и соответствующий для него id
    if volumes[i] > max_volume:
        max_volume = volumes[i]
        max_id = i
# сортируем по возрастанию,
# чтобы взять два наибольших суммарных объёма
reserves.sort()
print(max_id, reserves[-1] + reserves[-2])
```

Ответ: 5433 99961

Задача №27

Задача 27.1 (Дальний восток)

Учёный решил провести кластеризацию некоторого множества звёзд по их расположению на карте звёздного неба. Кластер звёзд – это набор звёзд (точек) на графике. Каждый кластер имеет форму прямоугольника, причём эти прямоугольники между собой не пересекаются. Центр кластера – это одна из звёзд на графике, сумма расстояний от которой до всех остальных звёзд кластера минимальна.

В файле А хранятся данные о звёздах 2-х кластеров, в файле Б хранятся данные о звёздах 3-х кластеров. Для каждой звезды дана характеристика: тип цвета, тип светимости и её размер в соответствии с таблицей.

Обозначение	Цвет	Обозначение	Размер
G	белый	I	сверхгигант
J	зелёный	II	яркий гигант
L	синий	III	гигант
N	оранжевый	IV	субгигант
Y	красный	V	карлик
S	голубой	VI	субкарлик
Z	жёлтый	VII	квазар

Полученные значения записаны в характеристике слитно: обозначение цвета, светимость (обозначается цифрой 1...9) и обозначение размера (римские цифры).

Расстояние между двумя точками $A(x_1, y_1)$ и $B(x_2, y_2)$ вычисляется по формуле:

$$\rho(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Даны два входных файла (файл А и файл Б). Для файла А определите координаты центра каждого кластера, затем найдите два числа: A_1 – минимальное расстояние от центра кластера с наименьшим количеством точек до красного гиганта, и A_2 – максимальное расстояние от центра кластера с наименьшим количеством точек до красного гиганта.

Для файла Б определите координаты центра каждого кластера, затем найдите два числа: B_1 – минимальное расстояние между двумя различными жёлтыми карликами, расположенными в одном и том же кластере, и B_2 – расстояние между центрами кластеров с минимальным и максимальным количеством жёлтых карликов.

В ответе запишите четыре числа: в первой строке – целую часть произведения $A_1 \times 10000$, затем целую часть произведения $A_2 \times 10000$; во второй строке – сначала целую часть произведения $B_1 \times 10000$, затем целую часть произведения $B_2 \times 10000$.

Решение

Решение пункта А:

```
# Импортируем функцию расстояния
from math import dist
```

```
# реализация алгоритма DBSCAN
def dbscan(a, r):
    cl = [] # инициализируем список для хранения кластеров
    while a: # пока есть элементы в входном массиве 'a'
        cl.append([a.pop(0)])
        for i in cl[-1]: # проходим по элементам последнего кластера
            for j in a[:]:
                if dist(i[:2], j[:2]) <= r: # вычисляем расстояние между звездами
                    cl[-1].append(j) # добавляем 'j' в текущий кластер
                    a.remove(j) # удаляем 'j' из списка 'a'
    return cl # возвращаем список кластеров

# Открываем файл
file = open("27_A.txt")

# Список звёзд
stars = []

# список красных гигантов
red_giants = []

# Проходим по строкам файла
for line in file:
    # Заменяем запятые на точки и разбиваем по пробелам
    x, y, c = line.replace(",", ".").split()
    # Преобразуем координаты в числа
    x, y = float(x), float(y)
    # Добавляем звезду
    stars.append([x, y, c])

# создаем копию списка stars
stars_copy = [star for star in stars]

# Кластеры
clusters = dbscan(stars, 0.5)

# Центры кластеров
centers = []

# Проходим по кластерам
for cl in clusters:
    # Ищем минимальное суммарное расстояние
    min_dist = 10**100
```

```
# Перебираем кандидатов на центр
for cent in cl:
    # Суммарное расстояние от кандидата до остальных звёзд
    sum_dist = 0

    # Перебираем остальные звезды кластера
    for star in cl:
        # Суммируем расстояния
        sum_dist += dist(cent[:2], star[:2])

    # Обновляем минимальное суммарное расстояние и центр
    if sum_dist < min_dist:
        min_dist = sum_dist
        center = cent

# Добавляем центр в список
centers.append(center)

# Размеры кластеров
counts = [len(cl) for cl in clusters]
# Кластер с наименьшим количеством точек
min_cluster_index = counts.index(min(counts))
# Центр этого кластера
center = centers[min_cluster_index]

# Минимальное расстояние до красного гиганта
A1 = 10**100
# Максимальное расстояние до красного гиганта
A2 = -1
# Перебираем все звёзды
for star in stars_copy:
    # Смотрим, что звезда - красный гигант
    if star[2][0] == "Y" and star[2][2:] == "III":
        # Считаем расстояние до центра
        d = dist(center[:2], star[:2])
        # Обновляем минимальное расстояние
        if d < A1:
            A1 = d
        # Обновляем максимальное расстояние
        if d > A2:
            A2 = d

# Выводим ответ
print(int(A1 * 10_000), int(A2 * 10_000))
```


Решение пункта Б:

```
# Импортируем функцию расстояния
from math import dist

# реализация алгоритма DBSCAN
def dbscan(a, r):
    cl = [] # инициализируем список для хранения кластеров
    while a: # пока есть элементы в входном массиве 'a'
        cl.append([a.pop(0)])
        for i in cl[-1]: # проходим по элементам последнего кластера
            for j in a[:]:
                if dist(i[:2], j[:2]) <= r: # вычисляем расстояние между звездами
                    cl[-1].append(j) # добавляем 'j' в текущий кластер
                    a.remove(j) # удаляем 'j' из списка 'a'
    return cl # возвращаем список кластеров

# Открываем файл
file = open("27_B.txt")

# Список звёзд
stars = []

n = 3 # кол-во кластеров

# Проходим по строкам файла
for line in file:
    # Заменяем запятые на точки и разбиваем по пробелам
    x, y, c = line.replace(",", ".").split()
    # Преобразуем координаты в числа
    x, y = float(x), float(y)
    # Добавляем звезду
    stars.append([x, y, c])

# Кластеры
clusters = dbscan(stars, 0.5)

# Центры кластеров
centers = []

# Проходим по кластерам
for cl in clusters:
    # Ищем минимальное суммарное расстояние
```

```
min_dist = 10**100

# Перебираем кандидатов на центр
for cent in cl:
    # Суммарное расстояние от кандидата до остальных звёзд
    sum_dist = 0

    # Перебираем остальные звезды кластера
    for star in cl:
        # Суммируем расстояния
        sum_dist += dist(cent[:2], star[:2])

    # Обновляем минимальное суммарное расстояние и центр
    if sum_dist < min_dist:
        min_dist = sum_dist
        center = cent

# Добавляем центр в список
centers.append(center)

# список желтых карликов для каждого кластера
yellow_dwarfs = [[] for _ in range(n)]

# находим желтых карликов в каждом кластере

# Перебираем кластеры
for i in range(n):
    # Перебираем звёзды кластера
    for star in clusters[i]:
        # Смотрим, что звезда - желтый карлик
        if star[2][0] == "Z" and star[2][2:] == "V":
            # добавляем карлика
            yellow_dwarfs[i].append(star)

# количество желтых карликов в каждом кластере
counts = [len(i) for i in yellow_dwarfs]

# B1 - минимальное расстояние между двумя желтыми карликами в одном
# кластере
B1 = 10**10

# Перебираем кластеры
for i in range(n):
    # Перебираем все пары желтых карликов
```

```
for j in range(len(yellow_dwarfs[i])):
    for k in range(j + 1, len(yellow_dwarfs[i])):
        # Ищем минимальное расстояние
        B1 = min(B1, dist(yellow_dwarfs[i][j][:2], yellow_dwarfs[i][k][:2]))

# Индексы кластеров с максимальным и минимальным количеством желтых
# карликов
max_cluster_index = counts.index(max(counts))
min_cluster_index = counts.index(min(counts))

# B2 - расстояние между их центрами
B2 = dist(centers[max_cluster_index][:2], centers[min_cluster_index][:2])

# Выводим ответ
print(int(B1 * 10_000), int(B2 * 10_000))
```

Ответ:

4940 74302
2199 189261

Задача 27.2 (Дальний восток)

Фрагмент звёздного неба спроецирован на плоскость с декартовой системой координат. Учёный решил провести кластеризацию полученных точек, являющихся изображениями звёзд, то есть разбить их множество на N непересекающихся непустых подмножеств (кластеров) таких, что точки каждого подмножества лежат внутри прямоугольника со сторонами длиной H и W , причём эти прямоугольники не пересекаются между собой. Стороны прямоугольников не обязательно параллельны координатным осям. Гарантируется, что такое разбиение существует и единственно для заданных размеров прямоугольников.

Будем называть центром кластера точку этого кластера, сумма расстояний от которой до всех остальных точек кластера минимальна. Для каждого кластера гарантируется единственность центра.

Расстояние между двумя точками на плоскости $A(x_1, y_1)$ и $B(x_2, y_2)$ вычисляется по формуле евклидова расстояния:

$$\rho(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

В файле A хранятся данные о звёздах двух кластеров, где $H = 6$, $W = 4,5$ для каждого кластера. В каждой строке записана информация о расположении на карте одной звезды: сначала координата x , затем координата y . Значения даны в условных единицах. Известно, что количество звёзд не превышает 1000.

Известно, что в файле имеются координаты ровно трёх «лишних» точек, представляющих аномалии, которые возникли в результате помех при передаче данных. Эти точки не

относятся ни к одному из кластеров, их учитывать не нужно.

Для файла определите координаты центра каждого кластера, затем найдите два числа: P_x – расстояние по оси абсцисс между центрами кластеров, и P_y – расстояние по оси ординат между центрами кластеров. Гарантируется, что во всех кластерах количество точек различно.

В ответе запишите два числа: сначала целую часть $P_x \times 10000$ и целую часть произведения $P_y \times 10000$ для файла A.

Решение

#Импортируем функцию расстояния

```
from math import dist
```

реализация алгоритма DBSCAN

```
def dbscan(a, r):
```

```
    cl = [] # инициализируем список для хранения кластеров
```

```
    while a: # пока есть элементы в входном массиве 'a'
```

```
        cl.append([a.pop(0)])
```

```
        for i in cl[-1]: # проходим по элементам последнего кластера
```

```
            for j in a[:]:
```

```
                if dist(i, j) <= r: # вычисляем расстояние между звездами
```

```
                    cl[-1].append(j) # добавляем 'j' в текущий кластер
```

```
                    a.remove(j) # удаляем 'j' из списка 'a'
```

```
    return cl #возвращаем список кластеров
```

Открываем файл

```
file = open("27_A.txt")
```

Список звёзд

```
stars = []
```

Проходим по строкам файла

```
for line in file:
```

```
    star = list(map(float, line.replace(',', ' ').split()))
```

```
    # Добавляем звезду
```

```
    stars.append(star)
```

Кластеры

```
clusters = dbscan(stars, 0.5)
```

избавляемся от аномалий

```
cl_total = [cluster for cluster in clusters if len(cluster) > 10]
```

Центры кластеров

```
centers = []

# Проходим по кластерам
for cl in cl_total:
    # Ищем минимальное суммарное расстояние
    min_dist = 10 ** 100

    # Перебираем кандидатов на центр
    for cent in cl:
        # Суммарное расстояние от кандидата до остальных звёзд
        sum_dist = 0

        # Перебираем остальные звезды кластера
        for star in cl:
            # Суммируем расстояния
            sum_dist += dist(cent[:2], star[:2])

        # Обновляем минимальное суммарное расстояние и центр
        if sum_dist < min_dist:
            min_dist = sum_dist
            center = cent

    # Добавляем центр в список
    centers.append(center)

# расстояние по оси абсцисс между центрами кластеров
px = abs(centers[0][0] - centers[1][0])

# расстояние по оси ординат между центрами кластеров
py = abs(centers[0][1] - centers[1][1])

# Выводим ответ
print(int(px * 10_000), int(py * 10_000))
```

Ответ:

28147 126127

Задача 27.3 (Дальний восток)

Фрагмент звёздного неба спроецирован на плоскость с декартовой системой координат. Учёный решил провести кластеризацию полученных точек, являющихся изображениями звёзд, то есть разбить их множество на непересекающихся непустых подмножеств (кластеров), таких что точки каждого подмножества лежат внутри прямоугольника со сторонами длиной H и W , причём эти прямоугольники между собой не пересекаются. Стороны прямоугольни-

ков не обязательно параллельны координатным осям. Гарантируется, что такое разбиение существует и единственно.

Для каждой звезды дана характеристика: тип цвета, тип светимости и её размер в соответствии с таблицей.

Обозначение	Цвет	Обозначение	Размер
G	белый	I	сверхгигант
J	зелёный	II	яркий гигант
L	синий	III	гигант
N	оранжевый	IV	субгигант
Y	красный	V	карлик
S	голубой	VI	субкарлик
Z	жёлтый	VII	белый карлик

Полученные значения записаны в характеристике слитно: обозначение цвета, светимость (арабская цифра) и обозначение размера.

Будем называть центром кластера точку этого кластера, сумма расстояний от которой до всех остальных точек кластера минимальна (центроид).

В файле А хранятся данные о звёздах двух кластеров, где $H = 6,5$, $W = 4,5$ для каждого кластера. В каждой строке записана координата x , затем координата y , а затем её характеристика. В файле Б хранятся аналогичные данные о звёздах трёх кластеров.

Определите координаты центра каждого кластера для файла А, затем найдите два числа: A_1 – абсцисса центра кластера с наименьшим количеством звёзд светимости 2, и A_2 – ордината центра кластера с наибольшим количеством звёзд светимости 2.

Определите координаты центра каждого кластера для файла Б, затем найдите два числа: B_1 – расстояние между центрами кластеров с минимальным и максимальным количеством красных звёзд, и B_2 – наибольшее расстояние между центром кластера и красной звездой из этого же кластера.

В ответе укажите сначала целые части произведений $A_1 \times 10000$ и $A_2 \times 10000$, а во второй строке – $B_1 \times 10000$ и $B_2 \times 10000$.

Решение

Решение пункта А:

```
# Импортируем функцию расстояния
from math import dist

# реализация алгоритма DBSCAN
def dbscan(a, r):
    cl = [] # инициализируем список для хранения кластеров
    while a: # пока есть элементы в входном массиве 'a'
        cl.append([a.pop(0)])
        for i in cl[-1]: # проходим по элементам последнего кластера
            for j in a[:]:
```

```
        if dist(i[:2], j[:2]) <= r: # вычисляем расстояние между звездами
            cl[-1].append(j) # добавляем 'j' в текущий кластер
            a.remove(j) # удаляем 'j' из списка 'a'
    return cl # возвращаем список кластеров

# Открываем файл
file = open("27_A.txt")

# Список звёзд
stars = []

# список красных гигантов
red_giants = []

# Проходим по строкам файла
for line in file:
    # Заменяем запятые на точки и разбиваем по пробелам
    x, y, c = line.replace(",", ".").split()
    # Преобразуем координаты в числа
    x, y = float(x), float(y)
    # Добавляем звезду
    stars.append([x, y, c])

# Кластеры
clusters = dbscan(stars, 0.5)

# Центры кластеров
centers = []

# Проходим по кластерам
for cl in clusters:
    # Ищем минимальное суммарное расстояние
    min_dist = 10 ** 100

    # Перебираем кандидатов на центр
    for cent in cl:
        # Суммарное расстояние от кандидата до остальных звёзд
        sum_dist = 0

        # Перебираем остальные звезды кластера
        for star in cl:
            # Суммируем расстояния
            sum_dist += dist(cent[:2], star[:2])
```

```
# Обновляем минимальное суммарное расстояние и центр
if sum_dist < min_dist:
    min_dist = sum_dist
    center = cent

# Добавляем центр в список
centers.append(center)

# Считаем количество звезд светимости 2 в каждом кластере
counts = []
# Перебираем кластеры
for cl in clusters:
    # Количество звезд светимости 2 в текущем кластере
    count = 0
    # Перебираем звёзды кластера
    for star in cl:
        # Смотрим, что светимость 2
        if star[2][1] == "2":
            # Увеличиваем количество
            count += 1
    # Добавляем количество
    counts.append(count)

# Индексы кластеров с максимальным и минимальным количеством звезд светимости 2
max_cluster_index = counts.index(max(counts))
min_cluster_index = counts.index(min(counts))

# абсцисса центра кластера с наименьшим кол-вом звезд светимости 2
A1 = centers[min_cluster_index][0]
# ордината центра кластера с наибольшим кол-вом звезд светимости 2
A2 = centers[max_cluster_index][1]

# Выводим ответ
print(int(A1 * 10_000), int(A2 * 10_000))
```

Решение пункта Б:

```
# Импортируем функцию расстояния
from math import dist

# реализация алгоритма DBSCAN
def dbscan(a, r):
```

```
cl = [] # инициализируем список для хранения кластеров
while a: # пока есть элементы в входном массиве 'a'
    cl.append([a.pop(0)])
    for i in cl[-1]: # проходим по элементам последнего кластера
        for j in a[:]:
            if dist(i[:2], j[:2]) <= r: # вычисляем расстояние между звездами
                cl[-1].append(j) # добавляем 'j' в текущий кластер
                a.remove(j) # удаляем 'j' из списка 'a'
return cl # возвращаем список кластеров

# Открываем файл
file = open("27_B.txt")

n = 3 # кол-во кластеров

# Список звёзд
stars = []

# Проходим по строкам файла
for line in file:
    # Заменяем запятые на точки и разбиваем по пробелам
    x, y, c = line.replace(",", ".").split()
    # Преобразуем координаты в числа
    x, y = float(x), float(y)
    # Добавляем звезду
    stars.append([x, y, c])

# Кластеры
clusters = dbscan(stars, 0.5)

# Центры кластеров
centers = []

# Проходим по кластерам
for cl in clusters:
    # Ищем минимальное суммарное расстояние
    min_dist = 10 ** 100

    # Перебираем кандидатов на центр
    for cent in cl:
        # Суммарное расстояние от кандидата до остальных звёзд
        sum_dist = 0
```

```
# Перебираем остальные звезды кластера
for star in cl:
    # Суммируем расстояния
    sum_dist += dist(cent[:2], star[:2])

# Обновляем минимальное суммарное расстояние и центр
if sum_dist < min_dist:
    min_dist = sum_dist
    center = cent

# Добавляем центр в список
centers.append(center)

# список красных звезд для каждого кластера
red_stars = [[] for _ in range(n)]

# находим красные звезды в каждом кластере

# Перебираем кластеры
for i in range(n):
    # Перебираем звёзды кластера
    for star in clusters[i]:
        # Смотрим, что звезда красная
        if star[2][0] == "Y":
            # добавляем звезду
            red_stars[i].append(star)

# количество красных звезд в каждом кластере
counts = [len(i) for i in red_stars]

# Индексы кластеров с максимальным и минимальным количеством красных звезд
max_cluster_index = counts.index(max(counts))
min_cluster_index = counts.index(min(counts))

# расстояние между центрами кластеров с минимальным и максимальным кол-вом
# красных звезд
B1 = dist(centers[min_cluster_index][:2], centers[max_cluster_index][:2])

# наибольшее расстояние между центром кластера и красной звездой из этого кластера
B2 = -1

# Перебираем кластеры
```



```

for i in range(n):
    # Перебираем все красные звезды
    for red_star in red_stars[i]:
        # Ищем максимальное расстояние
        B2 = max(B2, dist(centers[i][:2], red_star[:2]))
# Выводим ответ
print(int(B1 * 10_000), int(B2 * 10_000))

```

Ответ:

70391 61225
125591 20077

Задача 27.4 (Центр)

Фрагмент звёздного неба спроецирован на плоскость с декартовой системой координат. Учёный решил провести кластеризацию полученных точек, являющихся изображениями звёзд, то есть разбить их множество на непересекающихся непустых подмножеств (кластеров), таких что точки каждого подмножества лежат внутри прямоугольника со сторонами длиной H и W , причём эти прямоугольники между собой не пересекаются. Стороны прямоугольников не обязательно параллельны координатным осям. Гарантируется, что такое разбиение существует и единственно.

Для каждой звезды дана характеристика: тип цвета, тип светимости и её размер в соответствии с таблицей.

Обозначение	Цвет	Обозначение	Размер
G	белый	I	сверхгигант
J	зелёный	II	яркий гигант
L	синий	III	гигант
N	оранжевый	IV	субгигант
Y	красный	V	карлик
S	голубой	VI	субкарлик
Z	жёлтый	VII	белый карлик

Полученные значения записаны в характеристике слитно: обозначение цвета, светимость (арабская цифра) и обозначение размера.

Будем называть центром кластера точку этого кластера, сумма расстояний от которой до всех остальных точек кластера минимальна (центроид).

В файле A хранятся данные о звёздах двух кластеров, где $H = 6,5$, $W = 4,5$ для каждого кластера. В каждой строке записана координата x , затем координата y , а затем её характеристика.

Определите координаты центра каждого кластера для файла A , затем найдите два числа: A_x – абсцисса ближайшего к центроиду желтого карлика, кластера с наибольшим количеством звёзд, и A_y – ордината ближайшего к центроиду желтого карлика, кластера с наибольшим количеством звёзд.

В ответе укажите целые части произведений $A_1 \times 10000$ и $A_2 \times 10000$.

Решение

Решение пункта А:

```
# Импортируем функцию расстояния
from math import dist

# Открываем файл
file = open("27_1_A.txt")
# Список кластеров
clusters = [[] for _ in range(2)]

# Проходим по строкам файла
for line in file:
    # Заменяем запятые на точки и разбиваем по пробелам
    x, y, c = line.replace(",", ".").split()
    # Преобразуем координаты в числа
    x, y = float(x), float(y)

    # Делим на кластеры по прямой y = 10
    if y < 10:
        clusters[0].append([x, y, c])
    else:
        clusters[1].append([x, y, c])

# Центры кластеров
centers = []
# Проходим по кластерам
for cl in clusters:
    # Ищем минимальное суммарное расстояние
    min_dist = 10 ** 100
    # Перебираем кандидатов на центр
    for cent in cl:
        # Суммарное расстояние от кандидата до остальных звёзд
        sum_dist = 0
        # Перебираем остальные звезды кластера
        for star in cl:
            # Суммируем расстояния
            sum_dist += dist(cent[:2], star[:2])
        # Обновляем минимальное суммарное расстояние и центр
        if sum_dist < min_dist:
            min_dist = sum_dist
            center = cent
    # Добавляем центр в список
    centers.append(center)
```

```
# Размеры кластеров
counts = [len(cl) for cl in clusters]
# Кластер с наибольшим количеством точек
max_cluster_index = counts.index(max(counts))
# Центр этого кластера
center = centers[max_cluster_index]

# Минимальное расстояние до центра
min_d = 10 ** 10
# Координаты подходящего желтого карлика
ans = None
# Перебираем все звёзды
for star in clusters[max_cluster_index]:
    # Смотрим, что звезда - желтый карлик
    if star[2][0] == "Z" and star[2][2:] == "V":
        # Считаем расстояние до центра
        d = dist(center[:2], star[:2])
        # Если оно оказалось меньше текущего минимума,
        # то обновляем его и сохраняем координаты карлика
        if d < min_d:
            min_d = d
            ans = star

# Выводим ответ
print(int(ans[0] * 10_000), int(ans[1] * 10_000))
```

Ответ: 40410 62433